

Event Algebras and Applications to Cryptography

Konstantin Gegier¹ and Ueli Maurer¹

Department of Computer Science
ETH Zurich, Switzerland
`{konstantin.gegier,maurer}@inf.ethz.ch`

Abstract. Discrete-step models are ubiquitous in many disciplines, in particular in Computer Science (e.g., computer systems, distributed and cryptographic protocols, etc.). The space of possible developments forms a tree (or forest) whose branches correspond to the possible discrete steps. Events are monotone predicates (or downsets) on the tree. Examples of events are input, output, forgery, consistency failure, or authentication failure events. Statements of interest about events are, for example, that a certain (“bad”) event can not occur.

This paper introduces the concept of event algebras, a specific type of bounded distributive lattice $(E; \preceq, \wedge, \vee, \dot{-}, \top, \perp)$ with an additional operation $\dot{-}$, and shows that the event algebra axioms capture exactly and minimally the abstract mathematical structure of events in discrete-step models. An event inequality $e \preceq f$ can be read as “*event e can not occur without event f (having occurred).*”

The most basic type of event algebra theorems, which are the scope of this paper, are inequalities between algebraic terms, for example, $a \dot{-} b \preceq (a \dot{-} c) \vee (c \dot{-} b)$, which hold universally, i.e., for any choice of the variables and for any event algebra. It is demonstrated that many fundamental statements in cryptography and other fields are direct implications of specific such universal event inequalities. For example, in a nutshell, the theorem stating the security of the well-known Hash-then-Sign paradigm is, in abstract form, the event inequality $e \preceq f \vee g$, where e is the forgery event of the outer signature scheme, f is the forgery event of the inner signature scheme, and g is the (hash) collision event.

The abstract algebraic treatment comes with the usual advantages: (1) generality, i.e., independence of modeling aspects such as computational and communication models or complexity and efficiency notions, (2) natural theorem composition, and (3) purely algebraic, minimal, and even formal proofs (here done in the Lean theorem prover).

1 Introduction

1.1 Motivation

An important principle in mathematically rigorous scientific disciplines is to identify and capture the relevant mathematical structures by abstract algebraic

structures in terms of operations, relations, and axioms. Examples are partial order relations, groups, rings, fields, lattices, and vector spaces. Such algebraic structures can then be used, across disciplines, in all contexts where they are the adequate abstraction. They can also be concretized or instantiated to capture specific types or examples of such algebraic structures, for example, the integers as an instantiation of rings or the complex numbers as an instantiation of fields. For example, the concept of a vector space, which is defined abstractly by axioms, is of core relevance in many disciplines, ranging from physics to signal processing to machine learning to coding theory.

Abstraction means to identify the relevant aspects of a problem or context, while eliminating irrelevant details from consideration. The abstract algebraic treatment comes with many advantages: (1) generality of results, i.e., independence of modeling aspects and artifacts such as computational and communication models or complexity and efficiency notions, (2) natural theorem composition, (3) purely algebraic, minimal, and even formal proofs (here done in the Lean theorem prover), and typically also didactic suitability.¹

In this paper, we take a clean-slate approach to an abstract general treatment of discrete-step models² and events which are ubiquitous in many disciplines, in particular in Computer Science, and we illustrate the approach for an important type of statements in cryptography, called *universal event inequalities*.

1.2 Events and Discrete-step Models

Events are a universal concept appearing in most areas of reasoning. An event is *monotone* in the sense that once it has occurred, it cannot “unoccur”. One can think of an observer for the event who raises a flag as soon as the event’s condition is satisfied, and the flag is never lowered.

For example, when reasoning about everyday life, one can consider the event $\text{Birth}(x)$ that person x is born or the event $\text{Death}(x)$ that person x dies. In an economic context, one can consider trigger events for asset prices, such as the event $\text{Gold}(p)$ that the gold price reaches a certain price level p (for the first time). Most importantly, in Computer Science, events are of paramount importance, for example, the event that a system reaches a certain state, the event that a safety property is violated, the event that an algorithm run on a computer outputs the solution to a problem, the event that in a distributed protocol two parties output inconsistent values, etc.

The notion of events requires that one consider, at least implicitly, some type of discrete-step model which, in the most general sense, can be understood as the definition of the set of possible evolutions (also called developments or

¹ One can, of course, apply such abstract statements on a more concrete (less abstract) level if one defines a concrete model and verifies that it satisfies the abstract axioms.

² The concept of discrete-step models includes computational models (like a Turing machine or a higher-level algorithm description), distributed protocols (like broadcast, consensus, or blockchain protocols), and cryptographic protocols or games (like signature-forgery games or the Hash-then-Sign paradigm).

executions) in discrete steps. What is common to all discrete-step models is that the space of all possible developments forms a tree (or a forest)³ whose root(s) correspond to the possible starting point(s) and whose branches correspond to the possible discrete steps. A path from a root to a node is a particular possible evolution, while the tree or forest captures all possible evolutions (only one of which may actually occur).

Events are monotone predicates on the set of nodes of the tree (see Section 2.1). One can define natural operations on events e and f : $e \wedge f$ (both have occurred, “ e and f ”), $e \vee f$ (at least one has occurred, “ e or f ”), and the event $e \dot{\vee} f$ (e has occurred without f having occurred, “ e without f ”). There is also a natural partial order relation on events, denoted $\preceq$, where $e \preceq f$ is read as “ e cannot occur without f having occurred” or “ e implies f ”.

This leads to the concept of *event algebras* $(E; \preceq, \wedge, \vee, \dot{\vee}, \top, \perp)$ with the above operations and special elements $\top$ (the “certain” event) and $\perp$ (the “impossible” event). It is proved that, for a natural choice of axioms, event algebras capture exactly the laws of events in any discrete-step model. This defines *event theory*, the theory of true statements about event algebras.

The most basic type of event algebra theorems, which are the scope of this paper, are *universal event inequalities (UEIs)* between algebraic expressions that hold for any choice of the variables (and for any event algebra). A simple (and not yet very useful) example is $a \dot{\vee} b \preceq (a \dot{\vee} c) \vee (c \dot{\vee} b)$.

It may be surprising that such UEIs are the abstract formulations of many types of guarantees which are typically phrased for very particular discrete-step models (e.g., for a particular computational model) and explained in a particular narrative (e.g., involving the concept of an adversary).

For example, the security of the well-known Hash-then-Sign paradigm can be derived from an UEI as a single inequality

$$e \preceq f \vee g,$$

for well-chosen event-algebra expressions e , f , and g , where e is the forgery event of the outer signature scheme, f is the forgery event of the inner signature scheme, and g is the hash collision event (see Section 4.5).

1.3 Contributions

This paper makes two main contributions with many implications.

First, we introduce the *abstract event-algebraic framework* for stating and proving theorems that apply to any discrete-step model. It is proved that the event-algebra axioms exactly and minimally capture the structure of monotone predicates on a forest (Theorems 1 and 2), the common aspect of discrete-step models. Since (probability-theoretic) σ -algebras are a special case of event algebras, every inequality, for example, $e \preceq f \vee g$, immediately implies the corresponding probability-theoretic statement, for example, $\Pr[e] \leq \Pr[f] + \Pr[g]$.

³ This includes models like automata or transition systems corresponding to a directed acyclic graph (where paths can merge) by considering the sets of possible paths.

The second main contribution of the paper is the application of the event-algebraic approach to illustrate that many fundamental statements in cryptography can be stated, in a more general form, as *universal event inequalities (UEI)*, a specific type of event algebra theorem. The examples range from classical statements like the security of the Hash-then-Sign paradigm and different notions of unforgeability to the construction of strongly unforgeable signature schemes from unforgeable one-time signatures of [1], or the recent “BoP-1” combiner construction for signature schemes [2].

The approach is not specific to cryptography and can be used in all disciplines involving discrete-step models, hence potentially spanning much of computer science, including distributed protocols, algorithm theory, complexity, and even program correctness.⁴

The benefits of using the framework include the following aspects:

Universality/generality. UEIs are independent of a concrete model, hence hold universally for any concrete (discrete-step) model, which allows one to avoid making specific assumptions about the model. Specifically, there is no need (and no reason) to assume concrete computational models (e.g., Turing machines), complexity classes, asymptotics (e.g., polynomial time), etc.⁵ UEIs hold, in particular, for real-world executions, independent of an adversary model.⁶ Moreover, the probability-theoretic consequences (like $\Pr[e] \leq \Pr[f] + \Pr[g]$ mentioned above) hold universally for any choice of probability measure, for example, even if the keys are not uniform or not independent of other quantities.

UEIs are pure mathematical statements, and the introduction of a narrative, with terms like forgery or authenticity, is an explicit step without mathematical meaning, but allows for an intuitive understanding and for determining which abstract theorems one should aim for. This is in contrast to much of the literature, where theorems are stated in a particular narrative. For example, *Hash-then-Sign* is a narrative giving the abstract UEI an intuitive meaning, but a different narrative for exactly the same UEI yields the security proof for *Hash-then-MAC*.⁷

In this paper, we pay careful attention to separating mathematical theorems (UEIs) from possible narratives with intuitive, but not mathematical meaning.

⁴ Actually, a large class of statements in Computer Science can be seen as statements about events in some more abstract or more concretely defined discrete-step model.

⁵ One can observe that in the literature, often when results are stated for a concrete model (e.g., Turing machines), the result statements and proofs actually do not use or refer to these concrete aspects and are, in a sense, more general. For many such statements, UEIs directly imply the corresponding reductions appearing in a traditional proof.

⁶ They even hold if the adversary or environment includes humans who cannot be modeled as algorithms.

⁷ While this may perhaps not be very surprising, it should be pointed out that in the standard game-based approaches (e.g. [3]), the two proofs for *Hash-then-Sign* and *Hash-then-MAC* are quite similar but not identical because different games are involved.

Natural theorem composition. There has been considerable research effort towards understanding composability in cryptography. We note that event UEs can seamlessly and naturally be composed. For example, the two inequalities $e \preceq f \vee g$ and $g \preceq h \vee i$ directly imply $e \preceq f \vee h \vee i$.

Clean proofs and formalization. The proofs are minimal, based on a clearly defined set of axioms, and hence well-suited for being formalized in a *formal proof system*. The feasibility of the formalization has been demonstrated by formally verifying some of our results using `mathlib` in Lean, and the formal proofs are quite short because artifacts are avoided.

The paper is also the basis for many extensions. For example, more advanced guarantees (e.g., commitment-type guarantees) can be captured by a more complex type of event-algebraic theorems, which state that (at least) one of several inequalities is satisfied.

1.4 Related Work

We follow the abstract view of cryptography proposed in [4]. Our contributions can be seen as stated on an abstract level that is a priori incomparable to the abstract theory of systems of [4]; the two theories are compatible on a more concrete level (satisfying both sets of axioms). Special cases of our notion of events have been considered in [5,6,7]. A more detailed discussion of related work is deferred to Section C, and proceeds along the following four axes:

Cryptographic frameworks: Reductionist [8,9], game-based [3,10,11], symbolic [12], and simulation-based frameworks [13,14,15,16,17] all contain, implicitly, an event algebra.

Formal verification: Many special-purpose tools for the formal verification of cryptographic protocols have been developed (see, e.g., [18] for a recent survey). The minimal axiomatic basis of our framework is well-suited for mechanization in a general-purpose theorem prover, which is illustrated by formally verifying some of our results in Lean using `mathlib` (Section F.3).

Temporal logic and properties: Our notion of events can be seen as an abstraction of safety trace properties [19,20,21], and we make the relation to temporal logic precise by defining a logic of events in Section D.

Algebra: Event algebras are the dual of Gödel algebras [22], a special class of Heyting algebras [23,24]; the connection between Gödel algebras and forests was observed in [25] and is discussed in [26], but the interpretation as events appears not to have been considered before.

Paper Outline. In Section 2, we define and discuss event algebras. Section 3 presents the type of statements of interest (UEIs) and simple examples. In Section 4, we apply our approach to further cryptographic examples and compare our approach and game-based formulations. Section 5 gives a brief outlook on future work.

2 Event Algebras

Section A contains the mathematical background for this section. We state the definition of event algebras and afterwards discuss and justify it:

Definition 1. An event algebra $(E; \preceq, \wedge, \vee, \dot{\div}, \top, \perp)$ is a bounded lattice with an additional binary operation $\dot{\div}$, satisfying the following for any $a, b, c \in E$.⁸

$$a \dot{\div} a = \perp \tag{E1}$$

$$(a \dot{\div} b) \vee a = a \tag{E2}$$

$$(a \dot{\div} b) \vee b = a \vee b \tag{E3}$$

$$(a \vee b) \dot{\div} c = (a \dot{\div} c) \vee (b \dot{\div} c) \tag{E4}$$

$$(a \dot{\div} b) \wedge (b \dot{\div} a) = \perp \tag{E5}$$

Event algebras lie strictly between bounded distributive lattices and Boolean algebras. In the following section, we motivate Definition 1 and afterwards explain the operations, trivial elements $\top$ and $\perp$, as well as the axioms.

2.1 Event Algebras Capture Discrete-Step Models

The discussion so far hints at two seemingly contradictory aspects of events. On the one hand, they form a (*static*) algebra; on the other hand, events have a *dynamic* aspect, namely some form of evolution or development in the course of which an event can occur at some point. We explain how these two aspects can be reconciled.

For any discrete-step model, when considering a starting point of the reasoning (e.g., when a system is started in the initial state), there may be different possible evolutions, where the choice of which possibility is taken may be external (e.g., depends on which inputs are given). The complete characterization of the possible developments or evolutions corresponds to a tree (see Figure 1 (left)), where each branch corresponds to a (discrete) step of the evolution (e.g., a particular input being given), and a concrete evolution corresponds to a path in this tree starting in the root. A node in the tree (or, rather, the path leading to the node) corresponds to the history at that stage of the evolution. In many settings, it is natural and useful to consider several potential starting points; this means that we consider a forest (a tuple of trees) rather than a tree.⁹

We are interested in reasoning about the entire tree of possible evolutions, not in a particular evolution (path). Events are monotone predicates on the set of nodes of the tree, where monotonicity means that if the predicate is true for a node, then it is also true for its successors in the tree. An event can be

⁸ From here on, we will use the convention that $\dot{\div}$ binds stronger than $\wedge$ and $\vee$.

⁹ For example, one can think of each possible value on a random tape as defining a different starting point. However, one could also model the choice of the random value as the first branching in a tree.

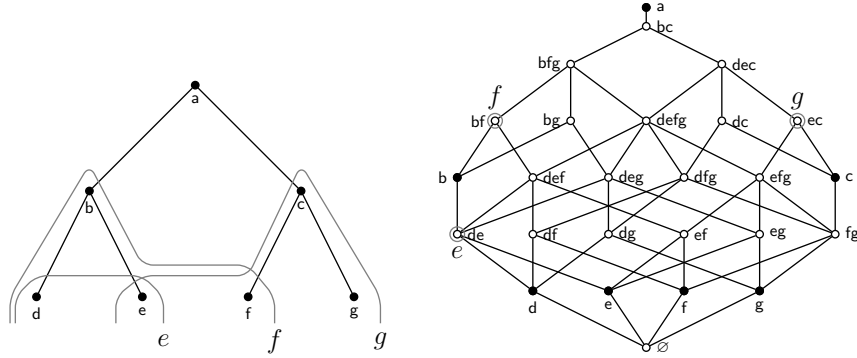


Fig. 1: Three events e, f, g in a discrete-step tree (left) and the Hasse diagram of the corresponding event algebra (right). We have $e \preceq f$, $f \not\preceq g$, and $g \not\preceq f$. The tree vertices (labeled with lowercase sans-serif letters) correspond exactly to the join-irreducibles (elements that cover exactly one element) in the algebra (filled dots). Algebra elements are labeled with the corresponding prefix-free set of tree nodes. Events e, f , and g as algebra elements are circled.

characterized by a (prefix-free) set of nodes, namely all those where the event becomes true or, alternatively stated, where the event *occurs*.^{10 11}

The following two theorems show that the event algebra axioms (Definition 1) capture abstractly, exactly, and minimally this general concept of discrete development. **Theorem 1** states that the set of monotone predicates (corresponding to down-sets) on a tree (or a forest), as described above, forms an event algebra:

Theorem 1. *For any forest poset (see Definition 9) $(P; \preceq)$, the set $(\text{Dn}(P); \subseteq, \cup, \cap, \div_P, P, \emptyset)$ is an event algebra, where $A \div_P B := (A \setminus B)^\downarrow$.*

Conversely, **Theorem 2** is a consequence of a result due to Birkhoff [27] and shows that every event algebra can naturally be interpreted as a tree (or forest) capturing the dynamic development aspect of the algebra:

Theorem 2. *For any finite¹² event algebra E , $\text{Ji}(E)$ (see Definition 10) is a forest poset and we have*

$$E \cong \text{Dn}(\text{Ji}(E)).$$

¹⁰ A special example of this natural viewpoint are (safety) trace properties, see Section B.2.

¹¹ For example, if one considers the tree of executions of a Turing machine (where branches correspond to the value read on the tape), then reaching a particular state s (e.g., the Halting state) is an event, namely the event consisting of all nodes where s is reached (for the first time).

¹² Infinite event algebras are discussed in Section B.4. We state only the finite case here for simplicity and because (sequences of) finite considerations seem to suffice for practical applications.

That is, every event algebra contains a forest (the set $\text{Ji}(E)$, a subset of E) that can be interpreted as describing possible evolutions. The lattice of down-sets (monotone predicates) on this forest is again (isomorphic to) E . Event algebras can compactly be described via the corresponding forests. The example in [Figure 1](#) (right) explains the tree contained as join-irreducibles in the corresponding algebra.

In general, events do not have complements (for example, event e in [Figure 1](#)), but every Boolean algebra, in particular every σ -algebra, is an event algebra. That is, probability-theoretic events are a special case of our notion of events. We discuss this and further examples of event algebras in more detail in [Section B.2](#).

Event algebra operations on forests. When interpreting an event algebra as describing possible evolutions, the composed events $e \wedge f$, $e \vee f$, and $e \dot{\div} f$ correspond to the events of reaching a forest node where *both* e and f occur, where *at least one of* e and f occurs, and where e occurs *without* f having occurred. $\top$ is the *certain event* that always occurs (from the start, i.e., it is the predicate that is true for all nodes in the forest). The *impossible event* $\perp$ cannot occur (i.e., it is the predicate that is false for all nodes in the forest).

The event algebra order relation $e \preceq f$ means that whenever e occurs, f must have already occurred before (or occurs in the same step where e occurs). Mathematically, we have $e \preceq f$ if for each node in the forest defining (the occurrence of) e , there is a predecessor (i.e., a node higher up or equal) in the forest among the nodes defining f . One can also say that e *implies* f or that e *cannot occur without* f . We have $\perp \preceq e$ and $e \preceq \top$ for any event e .

For two events e and f , it is possible that neither $e \preceq f$ nor $f \preceq e$ holds, i.e., that there are evolutions (paths) in the forest in which e occurs before f and other evolutions in which f occurs before e (e.g., events f and g in [Fig. 1](#)).

The axioms of [Definition 1](#) capture natural and intuitive properties of the event concept: (E1) states that no event can occur without itself. (E2) is equivalent to stating that $a \dot{\div} b$ can't occur without a . The remaining axioms can be described similarly. Note that (E5) captures that it can't be simultaneously the case that a occurred without b and vice versa, which implies that at any point in the development, the order in which the events have occurred so far is linear.

2.2 Relevant Algebraic Laws

We state the algebraic results about event algebras that we will use in [Sections 3](#) and [4](#). Proofs and further algebraic properties are deferred to [Section B.3](#).

Lemma 1. *The following hold for any elements a, b, c of an event algebra:*

- | | |
|--|---|
| (i) $a \dot{\div} b \preceq a$. | (v) $a = (a \wedge b) \vee (a \dot{\div} b)$ |
| (ii) If $a \preceq b$ then $a \dot{\div} c \preceq b \dot{\div} c$. | (vi) $(a \wedge b) \dot{\div} c = (a \dot{\div} c) \wedge (b \dot{\div} c)$ |
| (iii) $a \preceq b \vee c \iff a \dot{\div} b \preceq c$ | (vii) $a \dot{\div} (b \wedge c) = (a \dot{\div} b) \vee (a \dot{\div} c)$ |
| (iv) If $a \preceq b$ then $c \dot{\div} b \preceq c \dot{\div} a$. | (viii) $a \dot{\div} (b \vee c) = (a \dot{\div} b) \wedge (a \dot{\div} c)$ |

(i) states that event $a \dot{-} b$ can't occur without a occurring. (ii) and (iv) show that the operation $\dot{-}$ is monotone in its first and antitone in its second argument. (iii) gives a characterization of $\dot{-}$ in terms of the order relation $\preceq$ and $\vee$. (v) expresses a form of case distinction for events, stating that for any events a and b , if a occurs, either a occurred without b , or b must have occurred as well. Finally, (vi) to (viii) together with (E4) show how $\dot{-}$ distributes over $\wedge$ and $\vee$ ¹³. The distributive laws (D) hold in any event algebra¹⁴:

Lemma 2. *Every event algebra is a distributive lattice.*

Corollary 1. *The following hold for any elements a, b, c of an event algebra:*

- (i) $a \dot{-} b = \perp \iff a \preceq b$
- (ii) $a \dot{-} b \preceq ((a \wedge c) \dot{-} b) \vee (a \dot{-} c)$
- (iii) $a \dot{-} b \preceq (a \dot{-} c) \vee (c \dot{-} b)$

3 Principles of Event-Based Reasoning

3.1 Guarantees from Event Inequalities

The purpose of considering events and event algebras is to state relevant *guarantees*, for example, security or correctness guarantees, in a minimal and general form, and to prove them by clean and formal algebraic derivations.

Many guarantees are of the form that some well-defined event e (e.g., an authentication failure event, a consistency failure event, or a certain type of fault in a computer system) does *not* occur (i.e., that e never occurs, meaning $e = \perp$). The goal of a construction is to *prevent* e from occurring.¹⁵

However, it is often not possible to give such a guarantee (that e will not occur) in an absolute, unconditional sense because there are conceivable evolutions of the world where e does occur.¹⁶ But one can possibly prove the weaker statement

$$e \preceq f$$

for a well-chosen event f , i.e., that e can not occur without f . This means that, as long as f does not occur, e also can not occur. Stated differently: *If the goal is to prevent event e , this can be achieved by preventing f .* By Corollary 1.(i), we have

$$e \preceq f \iff e \dot{-} f = \perp.$$

¹³ Note the inversion of $\wedge$ and $\vee$ in (vii) and (viii) due to $\dot{-}$'s antitonicity in the second argument.

¹⁴ The proof of Lemma 2, showing that event algebras are distributive, only uses axioms (E1) - (E4). A bounded lattice satisfying these axioms is called a *dual Heyting lattice*.

¹⁵ One is also often interested in assuring that a certain event *does* occur. We briefly discuss this in Section E.1.

¹⁶ For example, e could have only a very small probability, or e could only occur if an unreasonable computing effort is performed (e.g., by an adversary), but in both cases e can occur in principle, i.e., $e \neq \perp$.

Thus, a guarantee of the form $e \preceq f$ can equivalently be stated in the form that a certain event, $e \dot{-} f$, never occurs.

For example, one can consider the guarantee that the authentication failure event e does not occur *unless* a signature forgery event f has occurred, written as $e \preceq f$, capturing the intuitive idea that if the signature scheme is secure (and hence it is reasonable to assume that the signature forgery event does not occur), the authentication system is secure (i.e., the authentication failure does not occur).

In such arguments, events are often composed of several events, for example, $e \preceq f_1 \vee f_2 \vee \dots \vee f_k$ for some events $f_1, \dots, f_k$. This states that in order to prevent e , it suffices to prevent each of the events $f_1, \dots, f_k$.

Such statements are naturally composable. For example, if we have proven $e \preceq f \vee g$ and $g \preceq h \vee i$, then it follows directly (as a simple algebraic derivation) that

$$e \preceq f \vee h \vee i.$$

3.2 Parameterized Events and Terms

Often, it will be useful to consider events that are parameterized by a set X , for example, events parameterized by some set of messages M .

That is, for an event algebra E and a finite¹⁷ set X , we consider functions (parameterized events) $E : X \rightarrow E$ (one may also think of families of events indexed by X). This will allow us to form terms using such functions and form meets and joins over parameter sets. For example, for a fixed $X = \{x_1, \dots, x_n\}$ we write $\bigvee_{x \in X} E(x)$ or simply $\bigvee_x E(x)$ for $E(x_1) \vee \dots \vee E(x_n)$.

Notation. From here on, we denote parameterized events by uppercase sans-serif letters $E, F, G \dots$ and parameter sets by uppercase italic letters $X, Y, Z \dots$. We will write E_x as shorthand notation for $E(x)$ and understand $\bigvee_x$ as applying to all x in the corresponding set X .

It will be convenient to introduce abbreviations for certain terms, both to emphasize their importance and to make them easily recognizable. For example, the following term (which we will use later) expresses that F_x does not precede E_x for some x in a parameter set X and two parameterized events E, F :

$$\text{nPre}_{E,F}^X := \bigvee_x (E_x \dot{-} F_x) \tag{1}$$

Next, we discuss the type of statements we prove in the remaining paper.

¹⁷ It is possible to prove analogous statements for infinite sets, see [Section B.4](#). Since results about arbitrary but finite sets seem to suffice for practical applications, we consider the finite case only, which allows for axiom simplicity.

3.3 Universal Event Inequalities

An important benefit of introducing event algebras as an abstract algebraic structure with a well-defined set of axioms is that it allows us, as for any such algebra, to prove statements that follow purely from the axioms.¹⁸ We apply this approach to event algebras: We prove universally quantified event inequalities (*universal event (algebra) inequalities*, UEI), that is, statements of the form

“For any event algebra E , any sets $X, Y, \dots$, any $E, F, \dots$ we have $t_1 \preceq t_2$ ”,

where t_1 and t_2 are terms that are built from the parameterized events $E, F, \dots$ and event algebra operations¹⁹. The following is a concrete example of a universal event inequality (which is proved in [Section E.3](#)):

Lemma 3. *For any event algebra E , any sets X , any $E, F, G : X \rightarrow E$, we have*

$$\text{nPre}_{E,G}^X \preceq \text{nPre}_{E,F}^X \vee \text{nPre}_{F,G}^X$$

UEIs hold in any event algebra (and for any choice of parameter sets and parameterized events). To use this statement for a concrete (discrete-step) model, one must only verify that the axioms hold (for example, by verifying that the model defines a forest and using [Theorem 1](#)). This is the precise sense in which the statements we prove are *model-independent*.

3.4 Introducing Narratives

To understand and apply UEIs in a particular context, but also to decide which UEIs are relevant, it is often useful to give suggestive names to the variables, such as $\text{In}(m)$, standing for “*input m is given*”, or $\text{App}_A(x)$, standing for “*value x appears at (a fixed) location A* ”. Such names are to be understood as *narratives*: they convey what the event is intended to represent and simply let us talk about the setting. Crucially, they are only renamings of the parameterized events introduced in [Section 3.2](#) and carry no additional mathematical meaning. In [Section 3.5](#), we present examples that illustrate the difference between the abstract level and the narratives.

If desired, a narrative can be replaced by a *concretization* that gives $\text{App}_A(x)$ a precise meaning in some concrete model²⁰. Proposing possible such concretizations is a research direction orthogonal to this paper and is not necessary for proving and using our results.

¹⁸ Consider, as an analogy, the statement “For any element a of a ring R , we have $a \cdot 0 = 0$ ”. This statement can be derived directly from the ring axioms, and, since we have not made any assumptions about R and a , this obviously holds for any element of any ring. To use this statement in a concrete structure (such as the integers), one only needs to verify that the ring axioms hold.

¹⁹ From a universal algebra perspective, we prove inequalities in the equational theory of event algebras; that is, they hold in the free event algebra on the corresponding (families of) variables. We discuss free event algebras in [Section E.2](#).

²⁰ $\text{App}_A(x)$ could, for example, be the event that the string x appears at a particular location, in a particular manner, on a particular tape of a particular Turing machine.

3.5 Authentication and Authentication Amplification

We present a narrative in which the term $\text{nPre}_{E,F}^X$ and [Lemma 3](#) have cryptographic meaning.

Narrative: Authentication. Guaranteeing the authenticity of messages $m \in M$ in a certain context is a fundamental cryptographic task. For example, consider a communication channel for transmitting messages $m \in M$ from point A to B (for example, two parties, cf. [Figure 2](#)).

(A simple form of) authenticity is the guarantee that only messages that have previously appeared at point A appear at point B. In order to express this guarantee, we introduce two (suggestively named) parameterized events $\text{App}_A(m)$ and $\text{App}_B(m)$, expressing that m appears at the corresponding location. The authentication guarantee can now be expressed as the following event not occurring:

$$\text{nPre}_{\text{App}_B, \text{App}_A}^M = \bigvee_{m \in M} (\text{App}_B(m) \div \text{App}_A(m)).$$

Note that all we have done is rename the variables in [Equation \(1\)](#). It may also be helpful to introduce a narrative-appropriate name for the above composed event, such as $\text{AutF}_{A,B} := \text{nPre}_{\text{App}_B, \text{App}_A}^M$. In [Section 4.6](#), we will give a different narrative for the abstract event nPre .

Example 1 (Transitivity of Authentication). One may now ask how one would state and prove the transitivity of authenticity. Stated in the narrative, it reads as follows: “If m can’t appear at B without having appeared at A, and if m can’t appear at C without having appeared at B, then we can conclude that m can’t appear at C without having appeared at A.” In the suggestive notation, it can be written as

$$\text{AutF}_{A,C} \preceq \text{AutF}_{A,B} \vee \text{AutF}_{B,C},$$

which holds by [Lemma 3](#) via renaming $X, E, F, G \mapsto M, \text{App}_C, \text{App}_B, \text{App}_A$.

This is, of course, a very simple example and (almost) obvious to a computer scientist. In fact, such “obvious” proofs that are actually at the level of a narrative, like the above proof in quotes, are not uncommon in the literature.

Example 2 (Authentication Amplification). As a slightly more involved example that is for now only discussed informally, but which we will return to in [Section 4.4](#), consider a setting (see [Figure 3](#)) in which two parties, A and B, have

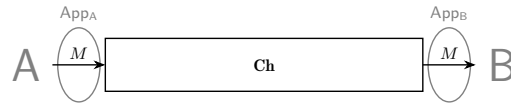


Fig. 2: A setting (narrative) in which A can send messages from a set M to B via channel **Ch**. Events $\text{App}_A(m)$ and $\text{App}_B(m)$ occur when the respective party sends or receives m .

access to two communication channels for transmitting messages from A to B, an *insecure* channel for long messages (in a set M), and an *authenticated* channel for short messages (in a much smaller set H).²¹ How can A send a long message authentically to B?

The following simple protocol works: A and B use a hash function $H : M \rightarrow H$. To (authentically) transmit $m \in M$, A sends m over the insecure channel and $h = H(m)$ over the authentic channel. Upon receiving $m' \in M$ via the insecure channel and h' via the authenticated channel, B accepts m' if $h' = H(m')$.

In this narrative, the following events are relevant: The overall authentication-failure event $\text{AutF}_{A,B}^M$ that B accepts a message $m' \in M$ that was not input by A, the authentication-failure event $\text{AutF}_{A,B}^H$ that the channel for H outputs a value h' that was not input by A, and the event $\text{Coll}_{A,B}$ that a (cross-party) hash collision occurs, i.e., the event that A computes $h = H(m)$ and that B computes $h = H(m')$ (for the same h). Moreover, there are events S_A and S_B that occur when A (respectively B) deviates from the protocol specification (for example, when A sends over the authentic channel an h which it did not obtain by hashing some m). The security of the protocol is thus captured by the event inequality

$$\text{AutF}_{A,B}^M \preceq \text{AutF}_{A,B}^H \vee \text{Coll}_{A,B} \vee S_A \vee S_B. \quad (2)$$

It can be read as follows: *Assuming a correct execution of the protocol (i.e., that S_A and S_B don't occur), an authentication failure for messages in M cannot occur without either an authentication failure for messages in H or a hash collision occurring.* As in the previous example, (2) can be understood as obtained from an abstract result via renaming: This is discussed in Section 4.4, where also the structure of events $\text{Coll}_{A,B}$, S_A , and S_B will be defined.

3.6 Local Events as Protocol Specifications.

The right-hand side of inequality (2) decomposes naturally into cross-party events ($\text{AutF}_{A,B}^M$ and $\text{Coll}_{A,B}$) and the local events S_A and S_B . This decomposition is purely narrative, but useful: S_A and S_B may be understood as per-party *protocol specifications*. That is, S_A and S_B specify local properties, namely, for each party, the event that is prevented by the protocol (engine), if executed correctly.

²¹ For example, the first channel could be an (insecure) e-mail, and the second channel could be a video-chat allowing A to read a few digits (authentically) to B.

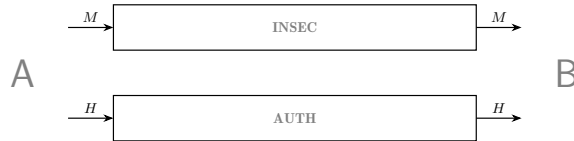


Fig. 3: Party A can transmit messages to B via two channels, one insecure and one authenticated.

Making the local events explicit is necessary for the inequality (2) to hold, but it is also beneficial for several reasons. First, setting $S_A = S_B = \perp$ (or, equivalently, working in an event algebra in which these events cannot occur) corresponds to the assumption that both parties follow the protocol. The event-algebraic formulation makes this assumption explicit instead of leaving it tacit. Second, S_A and S_B may be read as (temporal²²) safety properties, against which a concrete implementation can be model-checked. Third, the cross-party/local decomposition is helpful in determining which among the many UEIs are of interest: Those that admit a cross-party/local decomposition.

It is straightforward to extend this narrative to more than two parties. The narratives presented in Section 4 will remain within the two-party narrative.

4 Cryptographic Applications

In this section, we will illustrate that many statements of cryptographic interest can be stated and proved on the general, minimal level of universal event inequalities. We give several examples, ordered by the number of parameter sets and parameterized events, where we will first motivate a cryptographic setting (narrative), then state and prove relevant universal event inequalities, and then obtain the cryptographic narrative statement by renaming, as in Section 3.5.

Concretely, we discuss unforgeability guarantees (4.1), how to obtain authenticity guarantees from unforgeability guarantees (4.2), and collision-freeness guarantees (4.3). In (4.4), we revisit the example of amplifying authenticity (cf. Example 2). Afterwards, we discuss the security of the Hash-then-Sign (and Hash-then-MAC) paradigm (4.5). (4.6) discusses variants of unforgeability guarantees, and in (4.7), we prove the security of a generic construction of strongly unforgeable signature schemes due to [1] as a universal event inequality. As a final example, in (4.8), we give a UEI proof of security of the recent “BoP-1”-combiner [2]. To conclude, we compare our abstract approach to traditional game-based approaches (4.9).

4.1 Unforgeability Guarantees

A fundamental idea to ensure the authenticity of a message m in some message set M is to compute an additional value t (for example, a signature, a message authentication code (MAC) tag, or a tag produced by an information-theoretic authentication system) in some set of *tags* T , such that the pair (m, t) allows the verification of m ’s the authenticity. For any such scheme, which we will call *tagging schemes*, the guarantee one is interested in is *unforgeability*: the verification must only accept such message-tag pairs that have been created by a certain party (e.g., the party that possesses a secret signing key).

To express the event-algebraic principle of unforgeability, we need to consider two parameterized sets of events: the event that a tag t is created for a message

²² See also Section D.

m (at a certain location A , for example a party), denoted $\text{Tag}_A(m, t)$, and the event that a message-tag pair (m, t) is accepted by the verification at B , denoted $\text{Acc}_B(m, t)$ (cf. Figure 4).

We emphasize again that the terms (e.g., “tagging scheme”) and symbols (e.g., Tag) we use suggest a certain (more or less) concrete narrative, but do not have any mathematical meaning; what matters is that there are two parameter sets (M and T) and two parameterized events ($\text{Tag}(m, t)$ and $\text{Acc}(m, t)$). The abstract event capturing the structure of forgery²³ events is the following (for sets X, Y , and 2-argument parameterized events $E_{x,y}$ and $F_{x,y}$):

$$\text{pnPre}_{E,F}^{X,Y} := \bigvee_{x,y} \left(E_{x,y} \div \bigvee_{y'} F_{x,y'} \right). \quad (3)$$

In the suggestive notation, this becomes:

$$\text{Forg}_{A,B}^{M,T} := \text{pnPre}_{\text{Acc}_B, \text{Tag}_A}^{M,T} = \bigvee_{\substack{m \in M \\ t \in T}} \left(\text{Acc}_B(m, t) \div \bigvee_{t' \in T} \text{Tag}_A(m, t') \right),$$

4.2 Authenticity Guarantees from Unforgeability Guarantees

We first prove an abstract event inequality that relates events of the structure nPre (1) and pnPre (3) and then discuss this result in terms of the narrative introduced above. Proofs are deferred to Section F.2.

Lemma 4. *For any event algebra E , any finite sets X, Y , any $E, F : X \rightarrow E$, $G, H : X \times Y \rightarrow E$ we have*

$$\text{nPre}_{E,F}^X \preceq \text{pnPre}_{G,H}^{X,Y} \vee \text{P1}_{H,F}^{X,Y} \vee \text{P2}_{E,G}^{X,Y}$$

where $\text{P1}_{H,F}^{X,Y} := \bigvee_{x,y} (H_{x,y} \div F_x)$ and $\text{P2}_{E,G}^{X,Y} := \bigvee_x (E_x \div \bigvee_{y'} G_{x,y'})$

²³ We discuss some variants of forgeries in Section 4.6.

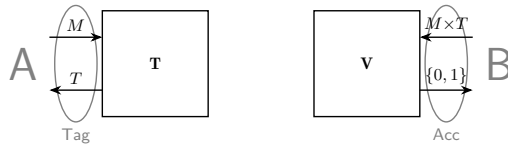


Fig. 4: A setting in which A produces tags for messages in a set M using system T and B can verify message-tag pairs in $M \times T$ using system V . Event $\text{Tag}_A(m, t)$ occurs when T outputs tag t for message m and $\text{Acc}_B(m, t)$ occurs when V outputs bit 1 for input (m, t) .

Following the authentication and unforgeability narratives from [Sections 3.5](#) and [4.1](#), the above lemma can be written as:

$$\text{AutF}_{A,B}^M \preceq \text{Forg}_{A,B}^{M,T} \vee S_A^{(4)} \vee S_B^{(4)} \quad (4)$$

$$\begin{aligned} \text{where } S_A^{(4)} &= P1_{\text{Tag}_A, \text{App}_A}^{M,T} = \bigvee_{m \in M, t \in T} (\text{Tag}_A(m, t) \div \text{App}_A(m)), \\ S_B^{(4)} &= P2_{\text{App}_B, \text{Acc}_B}^{M,T} = \bigvee_{m \in M} \left(\text{App}_B(m) \div \bigvee_{t \in T} (\text{Acc}_B(m, t)) \right). \end{aligned}$$

Inequality (4) can be read as: *an authentication failure can't occur without a forgery or events $S_A^{(4)}$ or $S_B^{(4)}$* . In this narrative, [Lemma 4](#) implies the security of the standard use of tagging schemes: Accept only such messages m as authentic (event $\text{App}_B(m)$) that have been verified for some tag t .

[Figure 5](#) illustrates this in a refined narrative: A pair of systems **T** and **V** jointly define event $\text{Forg}_{A,B}^{M,T}$ (as in [Figure 4](#)). Furthermore, parties A and B each execute a protocol machine²⁴ C_A (resp. C_B) that has access to **T** (resp. **V**). C_A and C_B (jointly) define event $\text{AutF}_{A,B}^M$. Let us discuss events $S_A^{(4)}$ and $S_B^{(4)}$ following this narrative: Preventing $S_A^{(4)}$ corresponds to the guarantee that tags are only produced for messages that have been input to C_A . Preventing $S_B^{(4)}$ corresponds to C_B only outputting messages m that have been accepted by **V** (for some tag).

Note that $S_A^{(4)}$ (resp. $S_B^{(4)}$) consists only of events with matching subscripts²⁵. In that sense, they only concern party A (resp. B).

[Section F.3](#) contains Lean code that allows to formally verify [Lemma 4](#).

4.3 Collision Freeness Guarantees and Domain Extension

The domain X of many cryptographic primitives can be extended by composing it with a collision-resistant hash function that maps elements of some larger

²⁴ The corresponding terminology of Constructive Cryptography [14] is *converter*, as applying a converter to a system converts it into a (usually) different system.

²⁵ That is, they are contained in the subalgebra generated by the elements with subscript A (resp. B).

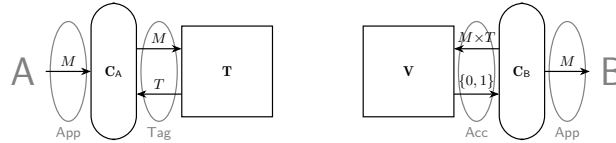


Fig. 5: The setting described above where both events $\text{Forg}_{A,B}^{M,T}$ and $\text{AutF}_{A,B}^M$ are defined, analogously to [Figures 2](#) and [4](#).

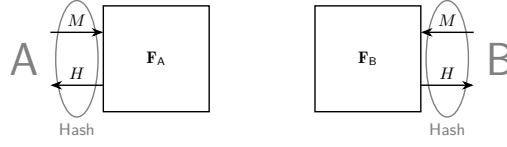


Fig. 6: A setting in which A and B can input values M to $\mathbf{F}_A$ (resp. $\mathbf{F}_B$) to obtain values from H . Event $\text{Hash}_A(m, h)$ (resp. $\text{Hash}_B(m, h)$) occurs, when $\mathbf{F}_A$ (resp. $\mathbf{F}_B$) produced output h for input m .

domain to X . For a hash function with domain M and codomain H , a collision occurs when the same hash value h is produced for two distinct inputs $m \neq m'$. Such a collision should not occur, no matter where the hash function is evaluated. In a setting (cf. Figure 6) involving a hash function and two parties A and B, the relevant events are the events that either party computes hash value h for input m , denoted $\text{Hash}_A(m, h)$ and $\text{Hash}_B(m, h)$, respectively.

That is, we have a setting with two parameter sets (M and H) and two parameterized events ($\text{Hash}_A(m, h)$ and $\text{Hash}_B(m, h)$). The abstract structure of a collision event²⁶ is:

$$\text{Agr}_{E,F}^{X,Y} := \bigvee_{\substack{x, x', y, \\ x \neq x'}} (E_{x,y} \wedge F_{x',y}) \quad (5)$$

Or, in suggestive notation:

$$\text{Coll}_{A,B}^{M,H} := \text{Agr}_{\text{Hash}_A, \text{Hash}_B}^{M,H} = \bigvee_{\substack{m, m' \in M, h \in H \\ m \neq m'}} (\text{Hash}_A(m, h) \wedge \text{Hash}_B(m', h)),$$

and the collision-freeness guarantee corresponds to this event not occurring.

We now give an abstract domain extension result which, depending on the narrative, implies the security of the authentication amplification example (cf. Example 2) and of the Hash-then-Sign paradigm (and other constructions). It relates events of the structure pnPre (3) and Agr (5):

Lemma 5. *For any event algebra E , any finite sets X, Y, Z , any $E, F : X \times Z \rightarrow E$, $G, H : X \times Y \rightarrow E$, $I, J : Y \times Z \rightarrow E$ we have*

$$\text{pnPre}_{E,F}^{X,Z} \preceq \text{Agr}_{G,H}^{X,Y} \vee \text{pnPre}_{I,J}^{Y,Z} \vee \text{P1}_{F,H,J}^{X,Y,Z} \vee \text{P2}_{E,G,I}^{X,Y,Z}$$

$$\text{where } \text{P1}_{F,H,J}^{X,Y,Z} := \bigvee_{x,y,z} ((H_{x,y} \wedge J_{y,z}) \dot{-} F_{x,z}) \vee \bigvee_{y,z} \left(J_{y,z} \dot{-} \bigvee_x H_{x,y} \right)$$

$$\text{and } \text{P2}_{E,G,I}^{X,Y,Z} := \bigvee_{x,z} \left(E_{x,z} \dot{-} \bigvee_y (G_{x,y} \wedge I_{y,z}) \right)$$

²⁶ We model the collision event for two parties/locations A and B. It should be clear how to define collisions for a different number of scopes.

4.4 Domain Extension for Authenticity Guarantees (Example 2)

We now return to Example 2 and show how Lemma 5 (or more precisely, the direct consequence Corollary 2, deferred to Section F.2) implies the security of the authentication amplification protocol. Concretely, renaming the variables in Lemma 5 as follows

$$X, Y, E, F, G, H, I, J \mapsto M, H, \text{App}_B, \text{App}_A, \text{Hash}_B, \text{Hash}_A, \text{App}'_B, \text{App}'_A,$$

allows us to write the security statement as

$$\text{AutF}_{A,B}^M \preceq \text{Coll}_{A,B}^{M,H} \vee \text{AutF}_{A,B}'^H \vee S_A^{(6)} \vee S_B^{(6)} \quad (6)$$

where $S_A^{(6)} = \text{P1}_{\text{App}_A, \text{Hash}_A, \text{App}'_A}^{M,H}$ and $S_B^{(6)} = \text{P2}_{\text{App}_B, \text{Hash}_B, \text{App}'_B}^{M,H}$.

Note that this is exactly Equation (2). In the corresponding narrative, preventing $S_A^{(6)}$ ensures that A only computes a hash h (and sends it via the authentic channel) for messages that it has also sent via the insecure channel. Moreover, A must only send hashes that have been computed by A. $S_B^{(6)}$ ensures that only messages m for which the computed hash matches one received via the authentic channel are considered authentic.

4.5 The Hash-then-Tag Paradigm

The domain extension principle (Section 4.3) can be applied to signature schemes and MACs. A signature or MAC scheme for a greater domain M can be constructed from a scheme for a smaller domain H and a collision-resistant hash function mapping M to H : Instead of signing messages directly, messages are first hashed, and then their hash values are signed. This is known as the “Hash-then-Sign” (sometimes “Hash-and-Sign”) paradigm or the “Hash-then-MAC” paradigm ([28]), respectively.

We illustrate this principle using the narrative of tagging schemes. Lemma 5 can be rewritten to obtain a statement for the security of the domain extension of tagging schemes, which we call the “Hash-then-Tag” paradigm:

$$\text{Forg}_{A,B}^{M,T} \preceq \text{Coll}_{A,B}^{M,H} \vee \text{Forg}_{A,B}'^{H,T} \vee S_A^{(7)} \vee S_B^{(7)} \quad (7)$$

where $S_A^{(7)} = \text{P1}_{\text{Tag}_A, \text{Hash}_A, \text{Tag}'_A}^{M,H,T}$ and $S_B^{(7)} = \text{P2}_{\text{Acc}_B, \text{Hash}_B, \text{Acc}'_B}^{M,H,T}$, via the following renaming:

$$X, Y, E, F, G, H, I, J \mapsto M, H, \text{Acc}_B, \text{Tag}_A, \text{Hash}_B, \text{Hash}_A, \text{Acc}'_B, \text{Tag}'_A.$$

Inequality (7) can be read as “the forgery event for the constructed tagging scheme cannot occur unless either the forgery event for the inner scheme, the hash collision event, or one of the events $S_A^{(7)}$ and $S_B^{(7)}$ occurs.”

We do not discuss the events $S_A^{(7)}$ and $S_B^{(7)}$ in detail, but point out (again) that they can be understood as protocol specifications that, for example, are satisfied by signature schemes following the Hash-then-Sign paradigm.

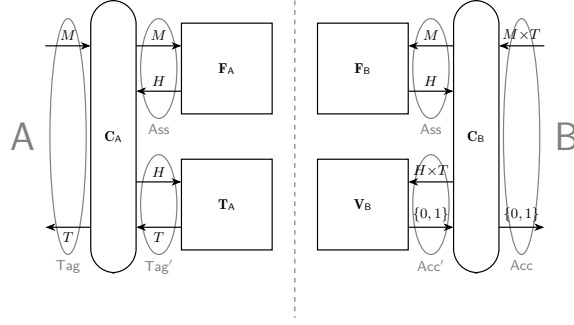


Fig. 7: The Hash-then-Tag paradigm. The relevant events are defined analogously to Figures 4 and 6.

Although the conventional reduction proofs for Hash-then-Sign and Hash-then-MAC are very similar, the typical game-based approach (e.g., [29]) requires a separate proof for each primitive since each primitive is associated with a different security game. Note that no keys appear in our statement: the forgery events are defined only for message and tag spaces. This means that (7) implies the security of the above construction for *any* schemes in which a forgery event is defined, such as MACs, signature schemes, or other types of schemes with a forgery event. That is, through the use of event algebras, the security of a large class of constructions (including Hash-then-Sign and Hash-then-MAC) can be stated as a single minimal statement. In Section 4.9, we use the Hash-then-Tag example to give a detailed comparison of our approach and game-based approaches.

4.6 Relations between Different Types of Unforgeability Guarantees.

The order relation on events allows us to directly establish relationships between different types of forgery events (and the corresponding notions of unforgeability). We first introduce different types of forgeries and afterwards discuss their relation.

Strong Unforgeability Guarantees. Strong unforgeability (for both MACs and signature schemes)²⁷ requires that it is infeasible to generate a new valid tag, even for messages that have already been tagged. A weak forgery²⁸ occurs

²⁷ MACs that are strongly unforgeable (such as most deterministic MACs) are used to generically construct authenticated encryption schemes (the encrypt-then-MAC paradigm requires a strongly unforgeable MAC [30]), while strongly unforgeable signatures are used in some constructions of CCA2-secure encryption schemes ([31,32,33] or group key exchange protocols[34]).

²⁸ Preventing weak forgeries over (plain) forgeries is a stronger unforgeability guarantee, since every forgery is also a weak forgery.

when a message-tag pair (m, t) is accepted that was not produced by the tagger. This event has the structure $\mathbf{nPre} (1)^{29}$ with pair-valued parameters:

$$\mathbf{wkForg}_{A,B}^{M,T} := \mathbf{nPre}_{\text{Acc}_B, \text{Tag}_A}^{(M \times T)} \bigvee_{m \in M, t \in T} (\text{Acc}_B(m, t) \dot{-} \text{Tag}_A(m, t).)$$

k -Time Unforgeability Guarantees. In many settings, unforgeability is only guaranteed up until a certain number k of message-tag pairs is produced: Examples include one-time signature schemes (e.g., [35,36]), nonce-based MAC schemes (e.g., [37]), or information-theoretic authentication schemes (e.g., [38]).

Abstractly, the event that E_x has occurred for at least k distinct parameters x is:

$$k\text{Mult}_E^X := \bigvee_{\substack{Y \subseteq X \\ |Y|=k}} \bigwedge_y E_y, \quad \text{which for } k=2 \text{ is } 2\text{Mult}_E^X = \bigvee_{x \neq x'} (E_x \wedge E_{x'}). \quad (8)$$

The one-time (k -time) forgery event occurs when the forgery event occurs *before* two $(k+1)$ message-tag pairs have been produced. Abstractly, this is expressed as:

$$k\text{pnPre}_{E,F}^{X,Y} := \text{pnPre}_{E,F}^{X,Y} \dot{-} (k+1)\text{Mult}_F^{(X \times Y)}, \quad (9)$$

$$k\text{nPre}_{E,F}^X := \text{nPre}_{E,F}^X \dot{-} (k+1)\text{Mult}_F^X \quad (10)$$

where, in the narrative, $k\text{pnPre}_{E,F}^X$ expresses k -time forgeries:

$$k\text{Forg}_{A,B}^{M,T} := k\text{pnPre}_{\text{Acc}_B, \text{Tag}_A}^{M,T} = \text{Forg}_{A,B}^{M,T} \dot{-} (k+1)\text{Mult}(M \times T)\text{Tag}_A,$$

and $k\text{nPre}_{E,F}^{X,Y}$ expresses weak k -time forgeries:

$$k\text{wkForg}_{A,B}^{M,T} := k\text{nPre}_{\text{Acc}_B, \text{Tag}_A}^{M,T} = \text{wkForg}_{A,B}^{M,T} \dot{-} (k+1)\text{Mult}(M \times T)\text{Tag}_A.$$

Establishing Relations. The event algebra axioms allow us to immediately establish relations between different notions of unforgeability: For example, we have

$$1\text{Forg}_{A,B}^{M,T} \preceq \text{Forg}_{A,B}^{M,T} \preceq \text{wkForg}_{A,B}^{M,T}$$

where the first inequality follows from [Lemma 1.\(i\)](#) and the second from [\(A3\)](#), [\(M\)](#) and [Lemma 1.\(iv\)](#). Note that there is no reduction needed, for example, to express that strong unforgeability implies (regular) unforgeability: it is simply the statement that the corresponding events are ordered.

²⁹ The same structure as the authentication failure event in a different narrative.

$\text{Sig}(sk, m):$	$\text{Vfy}(vk, m, \sigma):$
1 $(vk^{(1)}, sk^{(1)}) \leftarrow \text{Gen}^{(1)}()$	1 $(vk^{(1)}, \sigma^{(1)}, \sigma') \leftarrow \sigma$
2 $\sigma' \leftarrow \text{Sig}'(sk, (m, vk^{(1)}))$	2 $b' \leftarrow \text{Vfy}'(vk, (m, vk^{(1)}), \sigma')$
3 $\sigma^{(1)} \leftarrow \text{Sig}^{(1)}(sk^{(1)}, \sigma')$	3 $b^{(1)} \leftarrow \text{Vfy}^{(1)}(vk^{(1)}, \sigma', \sigma^{(1)})$
4 return $(vk^{(1)}, \sigma^{(1)}, \sigma')$	4 return $b' \wedge b^{(1)}$

Fig. 8: [1]'s construction of a strongly unforgeable signature scheme $\Pi=(\text{Gen}, \text{Sig}, \text{Vfy})$ from an unforgeable signature scheme $\Pi'=(\text{Gen}', \text{Sig}', \text{Vfy}')$ and a strongly unforgeable one-time signature scheme $\Pi^{(1)}=(\text{Gen}^{(1)}, \text{Sig}^{(1)}, \text{Vfy}^{(1)})$. Gen is defined as equal to Gen' .

4.7 Preventing Weak Forgeries via One-Time Weak Forgeries

In this section, we give an abstract proof of the security of [1]'s construction of a strongly unforgeable signature scheme Π from a (weakly) unforgeable signature scheme Π' and a strongly unforgeable one-time signature (OTS) scheme $\Pi^{(1)}$. The construction (see the pseudo-code description in Figure 8) works as follows: To sign a message m , an OTS key pair $(vk^{(1)}, sk^{(1)})$ is generated, the pair $(m, vk^{(1)})$ is signed using the weakly unforgeable signature scheme Π' , resulting in signature σ' , and finally σ' is signed using $sk^{(1)}$, resulting in $\sigma^{(1)}$. Overall, a signature consists of the sampled verification key $vk^{(1)}, \sigma^{(1)}$, and σ' . To verify, both $\sigma^{(1)}$ and σ' are verified using the corresponding keys.

To identify the relevant parameter sets and parameterized events, we first describe the construction in the narrative of tagging schemes, and subsequently prove an abstract result (Lemma 6) that implies the security of this construction. Consider the setting depicted in Figure 9, in which for each element i of some set I (corresponding to the OTS-public keys), a distinct (one-time) tagging scheme, called the i -th instance, for messages in some set T_1 (corresponding to signatures of Π') and tags in some set T_2 (corresponding to signatures of $\Pi^{(1)}$) is defined. Furthermore, there is a tagging scheme, called the inner instance (corresponding to Π'), for messages in $M \times I$ and tags in T_1 (M corresponds to the message space of Π).

That is, we have four parameter sets: M, T_1, T_2 , and I , and six parameterized events $\text{Tag}_A, \text{Acc}_B$, corresponding to the constructed tagging scheme, $\text{Tag}'_A, \text{Acc}'_B$, corresponding to the inner scheme, and $\text{Tag}_{iA}, \text{Acc}_{iB}$, corresponding to the one-time tagging schemes (we write the parameter $i \in I$ as part of the name). We can now state the corresponding universal event inequality:

Lemma 6. *For any event algebra E , any finite sets X, Y, Z, J , any $E, F : (X \times Y \times Z \times J) \rightarrow E$, $G, H : (X \times Y \times J) \rightarrow E$, $I, J : (Y \times Z \times J) \rightarrow E$ we have*

$$\begin{aligned} \text{pnPre}_{E,F}^{(X \times Y \times Z \times J)} &\preceq \text{pnPre}_{G,H}^{(X \times J), Y} \vee \bigvee_j \left(\text{1pnPre}_{I(\cdot, \cdot, j), J(\cdot, \cdot, j)}^{Y, Z} \right) \\ &\vee \text{P1}_{F,H,J}^{X,Y,Z,J} \vee \text{P2}_{E,G,I}^{X,Y,Z,J} \end{aligned}$$

$$\begin{aligned}
\text{where } P1_{F,H,J}^{X,Y,Z,J} &:= \bigvee_{x,y,z,j} ((H_{x,y,j} \wedge J_{y,z,j}) \dot{-} F_{x,y,z,j}) \\
&\quad \vee \bigvee_{y,z,j} \left(J_{y,z,j} \dot{-} \bigvee_{x'} H_{x',y,j} \right) \\
&\quad \vee \bigvee_j \text{Mult}_{J(\cdot,\cdot,j)}^{Y,Z} \vee \bigvee_j \text{Mult}_{H(\cdot,\cdot,j)}^{X,Y} \\
\text{and } P2_{E,G,I}^{X,Y,Z,J} &:= \bigvee_{x,y,z,j} (E_{x,y,z,j} \dot{-} (G_{x,y,j} \wedge I_{y,z,j}))
\end{aligned}$$

Lemma 6 can be rewritten to express how the weak forgery event for the outer instance can be prevented by preventing (stronger) forgeries for the inner instance and weak one-time forgeries for each instance i :

$$\text{wkForg}_{A,B}^{M,(I \times T_1 \times T_2)} \preceq \text{Forg}_{A,B}'^{(M \times I),T_1} \vee \bigvee_{i \in I} \text{1wkForg}_{A,B}^{T_1,T_2,i} \vee S_A^{(11)} \vee S_B^{(11)} \quad (11)$$

where $S_A^{(11)} = P1_{\text{Tag}_A, \text{Tag}'_A, \text{Tag}_{-A}}^{M,T_1,T_2,I}$ and $S_B^{(11)} = P1_{\text{Acc}_B, \text{Acc}'_B, \text{Acc}_{-A}}^{M,T_1,T_2,I}$. That is, we have renamed

$$X, Y, Z, J \mapsto M, T_1, T_2, I \text{ and } E, F, G, H, I, J \mapsto \text{Acc}_B, \text{Tag}_A, \text{Acc}'_B, \text{Tag}'_A,$$

and for every j , $I(\cdot, \cdot, j), J(\cdot, \cdot, j) \mapsto \text{Acc}_{-i_B}, \text{Tag}_{i_A}$.

The following requirements are captured by $S_A^{(11)}$:

- (i) Once message-tag pair $((m, i), t_1)$ is produced by the inner instance and message-tag pair (t_1, t_2) is produced by the i -th instance, (i, t_1, t_2) must be a tag for m in the outer instance.
- (ii) Only such t_1 that have been produced for (m', i) by the inner instance are tagged at instance i .

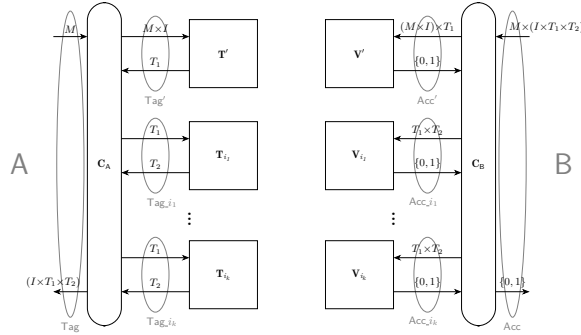


Fig. 9: The setting discussed in Section 4.7.

- (iii) For each instance i , at most one message-tag pair is produced.
- (iv) Each value i is tagged at most once at the inner instance.

$S_B^{(11)}$ defines the requirements for verification: A message-tag pair $(m, (i, t_1, t_2))$ should only be accepted by the outer instance if both $((m, i), t_1)$ was accepted by the inner instance and (t_1, t_2) was accepted by the i -th instance.

4.8 Tagging Combiners

The natural idea of combining multiple cryptographic primitives into a new primitive that is secure as long as one of the primitives is secure has recently received attention in the context of post-quantum cryptography. In particular, signature combiners that preserve strong unforgeability (see e.g. [2]) have been investigated. In this section, we will give an abstract proof of the security (strong unforgeability) of the “BoP-1”-construction of [2].

As in Section 4.7, we consider public keys as additional parameters in this section. That is, the relevant property of public keys is that for each key, corresponding tagging and accepting events are defined. Hence, we can equivalently think of a tagging scheme for messages M with public keys I as a single tagging scheme with an additional parameter I , which can be viewed as the message space having structure $(M \times I)$.

Unique Signature Schemes. BoP-1 constructs a strongly unforgeable signature scheme from two schemes: another strongly unforgeable signature scheme and a *unique* unforgeable signature scheme. A perfectly unique signature scheme has the property that for each public key and message, there is only one signature that verifies under the public key. Statistical or computational variants of this property require that it is infeasible to find a public key, a message, and two signatures that both verify.

In terms of events, the failure of these properties can be expressed as the event that, for a message m , two distinct tags are produced or accepted. Abstractly, the structure of this event can be expressed in terms of Agr (3):

$$\text{sAgr}_{E,F}^{X,Y} := \text{Agr}_{E,E}^{X,Y} \vee \text{Agr}_{E,F}^{X,Y} \vee \text{Agr}_{F,F}^{X,Y}. \quad (12)$$

In the narrative, the non-uniqueness event for tagging schemes is $\text{nUni}_{A,B}^{(M \times I), T} := \text{sAgr}_{\text{Tag}_A, \text{Acc}_B}^{T, (M \times I)}$. Note the change in the order of parameter sets, since Agr is defined as agreement in the second parameter for different values in the first parameter.

The BoP-1 Construction. The BoP-1 construction of Π from Π' and Π'' , described as pseudo-code in Figure 10, works as follows: Keys for Π are pairs of keys for Π' and Π'' . To sign a message m , the hash $h := H(m, vk', vk'')$ is computed, where (vk', vk'') is the corresponding pair of public keys. Afterwards, h is signed using Π'' , resulting in signature σ'' , and the pair (h, σ'') is signed using Π' , resulting in signature σ' . The overall signature is the pair (σ', σ'') . To verify, one first computes the hash and afterwards verifies both signatures.

Gen():	Sig(sk,m):	Vfy(vk,m,(σ',σ'')):
1 (vk', sk') ← Gen'()	1 ((sk', vk'), (sk'', vk'')) ← sk	1 h ← H(vk', vk'', m)
2 (vk'', sk'') ← Gen''()	2 h ← H(vk', vk'', m)	2 b' ← Vfy'((h, σ''), σ')
3 vk ← (vk', vk'')	3 σ'' ← Sig''(sk'', h)	3 b'' ← Vfy''(h, σ'')
4 sk ← ((sk', vk'), (sk'', vk''))	4 σ' ← Sig'(sk', (h, σ''))	4 return b' ∧ b''
5 return (vk, sk)	5 return (σ', σ'')	

Fig. 10: The construction BoP-1 of [2]. A signature scheme $\Pi=(\text{Gen}, \text{Sig}, \text{Vfy})$ is constructed from signature schemes $\Pi'=(\text{Gen}', \text{Sig}', \text{Vfy}')$ and $\Pi''=(\text{Gen}'', \text{Sig}'', \text{Vfy}'')$ and a hash function H . Π is strongly unforgeable if Π' is unique and unforgeable or Π'' is strongly unforgeable.

In [2], it is shown that Π is strongly unforgeable if Π' is unique and unforgeable or Π'' is strongly unforgeable.

We identify the relevant parameter sets: M, H, T_1, T_2, I, J , where M is the set of messages, H is the set of hash values, T_1 and T_2 correspond to signatures, and I and J correspond to public keys. The relevant parameterized events are three tagging schemes: The overall tagging scheme $\text{Tag}_A, \text{Acc}_B$ with messages in $(M \times I \times J)$ and tags in $(T_1 \times T_2)$, the unique tagging scheme $\text{Tag}'_A, \text{Acc}'_B$ with messages in $H \times I$ and tags in T_1 , and the tagging scheme $\text{Tag}''_A, \text{Acc}''_B$ with messages in $T_1 \times H \times J$ and tags in T_2 . Furthermore, there are hashing events $\text{Hash}_A, \text{Hash}_B$ with messages in $I \times J \times M$ and hashes H .

We now state the corresponding universal event inequality:

Lemma 7. *For any event algebra E , any finite sets I, J, W, X, Y, Z , any $E, F : (I \times J \times X \times Y \times Z) \rightarrow E$, $G, H : (I \times W \times Y) \rightarrow E$, $I, J : (J \times W \times Y \times Z) \rightarrow E$, $K, L : (I \times J \times X \times W) \rightarrow E$ we have*

$$\begin{aligned} \text{nPre}_{E,F}^{I \times J \times X \times Y \times Z} &\preceq \left(\text{nPre}_{G,H}^{I \times W \times Y} \wedge \text{pnPre}_{I,J}^{J \times W \times Y \times Z} \right) \vee \text{Agr}_{K,L}^{(I \times J \times X), W} \\ &\vee \text{Agr}_{I,I}^{(J \times W \times Y), Z} \vee \text{P1}_{F,H,I,L}^{I,J,X,Y,Z,W} \vee \text{P2}_{E,G,I,K}^{I,J,X,Y,Z,W} \end{aligned}$$

$$\begin{aligned} \text{where } \text{P1}_{F,H,I,L}^{I,J,X,Y,Z,W} &:= \bigvee_{i,j,w,x,y,z} ((L_{i,j,x,w} \wedge H_{i,w,y} \wedge J_{j,w,y,z}) \dot{-} F_{i,j,x,y,z}) \\ &\vee \bigvee_{\substack{i,j \\ x,y,w}} \left((H_{i,w,y} \wedge L_{i,j,x,w}) \dot{-} \bigvee_{z'} J_{j,w,y,z'} \right) \\ &\vee \bigvee_{i,j,x,y,z,w} ((K_{i,j,x,w} \wedge J_{j,w,y,z}) \dot{-} H_{i,w,y}) \\ &\vee \bigvee_{i,y,w} \left(H_{i,w,y} \dot{-} \bigvee_{j',x'} L_{i,j',x',w} \right) \end{aligned}$$

$$\text{and } P2_{E,G,I,K}^{I,J,X,Y,Z,W} := \bigvee_{i,j,x,y,z} \left(E_{i,j,x,y,z} \dot{-} \bigvee_w (K_{i,j,x,w} \wedge G_{i,w,y} \wedge I_{j,w,y,z}) \right)$$

Lemma 7 can be rewritten as (omitting the parameter sets):

$$\text{wkForg}_{A,B} \preceq (\text{wkForg}'_{A,B} \wedge \text{Forg}''_{A,B}) \vee \text{Coll}_{A,B} \vee \text{nUni}'_{A,B} \vee S_A^{(13)} \vee S_B^{(13)} \quad (13)$$

Where $S_A^{(13)} = P1_{\text{Tag}_A, \text{Tag}'_A, \text{Tag}''_A, \text{Hash}_A}^{I,J,M,T_1,T_2,H}$ and $S_B^{(13)} = P2_{\text{Acc}_B, \text{Acc}'_B, \text{Acc}''_B, \text{Hash}_B}^{I,J,M,T_1,T_2,H}$. As before, $S_A^{(13)}$ and $S_B^{(13)}$ capture sufficient protocol properties. Equation (13) can be read as “If Π' is unique, and both parties follow the protocol, a forgery for the overall scheme can’t occur without forgeries for *both* inner schemes or a collision.”

The renaming we have used is: $X, Y, Z, W \mapsto M, T_1, T_2, H$, and

$$E, F, G, H, I, J, K, L \mapsto \text{Acc}_B, \text{Tag}_A, \text{Acc}'_B, \text{Tag}'_A, \text{Acc}''_B, \text{Tag}''_A, \text{Hash}_B, \text{Hash}_A.$$

4.9 Comparison to the Game-Based Approach

We conclude this section with a discussion of how security statements phrased as UEIs compare to traditional game-based approaches.

At a high level, game-based notions of security are defined for (a random experiment on) the interaction between an adversary and a security game. There is a (usually implicit) execution model and, therefore, also an event algebra defined for this interaction. The events (e.g., that a forgery occurs) are also defined in this (implicit) model, and the UEIs stated at the abstract level hold unchanged in any such (implicit) concrete model.

Game-Based Approaches. More precisely, one first specifies a concrete computational model (e.g., Turing machines or circuits), based on which one defines the concept of an algorithm for the model and a communication model (e.g., based on tapes). Then, one defines a complexity notion (e.g., the number of steps an algorithm takes) and a notion of efficiency (e.g., polynomial in some parameter(s) in an asymptotic sense, or bounded by an explicit threshold). Finally, one typically defines the security of a cryptosystem as a binary property, namely as the infeasibility for the adversary to win a specific game. Reduction security statements take the form “if primitive P is secure, then primitive Q is secure,” and proofs of such statements proceed by defining a reduction algorithm B that, using an adversary A against the game G_Q corresponding to Q ’s security as a subroutine, is an adversary against the game G_P corresponding to P ’s security. The proof then argues that (i) when connected to G_P , B ’s behavior towards A is identical (or statistically/computationally close) to the behavior of G_Q , and (ii) the probability that A wins G_Q is bounded by (some expression including) the probability that B wins G_P . By contraposition, the hardness of G_P implies the hardness of G_Q .

UEIs. In [Section 4](#), we have exhibited several examples that prove statements of cryptographic interest on the abstract level of UEIs. Expressing these statements on this level rather than on the level of game-based reductions has several benefits:

UEIs avoid fixing a computational model, a communication model, a notion of complexity, or the notion of an adversary. In particular, they are not restricted to the setting where an adversary interacts with a game, and they hold in arbitrary contexts, for example, when a scheme is used within the broader context of some protocol, or in any real-world system in which the events are defined. Furthermore, the event-centered perspective of collisions discussed in [Section 4.3](#) is arguably better-suited to model the security assumption of existing hash functions (such as SHA-3), since it is not possible to define a meaningful complexity-based notion of hardness for the collision game for a concrete hash function³⁰.

Moreover, UEIs can be instantiated in many different settings, i.e., for different narratives. For example, [Lemma 5](#) directly implies the security of both the Hash-then-Sign and the Hash-then-MAC paradigms. In a game-based framework, the security of both paradigms must be stated and proved separately because the (existential) forgery game for signatures and the (existential) forgery game for MACs are defined differently.

Every σ -algebra is an event algebra (see [Section B.2](#)). Therefore, UEIs hold also in every σ -algebra that contains the corresponding events, in particular, in any random experiment between an adversary and a game. This means that statements involving an adversary’s success probability follow directly: If, for example, we have $e \preceq f \vee g$, then $\Pr(e) \leq \Pr(f) + \Pr(g)$ follows by the union bound, no matter how the random experiment is defined (that is, how the probability measure $\mathbf{P}$ is defined). We emphasize that no independence assumptions (for example, between the adversary’s and game’s randomness) need to be made. Concretely, the above includes cases where the adversary and the game have correlated randomness, and where keys or random challenges are not chosen uniformly. This also allows one to directly obtain probability bounds that would be expressed as an “index-guessing” argument in a game-based reduction, as in the example of [Section 4.7](#).

When a security statement can be expressed as a UEI, no simulation argument is needed. In game-based proofs, one usually proceeds by defining a reduction algorithm $\mathbf{B}$ that, when connected to the game $\mathbf{G}'$ one wants to reduce to, simulates the game $\mathbf{G}$ one wants to reduce from. It is necessary to argue that the behavior of $\mathbf{B}$ together with $\mathbf{G}'$ simulates the behavior of $\mathbf{G}$ well enough. That is, one compares probabilities in multiple random experiments. This step is required, even if the simulation is perfect and the reduction proceeds simply

³⁰ For any concrete hash function, there is an efficient algorithm that outputs a (hard-coded) collision. This fundamental issue is addressed in a complexity-based treatment by instead considering an asymptotic family of hash functions. However, SHA-3, for example, is not such an asymptotic family, and we want a security statement to hold for *any* concrete hash function, like SHA-3.

by forwarding. In contrast, UEIs are (universally quantified) statements about a single event algebra.

Reductions Beyond UEIs. Not every game-based reduction can be expressed on the level of UEIs. Concretely, game-based reductions in which the simulation argument is essential (including reductions for indistinguishability-style security notions), or reductions that use rewinding, are not expressible on this level alone. Finding the minimal level of abstraction to express such reductions is future work.

In [Section F.1](#), we identify the event-algebraic reasoning implicit in such game-based proofs using the Hash-then-Sign paradigm as an example.

5 Conclusions and Future Work

The paper has introduced the event algebra concept as a minimal and exact formalization of events in discrete-step models (corresponding to a tree or forest) and considered universal event inequalities (UEIs), a specific and basic type of event algebra theorem. As proof of concept for the approach, the abstract formulation of a significant number of cryptographic statements has been illustrated. The scope of UEIs and, more generally, of event-algebraic reasoning is, however, much broader, and the framework opens up a number of natural directions for future research, in three categories: 1) exploring the scope of UEIs, 2) exploring more general event algebra theorem types (than UEIs), and 3) going beyond event algebras (but related to them).

1) Exploring the scope of UEIs. The cryptographic examples expressed via UEIs in this paper are all within a two-party setting, but the framework extends in a straightforward manner to settings involving protocols between an arbitrary number of parties. Guarantees that hold even when some parties deviate from the protocol, a natural setting for distributed protocols, are of particular interest. The failure of consistency or validity and deviation from the protocol are expressible as events, as is the event that more than t protocol engines have deviated from the protocol (or, in a commonly-used narrative, that more than t parties have been corrupted). A systematic event-algebraic treatment of broadcast, consensus, and possibly blockchain protocols along these lines is a natural direction for future work.

Another quite straightforward goal is to model time by introducing a sub-algebra of timing events. This will allow us to capture guarantees of the form that a certain event (like the delivery of a message) occurs before or at a certain time. One can then, for example, prove tight conditions on the termination of a distributed protocol in terms of delay guarantees for the involved bilateral channels.

2) Beyond UEIs. UEIs are a particularly simple type of event-algebraic theorem, but the algebraic structure of event algebras is rich and allows more complex statements.

As a first observation, assuming that an event a cannot occur (i.e., $a = \perp$) corresponds to passing to the quotient with respect to an event-algebra congruence (an equivalence relation on events that is compatible with all event algebra operations), and one can show that every event-algebra congruence arises in this way from some event b .

Assumptions of the form $a = \perp$ are closed under conjunction, but their *disjunctions* are strictly more general: an assumption of the form “ $a = \perp$ or $b = \perp$ ” captures, for instance, the perfect binding guarantee of a commitment scheme, namely that (after a certain event) either opening to 0 *or* opening to 1 is impossible. Dually, assumptions of the form $a \neq \perp$ (the event a *can* occur) are closed under disjunction but not under conjunction. Such conjunctions allow us to express, for example, *binding failure*: (after a certain event) opening to 0 *and* opening to 1 are both possible.

Event algebras admit a *product*, which can intuitively be described as follows: Consider the pair of forests corresponding to two event algebras. The possible paths in the product forests are obtained by introducing a root for each pair of roots in the factors and adding as possible steps both individual and simultaneous steps in the factors. Products naturally yield a notion of *factorization* (of sub-event algebras). Factorization corresponds to *structural independence* in the sense that events from one factor place no constraint on those from the other. Assuming such structural independence allows us to prove impossibility statements. A promising direction for future work is to identify minimal assumptions that suffice for expressing relevant impossibility results, such as various Broadcast impossibility results.

3) Beyond Event Algebras. σ -algebras are event algebras, and UEs yield corresponding probability-theoretic statements via the union bound. A more ambitious goal is to generalize probability theory itself from σ -algebras to general event algebras. A promising starting point is the notion of *lattice valuations*, functions $m : E \rightarrow \mathbb{R}_{\geq 0}$ satisfying $m(\perp) = 0$, $m(\top) = 1$, monotonicity, and modularity ($m(a) + m(b) = m(a \wedge b) + m(a \vee b)$). Restricted to σ -algebras, lattice valuations coincide with probability measures, but on general event algebras, they form a strictly richer class of quantitative tools. A goal of this generalization is to express event-algebraic statements about indistinguishability, secrecy, and similar guarantees not captured by UEs.

References

1. M. Bellare and S. Shoup, “Two-tier signatures from the fiat–shamir transform, with applications to strongly unforgeable and one-time signatures,” *IET Information Security*, vol. 2, pp. 47–63, 2008. [Online]. Available: <https://digital-library.theiet.org/doi/abs/10.1049/iet-ifs%3A20070089>
2. J. Janneck, “Bird of prey: Practical signature combiners preserving strong unforgeability,” in *EUROCRYPT 2026*, ser. LNCS, J. Daemen and E. Thomé, Eds., vol. 16542. Springer, Heidelberg, May 2026, pp. 222–250.

3. M. Bellare and P. Rogaway, “Code-based game-playing proofs and the security of triple encryption,” Cryptology ePrint Archive, Paper 2004/331, 2004. [Online]. Available: <https://eprint.iacr.org/2004/331>
4. U. Maurer and R. Renner, “Abstract cryptography,” in *The Second Symposium on Innovations in Computer Science, ICS 2011*, B. Chazelle, Ed. Tsinghua University Press, 1 2011, pp. 1–21.
5. U. M. Maurer, “Indistinguishability of random systems,” in *EUROCRYPT 2002*, ser. LNCS, L. R. Knudsen, Ed., vol. 2332. Springer, Heidelberg, Apr. / May 2002, pp. 110–132.
6. U. M. Maurer, K. Pietrzak, and R. Renner, “Indistinguishability amplification,” in *CRYPTO 2007*, ser. LNCS, A. Menezes, Ed., vol. 4622. Springer, Heidelberg, Aug. 2007, pp. 130–149.
7. D. Jost and U. Maurer, “Overcoming impossibility results in composable security using interval-wise guarantees,” in *CRYPTO 2020, Part I*, ser. LNCS, D. Micciancio and T. Ristenpart, Eds., vol. 12170. Springer, Heidelberg, Aug. 2020, pp. 33–62.
8. S. Goldwasser and S. Micali, “Probabilistic encryption and how to play mental poker keeping secret all partial information,” in *14th ACM STOC*. ACM Press, May 1982, pp. 365–377.
9. A. C.-C. Yao, “Theory and applications of trapdoor functions (extended abstract),” in *23rd FOCS*. IEEE Computer Society Press, Nov. 1982, pp. 80–91.
10. V. Shoup, “Sequences of games: a tool for taming complexity in security proofs,” Cryptology ePrint Archive, Report 2004/332, 2004, <https://eprint.iacr.org/2004/332>.
11. M. Bellare, A. Desai, E. Jorjipii, and P. Rogaway, “A concrete security treatment of symmetric encryption,” in *38th FOCS*. IEEE Computer Society Press, Oct. 1997, pp. 394–403.
12. D. Dolev and A. Yao, “On the security of public key protocols,” *IEEE Transactions on Information Theory*, vol. 29, no. 2, pp. 198–208, 1983.
13. R. Canetti, “Universally composable security: A new paradigm for cryptographic protocols,” in *42nd FOCS*. IEEE Computer Society Press, Oct. 2001, pp. 136–145.
14. U. Maurer, “Constructive cryptography – a new paradigm for security definitions and proofs,” in *Theory of Security and Applications (TOSCA 2011)*, ser. Lecture Notes in Computer Science, S. Modersheim and C. Palamidessi, Eds., vol. 6993. Springer-Verlag, 4 2011, pp. 33–56.
15. M. Backes, B. Pfitzmann, and M. Waidner, “The reactive simulatability (rsim) framework for asynchronous systems,” *Information and Computation*, vol. 205, no. 12, pp. 1685–1720, 2007.
16. D. Hofheinz and V. Shoup, “GNU: A new universal composability framework,” *Journal of Cryptology*, vol. 28, no. 3, pp. 423–508, Jul. 2015.
17. R. Küsters, M. Tuengerthal, and D. Rausch, “The IITM model: A simple and expressive model for universal composability,” *Journal of Cryptology*, vol. 33, no. 4, pp. 1461–1584, Oct. 2020.
18. M. Barbosa, G. Barthe, K. Bhargavan, B. Blanchet, C. Cremers, K. Liao, and B. Parno, “SoK: Computer-aided cryptography,” in *2021 IEEE Symposium on Security and Privacy*. IEEE Computer Society Press, May 2021, pp. 777–795.
19. L. Lamport, “Proving the correctness of multiprocess programs,” *IEEE Transactions on Software Engineering*, vol. SE-3, no. 2, pp. 125–143, 1977.
20. B. Alpern and F. B. Schneider, “Defining liveness,” *Inf. Process. Lett.*, vol. 21, pp. 181–185, 1984. [Online]. Available: <https://api.semanticscholar.org/CorpusID:8707490>

21. M. R. Clarkson and F. B. Schneider, "Hyperproperties," in *CSF 2008 Computer Security Foundations Symposium*, A. Sabelfeld, Ed. IEEE Computer Society Press, 2008, pp. 51–65.
22. K. Gödel, *Zum intuitionistischen Aussagenkalkül*, Vienna, 1932.
23. T. Blyth, *Lattices and Ordered Algebraic Structures*. Springer London, 2005.
24. G. Grätzer, *Lattice Theory: Foundation*. Birkhäuser Basel, 2011. [Online]. Available: <https://doi.org/10.1007/978-3-0348-0018-1>
25. A. Horn, "Logic with truth values in a linearly ordered heyting algebra," *The Journal of Symbolic Logic*, vol. 34, no. 3, pp. 395–408, 1969. [Online]. Available: <http://www.jstor.org/stable/2270905>
26. A. Beckmann and N. Preining, "Linear kripke frames and gödel logics," *Journal of Symbolic Logic*, vol. 72, no. 1, p. 26–44, 2007.
27. G. Birkhoff, "Rings of sets," *Duke Mathematical Journal*, vol. 3, pp. 443–454, 1937. [Online]. Available: <https://api.semanticscholar.org/CorpusID:120818529>
28. Y. Dodis and J. P. Steinberger, "Message authentication codes from unpredictable block ciphers," in *CRYPTO 2009*, ser. LNCS, S. Halevi, Ed., vol. 5677. Springer, Heidelberg, Aug. 2009, pp. 267–285.
29. M. Bellare and P. Rogaway, "The security of triple encryption and a framework for code-based game-playing proofs," in *EUROCRYPT 2006*, ser. LNCS, S. Vaudenay, Ed., vol. 4004. Springer, Heidelberg, May / Jun. 2006, pp. 409–426.
30. M. Bellare and C. Namprempre, "Authenticated encryption: Relations among notions and analysis of the generic composition paradigm," in *ASIACRYPT 2000*, ser. LNCS, T. Okamoto, Ed., vol. 1976. Springer, Heidelberg, Dec. 2000, pp. 531–545.
31. J. H. An, Y. Dodis, and T. Rabin, "On the security of joint signature and encryption," in *EUROCRYPT 2002*, ser. LNCS, L. R. Knudsen, Ed., vol. 2332. Springer, Heidelberg, Apr. / May 2002, pp. 83–107.
32. D. Dolev, C. Dwork, and M. Naor, "Nonmalleable cryptography," *SIAM Review*, vol. 45, no. 4, pp. 727–784, 2003. [Online]. Available: <http://www.jstor.org/stable/25054463>
33. R. Canetti, S. Halevi, and J. Katz, "Chosen-ciphertext security from identity-based encryption," in *EUROCRYPT 2004*, ser. LNCS, C. Cachin and J. Camenisch, Eds., vol. 3027. Springer, Heidelberg, May 2004, pp. 207–222.
34. J. Katz and M. Yung, "Scalable protocols for authenticated group key exchange," in *CRYPTO 2003*, ser. LNCS, D. Boneh, Ed., vol. 2729. Springer, Heidelberg, Aug. 2003, pp. 110–125.
35. L. Lamport, "Constructing digital signatures from a one way function," *Technical Report SRI-CSL-98*, SRI International Computer Science Laboratory, October 1979.
36. R. C. Merkle, "A certified digital signature," in *CRYPTO'89*, ser. LNCS, G. Brassard, Ed., vol. 435. Springer, Heidelberg, Aug. 1990, pp. 218–238.
37. M. N. Wegman and J. Carter, "New hash functions and their use in authentication and set equality," *Journal of Computer and System Sciences*, vol. 22, no. 3, pp. 265–279, 1981.
38. G. J. Simmons, "Authentication theory/coding theory," in *CRYPTO'84*, ser. LNCS, G. R. Blakley and D. Chaum, Eds., vol. 196. Springer, Heidelberg, Aug. 1984, pp. 411–431.
39. B. Blanchet, "An Efficient Cryptographic Protocol Verifier Based on Prolog Rules," in *14th IEEE Computer Security Foundations Workshop (CSFW-14)*. Cape Breton, Nova Scotia, Canada: IEEE Computer Society, Jun. 2001, pp. 82–96.

40. C. J. Cremers, “Unbounded verification, falsification, and characterization of security protocols by pattern refinement,” in *Proceedings of the 15th ACM Conference on Computer and Communications Security*, ser. CCS ’08. New York, NY, USA: Association for Computing Machinery, 2008, p. 119–128. [Online]. Available: <https://doi.org/10.1145/1455770.1455787>
41. S. Meier, B. Schmidt, C. Cremers, and D. Basin, “The tamarin prover for the symbolic analysis of security protocols,” in *International conference on computer aided verification*. Springer, 2013, pp. 696–701.
42. G. Bana and H. Comon-Lundh, “A computationally complete symbolic attacker for equivalence properties,” in *Proceedings of the 2014 ACM SIGSAC Conference on Computer and Communications Security*, ser. CCS ’14. New York, NY, USA: Association for Computing Machinery, 2014, p. 609–620. [Online]. Available: <https://doi.org/10.1145/2660267.2660276>
43. D. Baelde, S. Delaune, C. Jacomme, A. Koutsos, and J.ALLEMAND, “The squirrel prover and its logic,” *ACM SIGLOG News*, vol. 11, no. 2, pp. 62–83, 2024.
44. B. Blanchet, “A computationally sound mechanized prover for security protocols,” in *2006 IEEE Symposium on Security and Privacy*, 2006, pp. 15 pp.–154.
45. G. Barthe, B. Grégoire, and S. Zanella Béguelin, “Formal certification of code-based cryptographic proofs,” in *Proceedings of the 36th Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, ser. POPL ’09. New York, NY, USA: Association for Computing Machinery, 2009, p. 90–101. [Online]. Available: <https://doi.org/10.1145/1480881.1480894>
46. G. Barthe, B. Grégoire, S. Heraud, and S. Zanella Béguelin, “Computer-aided security proofs for the working cryptographer,” in *CRYPTO 2011*, ser. LNCS, P. Rogaway, Ed., vol. 6841. Springer, Heidelberg, Aug. 2011, pp. 71–90.
47. M. Berg, “Formal verification of cryptographic security proofs,” Ph.D. dissertation, Saarland University, January 2013.
48. A. Petcher and G. Morrisett, “The foundational cryptography framework,” in *Proceedings of the 4th International Conference on Principles of Security and Trust - Volume 9036*. Berlin, Heidelberg: Springer-Verlag, 2015, p. 53–72. [Online]. Available: https://doi.org/10.1007/978-3-662-46666-7_4
49. N. Swamy, C. Hrițcu, C. Keller, A. Rastogi, A. Delignat-Lavaud, S. Forest, K. Bhargavan, C. Fournet, P.-Y. Strub, M. Kohlweiss, J.-K. Zinzindohoue, and S. Zanella-Béguelin, “Dependent types and multi-monadic effects in f*,” *SIGPLAN Not.*, vol. 51, no. 1, p. 256–270, Jan. 2016. [Online]. Available: <https://doi.org/10.1145/2914770.2837655>
50. D. A. Basin, A. Lochbihler, and S. R. Sefidgar, “CryptHOL: Game-based proofs in higher-order logic,” *Journal of Cryptology*, vol. 33, no. 2, pp. 494–566, Apr. 2020.
51. P. G. Haselwarter, E. Rivas, A. Van Muylder, T. Winterhalter, C. Abate, N. Sidorenco, C. Hrițcu, K. Maillard, and B. Spitters, “Ssprover: A foundational framework for modular cryptographic proofs in coq,” *ACM Trans. Program. Lang. Syst.*, vol. 45, no. 3, Jul. 2023. [Online]. Available: <https://doi.org/10.1145/3594735>
52. A. Lochbihler, S. R. Sefidgar, D. A. Basin, and U. Maurer, “Formalizing constructive cryptography using CryptHOL,” in *CSF 2019 Computer Security Foundations Symposium*, S. Delaune and L. Jia, Eds. IEEE Computer Society Press, 2019, pp. 152–166.
53. D. A. Basin, A. Lochbihler, U. Maurer, and S. R. Sefidgar, “Abstract modeling of system communication in constructive cryptography using CryptHOL,” in *CSF 2021 Computer Security Foundations Symposium*, R. Küsters and D. Naumann, Eds. IEEE Computer Society Press, 2021, pp. 1–16.

54. L. De Moura, S. Kong, J. Avigad, F. Van Doorn, and J. von Raumer, “The lean theorem prover (system description),” in *International Conference on Automated Deduction*. Springer, 2015, pp. 378–388.
55. G. D. Giacomo and M. Y. Vardi, “Linear temporal logic and linear dynamic logic on finite traces,” in *IJCAI 2013, Proceedings of the 23rd International Joint Conference on Artificial Intelligence, Beijing, China, August 3-9, 2013*, F. Rossi, Ed. IJCAI/AAAI, 2013, pp. 854–860.
56. A. Heyting, “Die formalen regeln der intuitionistischen logik,” *Sitzungsbericht PreuBische Akademie der Wissenschaften Berlin, physikalisch-mathematische Klasse II*, pp. 42–56, 1930.
57. A. Horn, “Free l-algebras,” *The Journal of Symbolic Logic*, vol. 34, no. 3, pp. 475–480, 1969. [Online]. Available: <http://www.jstor.org/stable/2270910>
58. O. M. D’Antona and V. Marra, “Computing coproducts of finitely presented gödel algebras,” *Annals of Pure and Applied Logic*, vol. 142, no. 1, pp. 202–211, 2006. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0168007206000029>
59. R. N. Almeida, “Colimits of heyting algebras through esakia duality,” 2026. [Online]. Available: <https://arxiv.org/abs/2402.08058>

A Mathematical Background

The powerset of a set A is denoted $\mathcal{P}(A)$, the empty set is denoted $\emptyset$.

An *order relation* on a set P is a relation on P that is reflexive, antisymmetric, and transitive. A *partially ordered set (poset)* $(P; \preceq)$ is a set P together with an order relation $\preceq$ on P . For every poset $(P; \preceq)$, the *dual poset* P^d is defined as $P^d := (P; \succeq)$ where $a \succeq b : \iff b \preceq a$. For every statement about P , there is an equivalent statement about P^d and vice versa.

Let $(P; \preceq)$ be a poset. For $a, b \in P$, we say a and b are *comparable* if $a \preceq b$ or $b \preceq a$, and *incomparable* otherwise. P is *totally ordered* if all elements in P are comparable. A subset $Q \subseteq P$ is a *chain* if Q is totally ordered. Q is an *antichain* if all $a, b \in Q$ are incomparable. $a \in P$ is a *minimal (maximal) element* of P if there is no $b \in P$ with $b \neq a$ and $b \preceq a$ ($a \preceq b$). $a \in P$ is the *least (greatest) element* of P if $a \preceq b$ ($b \preceq a$) for all $b \in P$. We write $\perp$ and $\top$ for the least and greatest elements of a poset, respectively.

For a subset $Q \subseteq P$, a is a *lower (upper) bound* of Q if $a \preceq b$ ($b \preceq a$) for all $b \in Q$. a is the *greatest lower bound (least upper bound)* of Q , denoted $\inf Q$ ($\sup Q$), if a is the greatest (least) element of the set of lower (upper) bounds of Q .

Definition 2. For two posets $(P; \preceq_1)$ and $(Q; \preceq_2)$ a function $f : P \rightarrow Q$ is called *monotone* if

$$a \preceq_1 b \Rightarrow f(a) \preceq_2 f(b) \text{ for all } a, b \in P.$$

f is called *antitone* if

$$a \preceq_1 b \Rightarrow f(b) \preceq_2 f(a) \text{ for all } a, b \in P.$$

Definition 3. A bijective mapping $\phi : P \rightarrow Q$ between two posets $(P; \preceq_1)$ and $(Q; \preceq_2)$ is an *order isomorphism* if

$$a \preceq_1 b \iff \phi(a) \preceq_2 \phi(b) \text{ for all } a, b \in P.$$

We say P and Q are *isomorphic* (we write $P \cong Q$), if there is an order isomorphism between them.

Definition 4. Let $(P; \preceq)$ be a poset and $a, b \in P$. If the set $\{a, b\}$ has a greatest lower bound, then it is called the *meet* of a and b , denoted $a \wedge b$. If $\{a, b\}$ has a least upper bound, then it is called the *join* of a and b , denoted $a \vee b$.

Definition 5. A poset $(L; \preceq)$ in which for every $a, b \in L$ the set $\{a, b\}$ has a meet, denoted $a \wedge b$, and a join, denoted $a \vee b$, is called a *lattice*. A lattice $(L; \preceq, \wedge, \vee)$ is *bounded* if it has a least and a greatest element.

In a lattice $(L; \preceq)$, meet and join can be understood as binary operations, satisfying the following axioms for any $a, b \in L$:

$$a = a \wedge a \quad (\text{I1}) \quad a = a \wedge (a \vee b) \quad (\text{A1})$$

$$a = a \vee a \quad (\text{I2}) \quad a = a \vee (a \wedge b). \quad (\text{A2})$$

Conversely, any algebra $(A; \wedge, \vee)$ with two associative and commutative³¹ operations satisfying the above axioms is a lattice, with order relation defined as (for any $a, b \in A$):

$$a \preceq b \iff a \vee b = b \iff a \wedge b = a. \quad (\text{O})$$

The above axioms imply:

$$a \wedge b \preceq a \preceq a \vee b \quad (\text{A3})$$

If $(L; \preceq)$ is a bounded lattice, the following holds for any $a \in L$:

$$a \vee \perp = a \quad \text{and} \quad a \wedge \perp = \perp \quad (\text{B})$$

$$a \wedge \top = a \quad \text{and} \quad a \vee \top = \top. \quad (\text{T})$$

Meet and join are monotone in both arguments: If $a \preceq b$ then

$$a \vee c \preceq b \vee c \quad \text{and} \quad a \wedge c \preceq b \wedge c. \quad (\text{M})$$

Definition 6. A lattice $(L; \preceq)$ is distributive if the following hold for any $a, b, c \in L$:

$$a \wedge (b \vee c) = (a \wedge b) \vee (a \wedge c) \quad \text{and} \quad a \vee (b \wedge c) = (a \vee b) \wedge (a \vee c) \quad (\text{D})$$

Definition 7. A subset B of an algebra $(A; f_1, \dots, f_k)$ is a subalgebra of A if B is closed under all operations, that is, if for all $1 \leq i \leq k$ and all $b_1, \dots, b_j \in B$, we have $f_i(b_1, \dots, b_j) \in B$ (where j is the arity of operation f_i). Note that this includes closure under nullary operations.

Definition 8. For a poset $(P; \preceq)$ and $Q \subseteq P$, the sets

$$Q^\uparrow := \{a \in P \mid b \preceq a \text{ for some } b \in Q\}$$

$$Q^\downarrow := \{a \in P \mid a \preceq b \text{ for some } b \in Q\}$$

are the upward closure and downward closure of Q , respectively. A subset $Q \subseteq P$ is an up-set (resp. down-set) of P if it is upward closed ($Q^\uparrow = Q$) (resp. downward closed ($Q^\downarrow = Q$)). Denote by $\text{Up}(P)$ ($\text{Dn}(P)$) the set of up-sets (down-sets) of P . An up-set (down-set) Q is a principal up-set (down-set) if it is the closure of a singleton, i.e., $Q = \{x\}^\uparrow$ ($Q = \{x\}^\downarrow$).

Definition 9. We call a poset $(P; \preceq)$ a forest poset if every principal up-set of P is a finite chain.

³¹ We will not make associativity and commutativity explicit.

Definition 10. An element a of a lattice $(L; \preceq)$ is join-irreducible if for all $B \subseteq L : a = \bigvee_{b \in B} b$ implies $a \in B$, i.e., a cannot be represented as the join of different elements. Denote by $\text{Ji}(L)$ the set of join-irreducible elements of L .

In a finite lattice, an element is join-irreducible if and only if it covers exactly one other element.

For a set $\mathcal{X}$, we write $\mathcal{X}^*$ for the set of finite sequences (or strings) over $\mathcal{X}$. We denote the empty string by ε and define $\mathcal{X}^+ := \mathcal{X}^* \setminus \{\varepsilon\}$. The concatenation of two strings $s, t \in \mathcal{X}^*$ is written as st . Concatenation defines an order relation on $\mathcal{X}^*$:

$$s \sqsubseteq t : \iff \exists u \in \mathcal{X}^* : s = tu.$$

Note that the empty string ε is the greatest element in this order.

B Appendix for Section 2 (Event Algebras)

B.1 Proofs for Section 2

Theorem 1. For any forest poset (see Definition 9) $(P; \preceq)$, the set $(\text{Dn}(P); \subseteq, \cup, \cap, \dot{\div}_P, P, \emptyset)$ is an event algebra, where $A \dot{\div}_P B := (A \setminus B)^\downarrow$.

Proof. It is easy to verify that $(\text{Dn}(F); \subseteq, \cap, \cup, P, \emptyset)$ is a bounded lattice. Note that for all $A, B \in \text{Dn}(F)$

$$(A \cup B)^\downarrow = A^\downarrow \cup B^\downarrow \tag{X}$$

which will be used several times below. We verify the axioms of Definition 1. Let $A, B, C \in \text{Dn}(P)$.

(E1): $A \dot{\div} A = (A \setminus A)^\uparrow = \emptyset$

(E2): We have

$$\begin{aligned} & (A \dot{\div}_P B) \cup A \\ &= (A \setminus B)^\downarrow \cup A && \text{(Def. } \dot{\div}_P \text{)} \\ &= (A \setminus B)^\downarrow \cup A^\downarrow && \text{(A down-set)} \\ &= ((A \setminus B) \cup A)^\downarrow && \text{(X)} \\ &= A^\downarrow = A && \text{(set theory, A down-set)} \end{aligned}$$

(E3): We have

$$\begin{aligned} & (A \dot{\div}_P B) \cup B \\ &= (A \setminus B)^\downarrow \cup B && \text{(Def. } \dot{\div}_P \text{)} \\ &= (A \setminus B)^\downarrow \cup B^\downarrow && \text{(B down-set)} \\ &= ((A \setminus B) \cup B)^\downarrow && \text{(X)} \\ &= (A \cup B)^\downarrow = A \cup B && \text{(set theory, A, B down-sets)} \end{aligned}$$

(E4):

$$\begin{aligned}
& (A \cup B) \dot{\div}_P C \\
&= ((A \cup B) \setminus C)^\downarrow && (\text{Def. } \dot{\div}_P) \\
&= ((A \setminus C) \cup (B \setminus C))^\downarrow && (\text{set theory}) \\
&= (A \setminus C)^\downarrow \cup (B \setminus C)^\downarrow && (X) \\
&= (A \dot{\div}_P C) \cup (B \dot{\div}_P C) && ((\text{Def. } \dot{\div}_P))
\end{aligned}$$

Note that so far we have not used that P is a forest. That is, we have shown that the down-sets on any poset are a dual Heyting algebra.

(E5): We need to prove $(A \dot{\div}_P B) \cap (B \dot{\div}_P A) = \emptyset$. Assume towards contradiction that $x \in (A \dot{\div}_P B) \cap (B \dot{\div}_P A) = (A \setminus B)^\downarrow \cap (B \setminus A)^\downarrow$. There must be y, z satisfying $x \preceq y, y \in A, y \notin B$ as well as $x \preceq z, z \in B, z \notin A$. Since $y, z \in \{x\}^\uparrow$ and P is a forest poset, we must have either $y \leq z$ or $z \leq y$. If $y \leq z$ then $z \in B, y \notin B$ contradicts $B \in \text{Dn}(P)$. Similarly, if $z \leq y$ then $y \in A, z \notin A$ contradicts $A \in \text{Dn}(P)$.

Theorem 2. For any finite³² event algebra E , $\text{Ji}(E)$ (see Definition 10) is a forest poset and we have

$$E \cong \text{Dn}(\text{Ji}(E)).$$

Proof. By Birkhoff's representation theorem for finite distributive lattices [27], we have $E \cong \text{Dn}(\text{Ji}(E))$. We need to verify that in the poset $\text{Ji}(E)$, every principal up-set is a finite chain. Since E is finite, so is $\text{Ji}(E)$ and hence every principal up-set of $\text{Ji}(E)$ is finite.

Let $x \in \text{Ji}(E)$ and consider $y, z \in \text{Ji}(E)$ with $x \leq y, x \leq z$, i.e., $y, z \in \{x\}^\uparrow$. Assume towards contradiction that y, z are incomparable. By Birkhoff's representation theorem, y and z can be represented as down-sets $Y = \{y\}^\downarrow, Z = \{z\}^\downarrow \in \text{Dn}(\text{Ji}(E))$. Since $y \not\leq z$, we have $y \notin Z$, hence $y \in Y \setminus Z$. Likewise, $z \in Z \setminus Y$. Moreover, since $x \leq y$ and $x \leq z$, we have $x \in (Y \setminus Z)^\downarrow$ and $x \in (Z \setminus Y)^\downarrow$ and hence $x \in (Y \setminus Z)^\downarrow \cap (Z \setminus Y)^\downarrow$. Consequently, $\emptyset \neq (Y \setminus Z)^\downarrow \cap (Z \setminus Y)^\downarrow$, which by Birkhoff's representation theorem is equivalent to $\perp \neq (y \dot{\div} z) \wedge (z \dot{\div} y)$, contradicting (E5).

B.2 More Examples of Event Algebras

In this section, we give several examples of event algebras, beyond the example in Figure 1: Boolean algebras, in particular σ -algebras, and safety properties.

³² Infinite event algebras are discussed in Section B.4. We state only the finite case here for simplicity and because (sequences of) finite considerations seem to suffice for practical applications.

Boolean Algebras. A bounded lattice $(L; \preceq)$ in which for every $a \in L$ there is an element $\bar{a} \in L$ such that $a \wedge \bar{a} = \perp$ and $a \vee \bar{a} = \top$ is called a *complemented lattice*. A distributive complemented lattice $(L; \preceq)$ is called a Boolean algebra. We show that Boolean algebras are a special case of event algebras:

Lemma 8. *Every Boolean algebra $(L; \preceq, \wedge, \vee, \neg, \top, \perp)$ is an event algebra, where $\div$ is defined as $a \div b := a \wedge \bar{b}$.*

Proof. We verify the axioms of Definition 1: Let $a, b, c \in L$. We have

(E1): $a \div a = a \wedge \bar{a} = \perp$

(E2):

$$\begin{aligned} (a \div b) \vee a &= (a \wedge \bar{b}) \vee a && \text{(Def. } \neg) \\ &= (a \vee a) \wedge (\bar{b} \vee a) && (D) \\ &= a \wedge (\bar{b} \vee a) && (I2) \\ &= a && (A1) \end{aligned}$$

(E3):

$$\begin{aligned} (a \div b) \vee b &= (a \wedge \bar{b}) \vee b && \text{(Def. } \neg) \\ &= (a \vee b) \wedge (\bar{b} \vee b) && (D) \\ &= (a \vee b) \wedge \top && \text{(Complement)} \\ &= (a \vee b) && (T) \end{aligned}$$

(E4):

$$\begin{aligned} (a \vee b) \div c &= (a \vee b) \wedge \bar{c} && \text{(Def. } \neg) \\ &= (a \wedge \bar{c}) \vee (b \wedge \bar{c}) && (D) \\ &= (a \div c) \vee (b \div c) && \text{(Def. } \neg) \end{aligned}$$

(E5):

$$\begin{aligned} (a \div b) \wedge (b \div a) &= (a \wedge \bar{b}) \wedge (b \wedge \bar{a}) && \text{(Def. } \neg) \\ &= (a \wedge \perp \wedge \bar{a}) && \text{(Complement)} \\ &= \perp && (B) \end{aligned}$$

σ -Algebras are Event Algebras. We now discuss in which sense probability-theoretic events are a special case of our notion of events. Let $(\Omega, \mathcal{F}, \text{Pr})$ be a probability space, i.e., let Ω be a non-empty set, $\mathcal{F} \subseteq \mathcal{P}(\Omega)$ be a σ -algebra over Ω , and Pr be a probability measure on $\mathcal{F}$. As every σ -algebra is also a Boolean algebra, σ -algebras are a special case of event algebras (by Lemma 8). In a σ -algebra, operation $\div$ is defined as $A \div B := A \cap (\Omega \setminus B)$.

This implies that any inequality in an event algebra may be understood as an inequality in *any* σ -algebra. For example, proving an inequality $e \preceq f \vee g$ that holds in any event algebra implies

$$\Pr(e) \leq \Pr(f) + \Pr(g)$$

in any probability space in which e, f , and g are defined, by the additivity of $\Pr$. Figure 11 gives a simple classification of event algebras relative to other types of lattices.

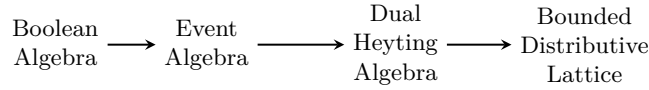


Fig. 11: The relation of event algebras relative to other types of algebras. Arrows should be read as "is a (proper) special case of".

Safety Properties. In formal methods, executions of concurrent programs are modeled as (infinite) sequences $t = s_1, s_2, \dots$ of program states from some set of states $\mathcal{X}$. A property (see e.g. [20]) is a set of such sequences. *Safety properties* are a special type of property, which can informally be characterized as prescribing that some "bad thing" does not happen. "Bad things" must be discrete and irremediable, meaning that there is a defined step in the sequence that causes the "bad thing" and that it is monotone in sequence extension. Clearly, such "bad things" are instances of our notion of events.

More precisely, a set of infinite sequences over $\mathcal{X}$ is a safety property if for every execution t not in the property, there is some finite prefix s of t such that every (infinite) extension of s is not in the property. Every safety property can be described by the (prefix-free) set of all such finite prefixes. That is, every safety property over $\mathcal{X}$ corresponds to an element of $\text{Dn}((\mathcal{X}^*; \sqsubseteq))$, which is an event algebra:

Lemma 9. *For every alphabet $\mathcal{X}$, the set $\text{Dn}((\mathcal{X}^*; \sqsubseteq))$ is an event algebra.*

Proof. We verify that the poset $(\mathcal{X}^*; \sqsubseteq)$ is a forest poset and conclude that $\text{Dn}((\mathcal{X}^*; \sqsubseteq))$ is an event algebra by Theorem 1. To show that every principal up-set is a chain, let $t, s, s' \in \mathcal{X}^*$ and assume $s \sqsubseteq t$ and $s' \sqsubseteq t$, hence $sr = t$ and $s'r' = t$. If $|s'| \leq |s|$ then $s' \sqsubseteq s$. If $|s| \leq |s'|$ then $s \sqsubseteq s'$. Thus, s and s' are comparable and therefore $\{t\}^\uparrow$ is a chain. Since t is a finite string, $\{t\}^\uparrow$ is also finite.

B.3 Appendix for Section 2.2 (Algebraic Properties of Event Algebras)

Lemma 1. *The following hold for any elements a, b, c of an event algebra:*

$$\begin{array}{ll}
(i) \ a \dot{\div} b \preceq a. & (v) \ a = (a \wedge b) \vee (a \dot{\div} b) \\
(ii) \ \text{If } a \preceq b \text{ then } a \dot{\div} c \preceq b \dot{\div} c. & (vi) \ (a \wedge b) \dot{\div} c = (a \dot{\div} c) \wedge (b \dot{\div} c) \\
(iii) \ a \preceq b \vee c \iff a \dot{\div} b \preceq c & (vii) \ a \dot{\div} (b \wedge c) = (a \dot{\div} b) \vee (a \dot{\div} c) \\
(iv) \ \text{If } a \preceq b \text{ then } c \dot{\div} b \preceq c \dot{\div} a. & (viii) \ a \dot{\div} (b \vee c) = (a \dot{\div} b) \wedge (a \dot{\div} c)
\end{array}$$

Proof. We first prove (i) to (iv) and then establish Lemma 2. Afterwards, we continue the proof of (v) to (viii) since it utilizes Lemma 2.

(i): Follows immediately from (E2) and (O).

(ii): Let $a \preceq b$. We have

$$\begin{aligned}
b \dot{\div} c &= (a \vee b) \dot{\div} c & (O) \\
&= (a \dot{\div} c) \vee (b \dot{\div} c) & (E4)
\end{aligned}$$

Hence, $a \dot{\div} c \preceq b \dot{\div} c$ by (O).

(iii): $a \preceq b \vee c$ implies $a \dot{\div} b \preceq (b \vee c) \dot{\div} b$ by Lemma 1.(ii). We have

$$\begin{aligned}
a \dot{\div} b &\preceq (b \vee c) \dot{\div} b = (b \dot{\div} b) \vee (c \dot{\div} b) & (E4) \\
&= \perp \vee (c \dot{\div} b) & (E1) \\
&= (c \dot{\div} b) & (B) \\
&\preceq c & (L.1.(i))
\end{aligned}$$

Conversely, $a \dot{\div} b \preceq c$ implies $(a \dot{\div} b) \vee c = c$, by (O). This implies $(a \dot{\div} b) \vee c \vee b = c \vee b$ whence by (E3) $a \vee c \vee b = c \vee b$. Using (O) yields $a \preceq b \vee c$.

(iv): Let $a \preceq b$. We have

$$\begin{aligned}
c &\preceq c \vee a & (M) \\
&= (c \dot{\div} a) \vee a & (E3) \\
&\preceq (c \dot{\div} a) \vee b & (\text{Ass.})
\end{aligned}$$

We obtain $c \dot{\div} b \preceq c \dot{\div} a$ by (Lemma 1.(iii)).

We now prove Lemma 2 and afterwards resume proving (v) to (viii).

Lemma 2. *Every event algebra is a distributive lattice.*

Proof. It suffices to show one of the distributive laws, the other law follows by duality.

Direction $a \vee (b \wedge c) \preceq (a \vee b) \wedge (a \vee c)$ holds in any lattice:

$$\begin{aligned}
a \vee (b \wedge c) &\preceq (a \vee (b \wedge c)) \wedge (a \vee (b \wedge c)) & (I1) \\
&\preceq (a \vee b) \wedge (a \vee c) & (M)
\end{aligned}$$

On the other hand, since $(a \vee b) \wedge (a \vee c) \preceq (a \vee b)$ (by A3), applying L.1.(iii) yields $((a \vee b) \wedge (a \vee c)) \dot{\div} a \preceq b$. Similarly, we can derive $((a \vee b) \wedge (a \vee c)) \dot{\div} a \preceq c$. Thus, $((a \vee b) \wedge (a \vee c)) \dot{\div} a$ is a lower bound of b and c , whence $((a \vee b) \wedge (a \vee c)) \dot{\div} a \preceq b \wedge c$. Again applying L.1.(iii) yields $(a \vee b) \wedge (a \vee c) \preceq a \vee (b \wedge c)$. $\square$

Having established [Lemma 2](#), we can complete the proof of [Lemma 1](#).
Proof of [Lemma 1](#) (Continued).

(v): We have

$$\begin{aligned}
(a \wedge b) \vee (a \dot{\div} b) &= (a \vee (a \dot{\div} b)) \wedge (b \vee (a \dot{\div} b)) && (D) \\
&= a \wedge (b \vee (a \dot{\div} b)) && (E2) \\
&= a \wedge (b \vee a) && (E3) \\
&= a && (A1)
\end{aligned}$$

(vi): We have

$$\begin{aligned}
&(a \dot{\div} c) \wedge (b \dot{\div} c) \\
&= (((a \dot{\div} b) \vee (a \wedge b)) \dot{\div} c) \\
&\quad \wedge (((b \dot{\div} a) \vee (a \wedge b)) \dot{\div} c) && (L.1.(v)) \\
&= ((a \dot{\div} b) \dot{\div} c \vee (a \wedge b) \dot{\div} c) \\
&\quad \wedge ((b \dot{\div} a) \dot{\div} c \vee (a \wedge b) \dot{\div} c) && (E4) \\
&= ((a \dot{\div} b) \dot{\div} c \wedge (b \dot{\div} a) \dot{\div} c) \vee (a \wedge b) \dot{\div} c && (D) \\
&\preceq ((a \dot{\div} b) \wedge (b \dot{\div} a)) \vee (a \wedge b) \dot{\div} c && (L.1.(i), M) \\
&\preceq \perp \vee (a \wedge b) \dot{\div} c = (a \wedge b) \dot{\div} c && (E5, B)
\end{aligned}$$

(vii): On the one hand, we have

$$\begin{aligned}
&(a \dot{\div} b) \vee (a \dot{\div} c) \\
&\preceq (a \dot{\div} (b \wedge c)) \vee (a \dot{\div} (b \wedge c)) && (L.1.(iv), A3) \\
&\preceq (a \dot{\div} (b \wedge c)) && (I2)
\end{aligned}$$

On the other hand:

$$\begin{aligned}
a &= a \wedge a && (I1) \\
&\preceq (a \vee b) \wedge (a \vee c) && (A3, M) \\
&= (a \dot{\div} b \vee b) \wedge (a \dot{\div} c \vee c) && (E3) \\
&= (a \dot{\div} b \wedge (a \dot{\div} c \vee b)) \vee (b \wedge (a \dot{\div} c \vee c)) && (D) \\
&\preceq (a \dot{\div} b) \vee (b \wedge (a \dot{\div} c \vee c)) && (A3, M) \\
&= (a \dot{\div} b) \vee (b \wedge (a \dot{\div} c)) \vee (b \wedge c) && (D) \\
&\preceq (a \dot{\div} b) \vee (a \dot{\div} c) \vee (b \wedge c) && (A3, M)
\end{aligned}$$

Applying [Lemma 1.\(iii\)](#) yields $a \dot{\div} (b \wedge c) \preceq (a \dot{\div} b) \vee (a \dot{\div} c)$.

(viii): We have

$$\begin{aligned}
&(a \dot{\div} b) \wedge (a \dot{\div} c) \\
&\preceq ((a \vee c) \dot{\div} b) \wedge ((a \vee b) \dot{\div} c) && (L.1.(ii), A3)
\end{aligned}$$

$$\begin{aligned}
&= ((a \dot{\div} c \vee c) \dot{\div} b) \wedge ((a \dot{\div} b \vee b) \dot{\div} c) & (E3) \\
&= ((a \dot{\div} c) \dot{\div} b \vee c \dot{\div} b) \wedge ((a \dot{\div} b) \dot{\div} c \vee b \dot{\div} c) & (E4) \\
&= (a \dot{\div} (b \vee c) \vee c \dot{\div} b) \wedge (a \dot{\div} (b \vee c) \vee b \dot{\div} c) & (L.11) \\
&= a \dot{\div} (b \vee c) \vee (c \dot{\div} b \wedge b \dot{\div} c) & (D) \\
&= a \dot{\div} (b \vee c) \vee \perp = a \dot{\div} (b \vee c) & (E5, B)
\end{aligned}$$

□

Corollary 1. *The following hold for any elements a, b, c of an event algebra:*

- (i) $a \dot{\div} b = \perp \iff a \preceq b$
- (ii) $a \dot{\div} b \preceq ((a \wedge c) \dot{\div} b) \vee (a \dot{\div} c)$
- (iii) $a \dot{\div} b \preceq (a \dot{\div} c) \vee (c \dot{\div} b)$

Proof. (i): Follows from Lemma 1.(iii) by choosing $c = \perp$.

(ii):

$$\begin{aligned}
a \dot{\div} b &\preceq ((a \wedge c) \vee (a \dot{\div} c)) \dot{\div} b & (L.1.(v)) \\
&\preceq ((a \wedge c) \dot{\div} b) \vee ((a \dot{\div} c) \dot{\div} b) & (E4) \\
&\preceq ((a \wedge c) \dot{\div} b) \vee (a \dot{\div} c) & (L.1.(i), M)
\end{aligned}$$

(iii): By Corollary 1.(ii), we have $a \dot{\div} b \preceq ((a \wedge c) \dot{\div} b) \vee (a \dot{\div} c)$. Applying Lemma 1.(ii), (A3) and (M) yields the claim.

□

We now discuss some further properties of event algebras. We begin by showing that a bounded lattice admits at most one operation $\dot{\div}$ satisfying the event algebra axioms:

Lemma 10. *$\dot{\div}$ is uniquely defined in every event algebra.*

Proof. Let E be a bounded lattice with two operations $\dot{\div}_1$ and $\dot{\div}_2$, both satisfying all event algebra axioms. Let $a, b \in E$. We have $a \dot{\div}_1 b = ((a \dot{\div}_2 b) \vee a) \dot{\div}_1 b = ((a \dot{\div}_2 b) \dot{\div}_1 b) \vee (a \dot{\div}_1 b) \preceq (a \dot{\div}_2 b) \vee (a \dot{\div}_1 b)$, whence $a \dot{\div}_2 b \preceq a \dot{\div}_1 b$. A symmetrical argument shows $a \dot{\div}_1 b \preceq a \dot{\div}_2 b$, hence $\dot{\div}_1 = \dot{\div}_2$.

The following lemma gives an alternative way to express chaining $\dot{\div}$ in the left argument:

Lemma 11. $a \dot{\div} (b \vee c) = (a \dot{\div} b) \dot{\div} c$

Proof. We have

$$\begin{aligned}
a &\preceq a \vee b & (A3) \\
&= (a \dot{\div} b) \vee b & (E3) \\
&\preceq (a \dot{\div} b) \vee b \vee c & (A3) \\
&= ((a \dot{\div} b) \dot{\div} c) \vee b \vee c & (E3)
\end{aligned}$$

Therefore, $(a \dot{\div} b) \dot{\div} c \preceq a \dot{\div} (b \vee c)$ by Lemma 1.(iii). On the other hand,

$$a \preceq a \vee (b \vee c) \quad (A3)$$

$$= (a \dot{\div} (b \vee c)) \vee (b \vee c) \quad (E3)$$

Applying Lemma 1.(iii) twice yields $a \dot{\div} b \preceq (a \dot{\div} (b \vee c)) \vee c$ and $(a \dot{\div} b) \dot{\div} c \preceq a \dot{\div} (b \vee c)$.

Both identities Lemmas 1.(vi) and 1.(viii) are equivalent to (E5) in the following sense:

Lemma 12. *Any lattice E satisfying (E1) - (E4) in which one of the following identities holds is an event algebra:*

$$(a \wedge b) \dot{\div} c = (a \dot{\div} c) \wedge (b \dot{\div} c) \quad (J1)$$

$$a \dot{\div} (b \vee c) = (a \dot{\div} b) \wedge (a \dot{\div} c) \quad (J2)$$

Proof. We only have to prove that (E5) is satisfied: Let $a, b \in E$.

(J2) implies (E5): We have

$$(a \dot{\div} b) \wedge (b \dot{\div} a) = (\perp \vee a \dot{\div} b) \wedge (\perp \vee b \dot{\div} a) \quad (B)$$

$$= (b \dot{\div} b \vee a \dot{\div} b) \wedge (a \dot{\div} a \vee b \dot{\div} a) \quad (E1)$$

$$= ((a \vee b) \dot{\div} b) \wedge ((a \vee b) \dot{\div} a) \quad (E4)$$

$$= ((a \vee b) \dot{\div} (a \vee b)) \quad (J2)$$

$$= \perp \quad (E1)$$

(J1) implies (E5): We have

$$(a \dot{\div} b) \wedge (b \dot{\div} a) = (\perp \vee a \dot{\div} b) \wedge (\perp \vee b \dot{\div} a) \quad (B)$$

$$= (a \dot{\div} a \vee a \dot{\div} b) \wedge (b \dot{\div} b \vee b \dot{\div} a) \quad (E1)$$

$$= (a \dot{\div} (a \wedge b)) \wedge (b \dot{\div} (a \wedge b)) \quad (L.1.(vii))$$

$$= ((a \wedge b) \dot{\div} (a \wedge b)) \quad (J1)$$

$$= \perp \quad (E1)$$

The proof of Lemma 1.(vii) and the results used therein only make use of event algebra axioms (E1) - (E4), therefore we can use it in the above.

That is, our choice of axioms is (unsurprisingly) not unique: One could choose Lemmas 1.(vi) and 1.(viii) as axioms and derive (E5) as a consequence.

B.4 Infinite Event Algebras

Theorem 2 only holds for finite distributive lattices. Indeed, an infinite event algebra may not have join-irreducibles. The result can be generalized if, instead of join-irreducibles, one considers the more general notion of *prime ideals*:

Definition 11. A subset $I \subset L$ of a lattice L is an *ideal* if I is a down-set ($I^\downarrow = I$) and I is closed under finite joins ($x, y \in I \implies x \vee y \in I$). An ideal I is *proper* if $I \neq L$. A proper ideal is *prime* if $x \wedge y \in I \implies (x \in I \text{ or } y \in I)$. Denote by $\text{Spec}(L)$ the set of prime ideals of L (called the *spectrum* of L).

Note that in a finite lattice, the prime ideals are exactly the principal down-sets, which (under inclusion) are order-isomorphic to the join-irreducibles.

The following is (the dual) of a result due to Horn [25]:

Theorem 3. A dual Heyting algebra (a bounded lattice satisfying (E1) up to (E4)) L is an event algebra iff $\text{Spec}(L)$ is a forest poset (allowing principal up-sets to be infinite chains).

Moreover (the following holds for every distributive lattice):

Lemma 13. Let E be an event algebra. For $a \in E$, let $\text{spec}(a) := \{I \in \text{Spec}(E) \mid a \notin I\}$. The map $\text{spec} : E \mapsto \text{Dn}(\text{Spec}(E))$ is injective.

Proof. We first verify that spec is well-defined: Let $a \in E, I, J \in \text{Spec}(E), a \notin I$. If $J \subset I$, also $a \notin J$, so $\text{spec}(a)$ indeed is a down-set.

spec is injective: Let $a, b \in E, a \neq b$. A standard result (e.g. [24, Corollary 117]) implies there is a prime ideal I with $a \in I, b \notin I$ (or vice-versa). Hence, $\text{spec}(a) \neq \text{spec}(b)$.

That is, every element a of an event algebra E can be represented as a down-set $\text{spec}(a)$ in a forest poset $(\text{Spec}(E))$ (if we relax the definition to allow infinite chains). Note that generally we do not have $\text{Dn}(\text{Spec}(E)) \cong E$ (as there may be down-sets not in the image of spec). However, the following holds: $\text{Dn}(\text{Im}(\text{spec}(E))) \cong E$, where $\text{Im}(\text{spec}(E))$ is the image of spec (which can be seen as the clopen down-sets of the Priestley topology generated by the sets $\text{spec}(a)$).

A more direct possibility for generalizing Theorem 2 to the infinite setting arises from observing that for any forest poset P , the set $\text{Dn}(P)$ is a *complete* lattice, i.e., arbitrary meets and joins (not just finite) exist. Since it is complete, an element of $\text{Dn}(P)$ is (completely) join-irreducible iff it is a principal down-set. Clearly, every element of $\text{Dn}(P)$ can be represented as the join of principal down-sets. If the definition of event algebras is extended to require completeness and that every element can be represented as the join of (completely) join-irreducible elements (such lattices are sometimes called *spatial*, see [24, Def.285]), one can again show $E \cong \text{Dn}(\text{Ji}(E))$. The prime ideals of a distributive spatial lattice are one-to-one with the (completely) join-irreducible elements. Note that any finite event algebra is complete and spatial, i.e., these additional requirements do not affect finite event algebras.

C Related Work (Continued)

We continue the discussion of related work (started in Section 1.4).

Cryptographic Frameworks The well-established reductionist approach to proving the security of cryptographic primitives is rooted in complexity theory and exemplified by foundational works such as Goldwasser and Micali [8] and Yao [9]. This approach resulted in game-based frameworks such as [3,10,11], in which the security of cryptographic primitives is proved via (polynomial-time) reductions. The symbolic analysis of security protocols was initiated by Dolev and Yao [12], where attackers’ capabilities are modeled as sequences of actions from a fixed set of actions. To address the need for composable security statements, several simulation-based frameworks have been proposed, such as [13,14,15,16,17]. As they are used to reason about idealized experiments or protocol executions, all such frameworks (reductionist, symbolic, and simulation-based) contain, implicitly, an event algebra.

Formal Verification of Cryptographic Protocols Many tools for the formal verification of cryptographic protocols have been developed, both for symbolic [39,40,41,42,43] and computational modeling [44,45,46,47,48,49,50,51]. Moreover, simulation-based frameworks have been formalized [52,53]. Such tools aim to capture the full expressiveness of corresponding frameworks and, consequently, are highly complex. [18] gives a relatively recent overview of the capabilities and differences of such tools.

Because our framework expresses cryptographic statements on the abstract level of universal event inequalities (see [Section 3.3](#)), no cryptography-specific verification infrastructure (for example, formalizing adversaries) is needed, and the statements can be formally verified in a general-purpose theorem prover. We demonstrate this by using `mathlib` in Lean [54] (see [Section F.3](#)).

Temporal Logic and Properties Our notion of events can be seen as an abstraction of safety trace properties [19,20,21] in formal methods. There, sets of “bad things” [20] characterizing safety properties may be regarded as a special instantiation of event algebras (obtained from the tree of finite sequences over the corresponding alphabet). Consequently, events are related to temporal logic. In [Section D](#), we define a logic of events and embed it into LTL_f (finite-trace LTL, see, e.g., [55]).

Algebra Our definition of event algebra ([Definition 1](#)) is the dual of Gödel algebras, a special type of Heyting algebra [23,24]. Heyting algebras were originally introduced to model a calculus for intuitionistic logic [56], while Gödel algebras (Heyting algebras that satisfy the dual of (E5)) were first defined by Gödel [22] to show that propositional intuitionistic logic is not finite-valued. The connection between Gödel algebras and forests was first observed in [25] and is discussed in [26], but, to the best of our knowledge, the interpretation as events has not been considered before. The connection between event algebras generated by a finite set X and the strictly monotone sequences of subsets of X offers a new perspective on the cardinality of free Gödel algebras [57].

D Events and Temporal Logic

A reader familiar with logic might ask why we do not simply use well-established temporal logic (such as LTL) to express the guarantees we are interested in. The main reason is minimality: our notion of events differs from arbitrary logical propositions in monotonicity, since events do not "unoccur". That is, using full-fledged propositional LTL (or even finite-trace LTL (denoted LTL_f)) to model events would be too expressive and therefore introduce unnecessary structure. Moreover, event algebras are locally finite (i.e., event algebras generated by a finite set are finite), while this is not the case for (the algebra induced by) (unbounded) LTL_f .

To make the relation between event algebras and temporal logic clear, we will define a minimal logic of events (EL) and show how it can be embedded into LTL_f . We will show that this logic exactly expresses the logic of events in the sense that the Lindenbaum-Tarski algebra of EL is an event algebra.

Event Logic. We define a (propositional) logic of events which we call EL and relate it to [Definition 1](#) and other logics. Let $\mathcal{X}$ be a set of propositional symbols.

Definition 12 (EL syntax). *We define the set of formulas $\mathcal{F}_{EL}$ recursively by:*

1. $\mathcal{X} \subseteq \mathcal{F}_{EL}$
2. *If $\phi, \psi \in \mathcal{F}_{EL}$ then $(\phi \wedge \psi), (\phi \vee \psi), (\phi \dot{\vee} \psi) \in \mathcal{F}_{EL}$.*

Elements of $\mathcal{F}_{EL}$ are interpreted over $\mathbf{sMonSeq}(\mathcal{X})$, the set of strictly monotone (nonempty) sequences over $\mathcal{P}(X)$:

Definition 13. *The set of nonempty strictly monotone sequences over subsets of $\mathcal{X}$ is defined as:*

$$\mathbf{sMonSeq}(\mathcal{X}) := \{s \in (\mathcal{P}(X))^+ \mid i < j \rightarrow s_i \subset s_j\}.$$

Definition 14 (EL semantics). *For $s \in \mathbf{sMonSeq}(\mathcal{X})$, $1 \leq l \leq |s|$ let $s_l := s_1, \dots, s_l$. We define the satisfaction relation $\models \subseteq \mathbf{sMonSeq}(\mathcal{X}) \times \mathcal{F}_{EL}$ as:*

$$\begin{aligned} s \models x &: \iff \exists i : x \in s_i \\ s \models (\phi \wedge \psi) &: \iff s \models \phi \text{ and } s \models \psi \\ s \models (\phi \vee \psi) &: \iff s \models \phi \text{ or } s \models \psi \\ s \models (\phi \dot{\vee} \psi) &: \iff \exists i : s_{i]} \models \phi \text{ and } s_{i]} \not\models \psi \end{aligned}$$

Relation to Event Algebras. $\models$ is monotone in sequence extension:

Lemma 14. *For any $s, t \in \mathbf{sMonSeq}(\mathcal{X})$ such that $s \sqsubseteq t$ and any $\phi \in \mathcal{F}_{EL}$ if $s \models \phi$ then $t \models \phi$.*

Proof. By structural induction on ϕ .

$\models$ can be extended to formulas in the usual way: For $\phi, \psi \in \mathcal{F}_{\text{EL}}$ we have

$$\phi \models \psi : \iff \forall s \in \mathbf{sMonSeq}(\mathcal{X}) : \text{if } s \models \phi \text{ then } s \models \psi.$$

Furthermore, let $\phi \equiv \psi : \iff \phi \models \psi$ and $\psi \models \phi$. $\equiv$ is an equivalence relation. The following theorem shows that the Lindenbaum-Tarski Algebra of EL is an event algebra:

Theorem 4. $\mathcal{F}_{\text{EL}}/\equiv$ is an event algebra.

Proof. Let $F, G, H \in \mathcal{F}_{\text{EL}}$. We begin by establishing the following:

$$F \dot{\vee} G \models F \tag{R1}$$

Let $s \in \mathbf{sMonSeq}(\mathcal{X})$. If $s \models (F \dot{\vee} G)$ then by Definition 14 there is some i such that $s_{[i]} \models F$ and $s_{[i]} \not\models G$. $s_{[i]} \models F$ implies $s \models F$ by Lemma 14. Moreover, the following holds:

$$F \models F \vee G \tag{R2}$$

$s \models F$ implies ($s \models F$ or $s \models G$), hence $s \models F \vee G$, by Definition 14.

Now we verify the axioms of Definition 1 for $\mathcal{F}_{\text{EL}}/\equiv$ by proving corresponding equivalences:

$F \dot{\vee} F \equiv \perp$ (E1) : We have by Def. 14 $s \models F \dot{\vee} F \iff \exists i : s_{[i]} \models F$ and $s_{[i]} \not\models F$.

This is not possible, hence $s \not\models F \dot{\vee} F$ for any s , i.e. $(F \dot{\vee} F) \equiv \perp$.

$F \equiv (F \dot{\vee} G) \vee F$ (E2) : $F \models (F \dot{\vee} G) \vee F$ follows directly from (R2). Next, we show $(F \dot{\vee} G) \vee F \models F$: $s \models (F \dot{\vee} G) \vee F$ implies $s \models (F \dot{\vee} G)$ or $s \models F$ by Def. 14. In the second case we are done. If $s \models (F \dot{\vee} G)$ we can apply (R1) to obtain the statement.

$F \vee G \equiv (F \dot{\vee} G) \vee G$ (E3) : We first show $F \vee G \models (F \dot{\vee} G) \vee G$: From $s \models F \vee G$ follows $s \models F$ or $s \models G$ by Def. 14. $s \models G$ implies $s \models G$ or $s \models (F \dot{\vee} G)$, hence $s \models (F \dot{\vee} G) \vee G$, again by Def. 14. If $s \models F$ then we have two sub-cases: If there is an i s.t. $s_{[i]} \models G$, then also $s \models G$ by Lemma 14 and the statement follows from (R2). If there is no i s.t. $s_{[i]} \models G$ then we have $s \not\models G$ from which we obtain $s \models (F \dot{\vee} G)$, applying Def. 14. Finally this implies $s \models (F \dot{\vee} G) \vee G$ by (R2).

Next, we show $(F \dot{\vee} G) \vee G \models F \vee G$: By Def. 14, $s \models (F \dot{\vee} G) \vee G$ implies $s \models (F \dot{\vee} G)$ or $s \models G$. If $s \models (F \dot{\vee} G)$ we have $s \models F$ by (R1) and derive $s \models F \vee G$ by (R2). If $s \models G$, $s \models F \vee G$ follows again by (R2).

$(F \vee G) \dot{\vee} H \equiv (F \dot{\vee} H) \vee (G \dot{\vee} H)$ (E4) : Let $s \models (F \vee G) \dot{\vee} H$. Then, by Def. 14, $\exists i : (s_{[i]} \models (F \vee G) \text{ and } s_{[i]} \not\models H)$. This implies $\exists i : (s_{[i]} \models F \text{ and } s_{[i]} \not\models H)$ or $\exists i : (s_{[i]} \models G \text{ and } s_{[i]} \not\models H)$. Applying Def. 14 yields $s \models F \dot{\vee} H$ or $s \models G \dot{\vee} H$, and finally $s \models (F \dot{\vee} H) \vee (G \dot{\vee} H)$.

Conversely, let $s \models (F \dot{\vee} H) \vee (G \dot{\vee} H)$. Applying Def. 14, we obtain $\exists i : (s_{[i]} \models F \text{ and } s_{[i]} \not\models H)$ or $\exists i : (s_{[i]} \models G \text{ and } s_{[i]} \not\models H)$. By (R2), this implies $\exists i : (s_{[i]} \models (F \vee G) \text{ and } s_{[i]} \not\models H)$ or $\exists i : (s_{[i]} \models (F \vee G) \text{ and } s_{[i]} \not\models H)$. This is equivalent to $\exists i : (s_{[i]} \models (F \vee G) \text{ and } s_{[i]} \not\models H)$ and applying Def. 14 yields $s \models (F \vee G) \dot{\vee} H$.

$(F \dot{\div} G) \wedge (G \dot{\div} F) \equiv \perp$ (**E5**) : Assume $s \models (F \dot{\div} G) \wedge (G \dot{\div} F)$. Then by Def. 14, there must be i and j such that $s_{i]} \models F$ and $s_{i]} \not\models G$ as well as $s_{j]} \models G$ and $s_{j]} \not\models F$. If $i = j$, we directly obtain a contradiction: $s_{i]} \models F$ and $s_{i]} \not\models F$. If $i < j$, then from $s_{i]} \models F$ follows $s_{j]} \models F$ by Lemma 14, a contradiction to $s_{j]} \not\models F$. Case $j < i$ is symmetrical. Hence, $s \not\models (F \dot{\div} G) \wedge (G \dot{\div} F)$ for all s , i.e. $(F \dot{\div} G) \wedge (G \dot{\div} F) \equiv \perp$.

Relation to Other Logics. EL can be embedded into LTL_f (finite-trace LTL). We show how to map EL formulas to LTL_f : We define a map $\alpha : \mathcal{F}_{\text{EL}} \rightarrow \mathcal{F}_{\text{LTL}_f}$ recursively as:

$$\begin{aligned} x &\mapsto x \\ \phi \wedge \psi &\mapsto \alpha(\phi) \wedge \alpha(\psi) \\ \phi \vee \psi &\mapsto \alpha(\phi) \vee \alpha(\psi) \\ \phi \dot{\div} \psi &\mapsto \neg\alpha(\psi) \mathbf{U}(\neg\alpha(\psi) \wedge \alpha(\phi)), \end{aligned}$$

where $\mathbf{U}$ is the "(strong) until" operator. Moreover, we define $\beta : \mathcal{F}_{\text{LTL}_f} \rightarrow \mathcal{F}_{\text{LTL}_f}$ as $\phi \mapsto \mathbf{F}(\phi)$, where $\mathbf{F}$ is the "finally" operator. Since $\text{sMonSeq}(\mathcal{X}) \subseteq \mathcal{P}(\mathcal{X})^*$ any EL structure is an LTL_f structure. We have for all s and ϕ

$$s \models \phi \iff s \models \beta \circ \alpha(\phi)$$

(for a standard LTL_f syntax and semantics). Since for any finite subset of $\mathcal{X}$, the set $\text{sMonSeq}(\mathcal{X})$ is finite, EL can be embedded into propositional logic (incurring an exponential blow-up in the number of variables).

E Appendix for Section 3 (Principles of Event-Based Reasoning)

E.1 Correctness Guarantees.

Guarantees of the type that a certain event f *does* occur are often called functionality or correctness guarantees. To be useful, such a guarantee should be relative to another event e , i.e., f is guaranteed to occur before (or until) e occurs. This can, again, be stated as the inequality

$$e \preceq f.$$

Here, e could be a timing event or some other event that is assumed to occur. Guarantees of the form that an event occurs *eventually*, some time in the future, are often called *liveness* guarantees. However, without further assumptions on the environment (e.g., bounded message delay), such guarantees are effectively meaningless. Many such assumptions implicitly define an event e , for example, the event that all channels in a protocol have delivered their messages.

This shows that guarantees of very different forms (e.g., security and correctness guarantees) are actually of the same form: One needs to prove a certain

event inequality (e.g., $e \preceq f$) in the considered event algebra or, equivalently, one needs to prove that a certain event (e.g., $f \dot{-} e$) does not occur.

Typically, the guarantees one wants to give are more complex combinations of several events. For example, in a distributed system (e.g., a blockchain), one can consider a separate failure event for every computer in the system, and one wants to guarantee that a “bad transaction” event does not occur unless more than a certain number k of computer failure events have occurred (which itself is an event).

Consider the following, slightly more involved example: in the context of a (single-use) communication channel C , one can consider the event $\text{In}(m)$ that message m is input to C and the event $\text{Out}(m)$ that message m is output from C . One can also consider the event $\text{Time}(t)$ that time t (for some notion of time, e.g., UTC-time) is reached. The inequality

$$\text{Time}(t + \delta) \dot{-} \text{Out}(m) \preceq \text{Time}(t) \dot{-} \text{In}(m)$$

states that “*time $t + \delta$ will not pass without the message m being output, unless time t has passed without the message having been input*”. Stated differently: “*If the message is input before t , then it is output before $t + \delta$* .” If we consider t as a parameter, then this expresses (one form of) the guarantee of a delay- δ channel. We leave a systematic treatment of timing events as future work.

E.2 Free Event Algebras.

We will prove statements that hold in *any* event algebra. As explained in [Section 3.3](#), our approach is, in a particular setting, to identify the set $\mathcal{A}$ of relevant events (variables) and then prove inequalities without assuming any structure beyond the event algebra axioms on this set. That is, we make statements in the *free event algebra generated by $\mathcal{A}$* . It is well-known in universal algebra that the equational theory of a variety coincides with that of its free algebras. Any inequality proved in the free event algebra generated by $\mathcal{A}$ holds in any concrete event algebra E under any assignment of the generators to elements of E . This is what allows us to abstract away the underlying model.

The event algebra generated by a finite set $\mathcal{A}$ can be described by the forest $\text{sMonSeq}(\mathcal{A})$ of nonempty strictly monotone sequences over subsets of $\mathcal{A}$ (see [Definition 13](#)). That is, the event algebra generated by $\mathcal{A}$ is isomorphic to $\text{Dn}((\text{sMonSeq}(\mathcal{A}); \sqsubseteq))$. [Figure 12](#) shows the event algebra generated by $\{e\}$ and the forest $\text{sMonSeq}(\{e\})$.

Horn’s [\[57\]](#) characterization of free Gödel algebras (the dual of free event algebras) is dually isomorphic to $\text{Dn}((\text{sMonSeq}(\mathcal{X}); \sqsubseteq))$ (for finite $\mathcal{X}$). [\[58\]](#) describes (free) Gödel algebras via so-called *merged shuffles*. [\[59\]](#) gives a characterization of infinitely generated free Gödel algebras.

E.3 Proofs for [Section 3](#) (Principles of Event-Based Reasoning)

We begin by proving a technical lemma that is used later:

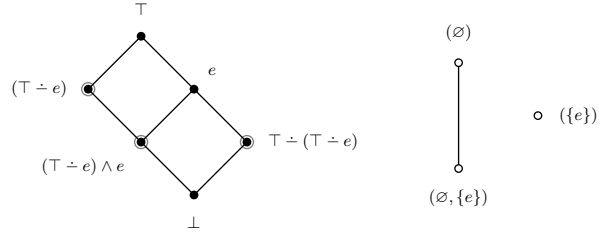


Fig. 12: An event algebra E (filled dots) of down-sets of a forest (open dots). The join-irreducibles of E are circled.

Lemma 15. *Let $\mathcal{L}$ be a lattice, X a finite set and $A, B : X \rightarrow \mathcal{L}$. We have*

$$\bigvee_{x \in X} (A(x) \vee B(x)) \preceq \bigvee_{x \in X} A(x) \vee \bigvee_{x \in X} B(x)$$

Proof.

$$\begin{aligned} & \bigvee_{x \in X} (A(x) \vee B(x)) \\ &= \bigvee_{\substack{x, x' \in X \\ x=x'}} (A(x) \vee B(x')) \\ &\preceq \bigvee_{\substack{x, x' \in X \\ x=x'}} (A(x) \vee B(x')) \vee \bigvee_{\substack{x, x' \in X \\ x \neq x'}} (A(x) \vee B(x')) \\ &= \bigvee_{x, x' \in X} (A(x) \vee B(x')) = \bigvee_{x \in X} A(x) \vee \bigvee_{x \in X} B(x) \end{aligned}$$

Lemma 3. *For any event algebra E , any sets X , any $E, F, G : X \rightarrow E$, we have*

$$\text{nPre}_{E,G}^X \preceq \text{nPre}_{E,F}^X \vee \text{nPre}_{F,G}^X$$

Proof. We have

$$\begin{aligned} \text{nPre}_{E,G}^X &= \bigvee_x (E_x \dot{-} G_x) \\ &\preceq \bigvee_x ((E_x \dot{-} F_x) \vee (F_x \dot{-} G_x)) && \text{(C.1.(iii))} \\ &\preceq \bigvee_x (E_x \dot{-} F_x) \vee \bigvee_x (F_x \dot{-} G_x) && \text{(L.15)} \\ &= \text{nPre}_{E,F}^X \vee \text{nPre}_{F,G}^X \end{aligned}$$

F Appendix for Section 4 (Cryptographic Applications)

F.1 Game-Based vs. Event-Based Hash-then-Sign.

The following sketches a typical game-based formulation of the security of the Hash-then-Sign paradigm (cf. Section 4.5) and its proof, with the purpose of illustrating the event-algebraic content embedded in standard reduction proofs.

A game-based statement is roughly: *If the inner signature scheme is existentially unforgeable and the hash function is collision-resistant, then the combined scheme is existentially unforgeable*, where both existential unforgeability and collision resistance are defined as no efficient adversary having more than sufficiently low (more than negligible, or more than an explicit threshold) advantage in a corresponding game. At a high level, the security proof proceeds as follows: Two reduction algorithms $\mathcal{B}$ and $\mathcal{C}$ are defined that transform an adversary $\mathcal{A}$ interacting with the existential unforgeability game for the combined scheme ($\mathbf{G}_1$) into an adversary for the collision-finding game for the hash function ($\mathbf{G}_2$) or the existential unforgeability game for the inner signature scheme ($\mathbf{G}_3$), respectively. Here, it is necessary to argue that both $\mathcal{B}$ and $\mathcal{C}$, together with their corresponding games, perfectly simulate the outer game. Subsequently, it is argued that if $\mathcal{A}$ has a low advantage in winning $\mathbf{G}_1$, then $\mathcal{B}$ (using $\mathcal{A}$ as a subroutine) has a low advantage in winning $\mathbf{G}_2$, or $\mathcal{C}$ (using $\mathcal{A}$ as a subroutine) has a low advantage in winning $\mathbf{G}_3$.

Figure 13 illustrates how the system in Figure 7 itself can be understood as the game-based reduction: Games $\mathbf{G}_1$ up to $\mathbf{G}_3$ are indicated as boxes, reduction algorithm $\mathcal{B}$ corresponds to $\mathbf{C}$ together with $\mathbf{G}_3$, and $\mathcal{C}$ corresponds to $\mathbf{C}$ together with $\mathbf{G}_2$.

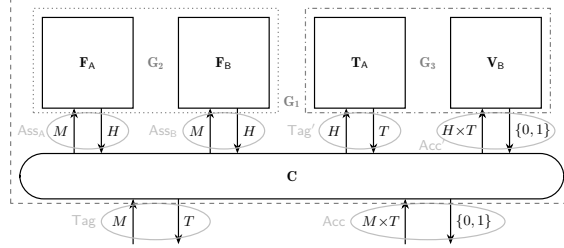


Fig. 13: A variant of Figure 7. Games $\mathbf{G}_1, \mathbf{G}_2$, and $\mathbf{G}_3$ correspond to the boxes with dashed, dotted and dash-dot lines, respectively.

A typical step in a game-based security argument would proceed along the lines of “Assume adversary $\mathcal{A}$ wins game $\mathbf{G}_1$ (produces a forgery for message m and signature s) and let $m_1, \dots, m_l$ be the messages submitted to the signing oracle by $\mathcal{A}$. Then we must have either $H(m) = H(m_i)$ for some i , or not. ...”. Note that this is an event-algebraic step: The case distinction uses the assumption that game $\mathbf{G}_1$, by construction, has the properties that $\mathbf{S}_A^{(7)}$ and $\mathbf{S}_B^{(7)}$

can't occur, which implies that hashes are only computed (within the game) for messages that have been input to the signing oracle or verification oracle.

F.2 Proofs for Section 4 (Further Cryptographic Applications)

Lemma 4. *For any event algebra E , any finite sets X, Y , any $E, F : X \rightarrow E$, $G, H : X \times Y \rightarrow E$ we have*

$$\text{nPre}_{E,F}^X \preceq \text{pnPre}_{G,H}^{X,Y} \vee \text{P1}_{H,F}^{X,Y} \vee \text{P2}_{E,G}^{X,Y}$$

where $\text{P1}_{H,F}^{X,Y} := \bigvee_{x,y} (H_{x,y} \dot{-} F_x)$ and $\text{P2}_{E,G}^{X,Y} := \bigvee_x (E_x \dot{-} \bigvee_{y'} G_{x,y'})$

Proof. We have

$$\begin{aligned} \text{nPre}_{E,F}^X &= \bigvee_x (E_x \dot{-} F_x) \\ &\preceq \bigvee_x \left(\left(E_x \dot{-} \bigvee_{y'} G_{x,y} \right) \vee \left(\bigvee_{y'} G_{x,y} \dot{-} F_x \right) \right) && (\text{C.1.(iii)}) \\ &\preceq \bigvee_x \left(E_x \dot{-} \bigvee_{y'} G_{x,y} \right) \vee \bigvee_{x,y} (G_{x,y} \dot{-} F_x) && (E4, \text{L.15}) \\ &= \text{P2}_{E,G}^{X,Y} \vee \bigvee_{x,y} (G_{x,y} \dot{-} F_x) \end{aligned}$$

Furthermore,

$$\begin{aligned} &\bigvee_{x,y} (G_{x,y} \dot{-} F_x) \\ &\preceq \bigvee_{x,y} \left(\left(G_{x,y} \dot{-} \bigvee_{y'} H_{x,y} \right) \vee \left(\left(\bigvee_{y'} H_{x,y} \dot{-} F_x \right) \right) \right) && (\text{C.1.(iii)}) \\ &\preceq \bigvee_{x,y} \left(G_{x,y} \dot{-} \bigvee_{y'} H_{x,y} \right) \vee \bigvee_{x,y} (H_{x,y} \dot{-} F_x) && (E4, \text{L.15}) \\ &= \text{pnPre}_{G,H}^{X,Y} \vee \text{P1}_{H,F}^{X,Y} \end{aligned}$$

The result is obtained by combining the above inequalities using monotonicity of $\vee$ (M).

Lemma 5. *For any event algebra E , any finite sets X, Y, Z , any $E, F : X \times Z \rightarrow E$, $G, H : X \times Y \rightarrow E$, $I, J : Y \times Z \rightarrow E$ we have*

$$\text{pnPre}_{E,F}^{X,Z} \preceq \text{Agr}_{G,H}^{X,Y} \vee \text{pnPre}_{I,J}^{Y,Z} \vee \text{P1}_{F,H,J}^{X,Y,Z} \vee \text{P2}_{E,G,I}^{X,Y,Z}$$

$$\text{where } P1_{F,H,J}^{X,Y,Z} := \bigvee_{x,y,z} ((H_{x,y} \wedge J_{y,z}) \dot{\div} F_{x,z}) \vee \bigvee_{y,z} \left(J_{y,z} \dot{\div} \bigvee_x H_{x,y} \right)$$

$$\text{and } P2_{E,G,I}^{X,Y,Z} := \bigvee_{x,z} \left(E_{x,z} \dot{\div} \bigvee_y (G_{x,y} \wedge I_{y,z}) \right)$$

Proof. Let $Q := \bigvee_{x,y,z} ((H_{x,y} \wedge J_{y,z}) \dot{\div} F_{x,z})$ and $R := \bigvee_{y,z} (J_{y,z} \dot{\div} \bigvee_x H_{x,y})$.
Note that $P1_{F,H,J}^{X,Y,Z} = Q \vee R$.

We have

$$\begin{aligned} \text{pnPre}_{E,F}^{X,Z} &= \bigvee_{x,z} \left(E_{x,z} \dot{\div} \bigvee_{z'} F_{x,z'} \right) \\ &\preceq \bigvee_{x,z} (E_{x,z} \dot{\div} F_{x,z}) && \text{(L.1.(iv), A3)} \\ &\preceq \bigvee_{x,z} \left(\left(E_{x,z} \dot{\div} \bigvee_y (G_{x,y} \wedge I_{y,z}) \right) \right. \\ &\quad \left. \vee \left(\bigvee_y (G_{x,y} \wedge I_{y,z}) \dot{\div} F_{x,z} \right) \right) && \text{(C.1.(iii))} \\ &\preceq \bigvee_{x,z} \left(E_{x,z} \dot{\div} \bigvee_y (G_{x,y} \wedge I_{y,z}) \right) \\ &\quad \vee \bigvee_{x,y} \left(\bigvee_y (G_{x,y} \wedge I_{y,z}) \dot{\div} F_{x,z} \right) && \text{(L.15)} \\ &= P2_{E,G,I}^{X,Y,Z} \vee \bigvee_{x,y} \left(\bigvee_y (G_{x,y} \wedge I_{y,z}) \dot{\div} F_{x,z} \right) \end{aligned}$$

Furthermore,

$$\begin{aligned} &\bigvee_{x,y} \left(\bigvee_y (G_{x,y} \wedge I_{y,z}) \dot{\div} F_{x,z} \right) \\ &= \bigvee_{x,y,z} ((G_{x,y} \wedge I_{y,z}) \dot{\div} F_{x,z}) && \text{(E4)} \\ &\preceq \bigvee_{x,y,z} \left((G_{x,y} \wedge I_{y,z}) \dot{\div} \bigvee_{z'} J_{y,z'} \right. \\ &\quad \left. \vee \left(G_{x,y} \wedge I_{y,z} \wedge \bigvee_{z'} J_{y,z'} \right) \dot{\div} F_{x,z} \right) && \text{(C.1.(ii))} \\ &\preceq \bigvee_{x,y,z} \left(I_{y,z} \dot{\div} \bigvee_{z'} J_{y,z'} \right) \end{aligned}$$

$$\begin{aligned}
& \vee \left(G_{x,y} \wedge \bigvee_{z'} J_{y,z'} \right) \dot{\div} F_{x,z} \quad (\text{L.1.}(ii), A3, M) \\
& \preceq \bigvee_{y,z} \left(I_{y,z} \dot{\div} \bigvee_{z'} J_{y,z'} \right) \\
& \vee \bigvee_{x,y} \left(\left(G_{x,y} \wedge \bigvee_z J_{y,z} \right) \dot{\div} F_{x,z} \right) \quad (\text{L.15}) \\
& = \bigvee_{y,z} \left(I_{y,z} \dot{\div} \bigvee_{z'} J_{y,z'} \right) \\
& \vee \bigvee_{x,y,z} ((G_{x,y} \wedge J_{y,z}) \dot{\div} F_{x,z}) \quad (D, E4) \\
& = \text{pnPre}_{I,J}^{Y,Z} \vee \bigvee_{x,y,z} ((G_{x,y} \wedge J_{y,z}) \dot{\div} F_{x,z})
\end{aligned}$$

Moreover,

$$\begin{aligned}
& \bigvee_{x,y,z} ((G_{x,y} \wedge J_{y,z}) \dot{\div} F_{x,z}) \\
& \preceq \bigvee_{x,y,z} \left((G_{x,y} \wedge J_{y,z}) \dot{\div} \bigvee_{x'} H_{x',y} \right) \\
& \vee \left(G_{x,y} \wedge J_{y,z} \wedge \bigvee_{x'} H_{x',y} \right) \dot{\div} F_{x,z} \quad (\text{C.1.}(ii)) \\
& \preceq \bigvee_{x,y,z} \left(J_{y,z} \dot{\div} \bigvee_{x'} H_{x',y} \right) \\
& \vee \left(G_{x,y} \wedge J_{y,z} \wedge \bigvee_{x'} H_{x',y} \right) \dot{\div} F_{x,z} \quad (\text{L.1.}(ii), M, A3) \\
& \preceq \bigvee_{y,z} \left(J_{y,z} \dot{\div} \bigvee_x H_{x,y} \right) \\
& \vee \bigvee_{x,y,z} \left(\left(G_{x,y} \wedge J_{y,z} \wedge \bigvee_{x'} H_{x',y} \right) \dot{\div} F_{x,z} \right) \quad (\text{L.15}) \\
& = R \vee \bigvee_{x,y,z} \left(\left(G_{x,y} \wedge J_{y,z} \wedge \bigvee_{x'} H_{x',y} \right) \dot{\div} F_{x,z} \right)
\end{aligned}$$

Finally,

$$\bigvee_{x,y,z} \left(\left(G_{x,y} \wedge J_{y,z} \wedge \bigvee_{x'} H_{x',y} \right) \dot{\div} F_{x,z} \right)$$

$$\begin{aligned}
&= \bigvee_{x,x',y,z} ((G_{x,y} \wedge J_{y,z} \wedge H_{x',y}) \dot{\div} F_{x,z}) && (D, E4) \\
&= \bigvee_{\substack{x,x',y,z \\ x \neq x'}} ((G_{x,y} \wedge J_{y,z} \wedge H_{x',y}) \dot{\div} F_{x,z}) \\
&\quad \vee \bigvee_{x,y,z} ((G_{x,y} \wedge J_{y,z} \wedge H_{x,y}) \dot{\div} F_{x,z}) \\
&\preceq \bigvee_{\substack{x,x',y,z \\ x \neq x'}} ((G_{x,y} \wedge H_{x',y}) \dot{\div} F_{x,z}) \\
&\quad \vee \bigvee_{x,y,z} ((J_{y,z} \wedge H_{x,y}) \dot{\div} F_{x,z}) && (L.1.(ii), M, A3) \\
&\preceq \bigvee_{\substack{x,x',y \\ x \neq x'}} (H_{x,y} \wedge G_{x',y}) \\
&\quad \vee \bigvee_{x,y,z} ((H_{x,y} \wedge J_{y,z}) \dot{\div} F_{x,z}) && (L.1.(i), M) \\
&= \text{Agr}_{G,H}^{X,Y} \vee Q && (L.1.(i), M)
\end{aligned}$$

The result is obtained by combining the above inequalities using monotonicity of $\vee$ (M).

Corollary 2 shows how Lemma 5 is instantiated to obtain inequality (6):

Corollary 2. *For any event algebra E , any finite sets X, Y , any $E, F : X \rightarrow E$, $G, H : X \times Y \rightarrow E$, $I, J : Y \rightarrow E$ we have*

$$\text{nPre}_{E,F}^X \preceq \text{Agr}_{G,H}^{X,Y} \vee \text{nPre}_{I,J}^Y \vee \text{P1}_{F,H,J}^{X,Y} \vee \text{P2}_{E,G,I}^{X,Y}$$

$$\text{where } \text{P1}_{F,H,J}^{X,Y,Z} := \bigvee_{x,y} ((H_{x,y} \wedge J_y) \dot{\div} F_x) \vee \bigvee_y \left(J_y \dot{\div} \bigvee_x H_{x,y} \right)$$

$$\text{and } \text{P2}_{E,G,I}^{X,Y,Z} := \bigvee_x \left(E_x \dot{\div} \bigvee_y (G_{x,y} \wedge I_y) \right)$$

Proof. This follows by invoking a lemma proved above (Lemma 5) by choosing a Z with $|Z| = 1$. Then $X \times Z \cong X$ and $Y \times Z \cong Y$. This allows us to understand $E, F : X \times Z \rightarrow E$ as functions $X \rightarrow E$ and $I, J : Y \times Z \rightarrow E$ as functions $Y \rightarrow E$. Consequently, we have $\bigvee_z (F_{x,z}) = F_x$ (and analogously for J). $\square$

Lemma 6. *For any event algebra E , any finite sets X, Y, Z, J , any $E, F : (X \times Y \times Z \times J) \rightarrow E$, $G, H : (X \times Y \times J) \rightarrow E$, $I, J : (Y \times Z \times J) \rightarrow E$ we have*

$$\text{nPre}_{E,F}^{(X \times Y \times Z \times J)} \preceq \text{pnPre}_{G,H}^{(X \times J), Y} \vee \bigvee_j \left(\text{1pnPre}_{I(\cdot, \cdot, j), J(\cdot, \cdot, j)}^{Y, Z} \right)$$

$$\vee P1_{F,H,J}^{X,Y,Z,J} \vee P2_{E,G,I}^{X,Y,Z,J}$$

$$\begin{aligned} \text{where } P1_{F,H,J}^{X,Y,Z,J} &:= \bigvee_{x,y,z,j} ((H_{x,y,j} \wedge J_{y,z,j}) \dot{\div} F_{x,y,z,j}) \\ &\vee \bigvee_{y,z,j} \left(J_{y,z,j} \dot{\div} \bigvee_{x'} H_{x',y,j} \right) \\ &\vee \bigvee_j \text{Mult}_{J(\cdot,\cdot,j)}^{Y,Z} \vee \bigvee_j \text{Mult}_{H(\cdot,\cdot,j)}^{X,Y} \\ \text{and } P2_{E,G,I}^{X,Y,Z,J} &:= \bigvee_{x,y,z,j} (E_{x,y,z,j} \dot{\div} (G_{x,y,j} \wedge I_{y,z,j})) \end{aligned}$$

Proof. We begin by deriving a property that we will need later, satisfied in any event algebra E , for any $a, b, c, d \in E$. We have

$$\begin{aligned} (a \wedge b) \dot{\div} (c \wedge d) &= ((a \wedge b) \dot{\div} c) \vee ((a \wedge b) \dot{\div} d) && \text{(L.1.(vi))} \\ &\preceq (a \dot{\div} c) \vee ((a \wedge b) \dot{\div} d) && \text{(L.1.(ii), A3)} \\ &\preceq (a \dot{\div} c) \vee (b \dot{\div} d) && \text{(L.1.(ii), A3)} \end{aligned}$$

Hence,

$$(a \wedge b) \dot{\div} (c \wedge d) \preceq (a \dot{\div} c) \vee (b \dot{\div} d). \quad (\text{P})$$

We introduce some abbreviations: let

$$P1_{F,H,J}^{X,Y,Z,J} := Q \vee R \vee \bigvee_j \text{Mult}_{J(\cdot,\cdot,j)}^{Y,Z} \vee \bigvee_j \text{Mult}_{H(\cdot,\cdot,j)}^{X,Y},$$

where $Q := \bigvee_{x,y,z,j} ((H_{x,y,j} \wedge J_{y,z,j}) \dot{\div} F_{x,y,z,j})$,
and $R := \bigvee_{y,z,j} (J_{y,z,j} \dot{\div} \bigvee_{x'} H_{x',y,j})$.

We have

$$\begin{aligned} \text{nPre}_{E,F}^{(X \times Y \times Z \times J)} &= \bigvee_{x,y,z,j} (E_{x,y,z,j} \dot{\div} F_{x,y,z,j}) \\ &= \bigvee_{x,y,z,j} (E_{x,y,z,j} \dot{\div} (G_{x,y,j} \wedge I_{y,z,j})) \\ &\quad \vee (G_{x,y,j} \wedge I_{y,z,j}) \dot{\div} F_{x,y,z,j} && \text{(C.1.(iii))} \\ &\preceq \bigvee_{x,y,z,j} (E_{x,y,z,j} \dot{\div} (G_{x,y,j} \wedge I_{y,z,j})) \\ &\quad \vee \bigvee_{x,y,z,j} ((G_{x,y,j} \wedge I_{y,z,j}) \dot{\div} F_{x,y,z,j}). && \text{(L.15)} \\ &= P2_{E,G,I}^{X,Y,Z,J} \vee \bigvee_{x,y,z,j} ((G_{x,y,j} \wedge I_{y,z,j}) \dot{\div} F_{x,y,z,j}) \end{aligned}$$

Moreover,

$$\begin{aligned}
& \bigvee_{x,y,z,j} ((G_{x,y,j} \wedge I_{y,z,j}) \dot{\div} F_{x,y,z,j}) \\
&= \bigvee_{x,y,z,j} \left((G_{x,y,j} \wedge I_{y,z,j}) \right. \\
&\quad \dot{\div} \left(\bigvee_{x',y'} (H_{x',y',j}) \wedge \bigvee_{y'',z'} (J_{y'',z',j}) \right) \\
&\quad \vee \left(G_{x,y,j} \wedge I_{y,z,j} \right. \\
&\quad \left. \wedge \bigvee_{x',y'} (H_{x',y',j}) \wedge \bigvee_{y'',z'} (J_{y'',z',j}) \right) \dot{\div} F_{x,y,z,j} \Big) \quad (\text{C.1.}(ii)) \\
&\preceq \bigvee_{x,y,j} \left(G_{x,y,j} \dot{\div} \bigvee_{x',y'} (H_{x',y',j}) \right) \\
&\quad \vee \bigvee_{y,z,j} \left(I_{y,z,j} \dot{\div} \bigvee_{y'',z'} (J_{y'',z',j}) \right) \\
&\quad \vee \bigvee_{\substack{x,y,y',y'' \\ z,z',j}} ((G_{x,y,j} \wedge I_{y,z,j} \wedge H_{x,y',j} \wedge J_{y'',z',j}) \dot{\div} F_{x,y,z,j}).
\end{aligned}$$

($D, P, \text{L.15}$)

Using that $\dot{\div}$ is antitone in the second argument (L.1.(iv)), the monotonicity of $\vee$ (M) and (A3) we have

$$\begin{aligned}
& \bigvee_{x,y,j} \left(G_{x,y,j} \dot{\div} \bigvee_{x',y'} (H_{x',y',j}) \right) \\
&\preceq \bigvee_{x,y,j} \left(G_{x,y,j} \dot{\div} \bigvee_{y'} (H_{x,y',j}) \right) = \text{pnPre}_{G,H}^{(X \times J), Y}
\end{aligned}$$

and

$$\bigvee_{y,z,j} \left(I_{y,z,j} \dot{\div} \bigvee_{y'',z'} (J_{y'',z',j}) \right) \preceq \bigvee_{y,z,j} (I_{y,z,j} \dot{\div} J_{y,z,j})$$

Furthermore, we have either $y' = y''$ or $y' \neq y''$, so:

$$\bigvee_{\substack{x,y,y',y'' \\ z,z',j}} ((G_{x,y,j} \wedge I_{y,z,j} \wedge H_{x,y',j} \wedge J_{y'',z',j}) \dot{\div} F_{x,y,z,j})$$

$$= \bigvee_{\substack{x,y,y' \\ z,z',j}} ((G_{x,y,j} \wedge I_{y,z,j} \wedge H_{x,y',j} \wedge J_{y',z',j}) \dot{-} F_{x,y,z,j}) \quad (14)$$

$$\vee \bigvee_{\substack{x,y,y',y'' \\ z,z',j \\ y' \neq y''}} ((G_{x,y,j} \wedge I_{y,z,j} \wedge H_{x,y',j} \wedge J_{y'',z',j}) \dot{-} F_{x,y,z,j}). \quad (15)$$

We first consider (15). We have

$$\begin{aligned} & \bigvee_{\substack{x,y,y',y'' \\ z,z',j \\ y' \neq y''}} ((G_{x,y,j} \wedge I_{y,z,j} \wedge H_{x,y',j} \wedge J_{y'',z',j}) \dot{-} F_{x,y,z,j}) \\ & \preceq \bigvee_{\substack{x,y,y',y'' \\ z,z',j \\ y' \neq y''}} (G_{x,y,j} \wedge I_{y,z,j} \wedge H_{x,y',j} \wedge J_{y'',z',j}) \quad (\text{L.1.}(i), \text{M}) \\ & = \bigvee_{\substack{x,y,y',y'' \\ z,z',j \\ y' \neq y''}} \left((G_{x,y,j} \wedge I_{y,z,j} \wedge H_{x,y',j} \wedge J_{y'',z',j}) \right. \\ & \quad \dot{-} \bigvee_{x'} (H_{x',y'',j}) \\ & \quad \left. \vee \bigvee_{x'} (G_{x,y,j} \wedge I_{y,z,j} \wedge H_{x,y',j} \wedge J_{y'',z',j} \wedge H_{x',y'',j}) \right) \\ & \quad (\text{L.1.}(v), D) \\ & \preceq \bigvee_{y'',z',j} \left(J_{y'',z',j} \dot{-} \bigvee_{x'} (H_{x',y'',j}) \right) \\ & \quad \vee \bigvee_{\substack{x,x',y',y'',j \\ y' \neq y''}} (H_{x,y',j} \wedge H_{x',y'',j}) \quad (\text{L.15, L.1.}(ii), \text{M, A3}) \\ & \preceq \bigvee_{y,z,j} \left(J_{y,z,j} \dot{-} \bigvee_{x'} (H_{x',y,j}) \right) \\ & \quad \vee \bigvee_{\substack{x,x',y,y',j \\ (x,y) \neq (x',y')}} (H_{x,y,j} \wedge H_{x',y',j}) \\ & = R \vee \bigvee_j \text{Mult}_{\text{H}(\cdot, \cdot, j)}^{X,Y} \end{aligned}$$

Returning to (14) we must have either $y = y'$ or $y \neq y'$:

$$\begin{aligned} & \bigvee_{\substack{x,y,y' \\ z,z',j}} ((G_{x,y,j} \wedge I_{y,z,j} \wedge H_{x,y',j} \wedge J_{y',z',j}) \div F_{x,y,z,j}) \\ &= \bigvee_{\substack{x,y \\ z,z',j}} ((G_{x,y,j} \wedge I_{y,z,j} \wedge H_{x,y,j} \wedge J_{y,z',j}) \div F_{x,y,z,j}) \end{aligned} \quad (16)$$

$$\vee \bigvee_{\substack{x,y,y' \\ z,z',j \\ y \neq y'}} ((G_{x,y,j} \wedge I_{y,z,j} \wedge H_{x,y',j} \wedge J_{y',z',j}) \div F_{x,y,z,j}). \quad (17)$$

In case (17), we have

$$\begin{aligned} & \bigvee_{\substack{x,y,y' \\ z,z',j \\ y \neq y'}} ((G_{x,y,j} \wedge I_{y,z,j} \wedge H_{x,y',j} \wedge J_{y',z',j}) \div F_{x,y,z,j}) \\ & \preceq \bigvee_{\substack{y,y' \\ z,z',j \\ y \neq y'}} (I_{y,z,j} \wedge J_{y',z',j}) \quad (\text{L.1.}(i), \text{M,A3}) \\ & \preceq \bigvee_{\substack{y,y' \\ z,z',j \\ y \neq y'}} ((I_{y,z,j} \wedge J_{y',z',j}) \div J_{y,z,j}) \\ & \vee (I_{y,z,j} \wedge J_{y,z',j} \wedge J_{y,z,j}) \quad (\text{L.1.}(v)) \\ & \preceq \bigvee_{y,z,j} (I_{y,z,j} \div J_{y,z,j}) \\ & \vee \bigvee_{\substack{y,y',z,z',j \\ (y,z) \neq (y',z')}} (J_{y',z',j} \wedge J_{y,z,j}). \quad (\text{L.15,L.1.}(ii), \text{M,A3}) \\ & = \bigvee_j \left(1\text{pnPre}_{I(\cdot,\cdot,j),J(\cdot,\cdot,j)}^{Y,Z} \right) \vee \bigvee_j \text{Mult}_{J(\cdot,\cdot,j)}^{Y,Z} \end{aligned}$$

In case (16), we have either $z = z'$ or $z \neq z'$:

$$\begin{aligned} & \bigvee_{\substack{x,y \\ z,z',j}} ((G_{x,y,j} \wedge I_{y,z,j} \wedge H_{x,y,j} \wedge J_{y,z',j}) \div F_{x,y,z,j}) \\ & \preceq \bigvee_{\substack{x,y \\ z,z',j \\ z \neq z'}} ((G_{x,y,j} \wedge I_{y,z,j} \wedge H_{x,y,j} \wedge J_{y,z',j}) \div F_{x,y,z,j}) \\ & \vee \bigvee_{x,y,z,j} ((G_{x,y,j} \wedge I_{y,z,j} \wedge H_{x,y,j} \wedge J_{y,z,j}) \div F_{x,y,z,j}) \end{aligned}$$

$$\begin{aligned}
& \leq \bigvee_{\substack{y,z,z',j \\ z \neq z'}} (I_{y,z,j} \wedge J_{y,z',j}) \\
& \quad \vee \bigvee_{x,y,z,j} ((H_{x,y,j} \wedge J_{y,z,j}) \dot{-} F_{x,y,z,j}). \\
& \hspace{15em} (\text{L.1.}(i), \text{L.1.}(ii), \text{M}, \text{A3}) \\
& = \bigvee_{\substack{y,z,z',j \\ z \neq z'}} (I_{y,z,j} \wedge J_{y,z',j}) \vee Q
\end{aligned}$$

We have

$$\begin{aligned}
& \bigvee_{\substack{y,z,z',j \\ z \neq z'}} (I_{y,z,j} \wedge J_{y,z',j}) \\
& \leq \bigvee_{\substack{y,z,z',j \\ z \neq z'}} \left((I_{y,z,j} \wedge J_{y,z',j}) \dot{-} J_{y,z,j} \right. \\
& \quad \left. \vee (I_{y,z,j} \wedge J_{y,z',j} \wedge J_{y,z,j}) \right) \hspace{10em} (\text{L.1.}(v)) \\
& \leq \bigvee_{y,z,j} (I_{y,z,j} \dot{-} J_{y,z,j}) \\
& \quad \vee \bigvee_{\substack{y,z,z',j \\ z \neq z'}} (J_{y,z',j} \wedge J_{y,z,j}) \hspace{10em} (\text{L.15}, \text{L.1.}(i), M, A3) \\
& \leq \bigvee_{y,z,j} (I_{y,z,j} \dot{-} J_{y,z,j}) \\
& \quad \vee \bigvee_{\substack{y,y',z,z',j \\ (y,z) \neq (y',z')}} (J_{y,z,j} \wedge J_{y',z',j}) \\
& = \bigvee_{y,z,j} (I_{y,z,j} \dot{-} J_{y,z,j}) \vee \bigvee_j \text{Mult}_{J(\cdot, \cdot, j)}^{Y,Z}
\end{aligned}$$

Finally, observe that

$$\begin{aligned}
& \bigvee_{y,z,j} (I_{y,z,j} \dot{-} J_{y,z,j}) \\
& \leq \bigvee_{y,z,j} \left((I_{y,z,j} \dot{-} J_{y,z,j}) \dot{-} \left(\bigvee_{\substack{y,y',z,z' \\ (y,z) \neq (y',z')}} (J_{y,z,j} \wedge J_{y',z',j}) \right) \right. \\
& \quad \left. \vee \bigvee_{\substack{y,y',z,z' \\ (y,z) \neq (y',z')}} (J_{y,z,j} \wedge J_{y',z',j}) \right) \hspace{10em} (\text{L.1.}(v), \text{M}, \text{A3})
\end{aligned}$$

$$\begin{aligned}
& \preceq \bigvee_{y,z,j} \left((I_{y,z,j} \dot{-} J_{y,z,j}) \dot{-} \left(\bigvee_{\substack{y,y',z,z',j \\ (y,z) \neq (y',z')}} (J_{y,z,j} \wedge J_{y',z',j}) \right) \right) \\
& \quad \vee \bigvee_{\substack{y,y',z,z',j \\ (y,z) \neq (y',z')}} (J_{y,z,j} \wedge J_{y',z',j}) \\
& = \bigvee_j \left(\text{1pnPre}_{I(\cdot,\cdot,j),J(\cdot,\cdot,j)}^{Y,Z} \right) \vee \bigvee_j \text{Mult}_{J(\cdot,\cdot,j)}^{Y,Z}
\end{aligned} \tag{L.15}$$

The result is obtained by combining the above inequalities using monotonicity and idempotence of $\vee$ (M,I2).

Lemma 7. *For any event algebra E , any finite sets I, J, W, X, Y, Z , any $E, F : (I \times J \times X \times Y \times Z) \rightarrow E$, $G, H : (I \times W \times Y) \rightarrow E$, $I, J : (J \times W \times Y \times Z) \rightarrow E$, $K, L : (I \times J \times X \times W) \rightarrow E$ we have*

$$\begin{aligned}
\text{nPre}_{E,F}^{I \times J \times X \times Y \times Z} & \preceq \left(\text{nPre}_{G,H}^{I \times W \times Y} \wedge \text{pnPre}_{I,J}^{J \times W \times Y \times Z} \right) \vee \text{Agr}_{K,L}^{(I \times J \times X), W} \\
& \quad \vee \text{Agr}_{I,I}^{(J \times W \times Y), Z} \vee \text{P1}_{F,H,I,L}^{I,J,X,Y,Z,W} \vee \text{P2}_{E,G,I,K}^{I,J,X,Y,Z,W}
\end{aligned}$$

$$\begin{aligned}
\text{where } \text{P1}_{F,H,I,L}^{I,J,X,Y,Z,W} & := \bigvee_{i,j,w,x,y,z} ((L_{i,j,x,w} \wedge H_{i,w,y} \wedge J_{j,w,y,z}) \dot{-} F_{i,j,x,y,z}) \\
& \quad \vee \bigvee_{\substack{i,j \\ x,y,w}} \left((H_{i,w,y} \wedge L_{i,j,x,w}) \dot{-} \bigvee_{z'} J_{j,w,y,z'} \right) \\
& \quad \vee \bigvee_{i,j,x,y,z,w} ((K_{i,j,x,w} \wedge J_{j,w,y,z}) \dot{-} H_{i,w,y}) \\
& \quad \vee \bigvee_{i,y,w} \left(H_{i,w,y} \dot{-} \bigvee_{j',x'} L_{i,j',x',w} \right) \\
\text{and } \text{P2}_{E,G,I,K}^{I,J,X,Y,Z,W} & := \bigvee_{i,j,x,y,z} \left(E_{i,j,x,y,z} \dot{-} \bigvee_w (K_{i,j,x,w} \wedge G_{i,w,y} \wedge I_{j,w,y,z}) \right)
\end{aligned}$$

Proof. We introduce some abbreviations: let $\text{P1}_{F,H,I}^{X,Y,Z,J} := Q \vee R \vee S \vee T$, where

$$Q := \bigvee_{i,j,w,x,y,z} ((L_{i,j,x,w} \wedge H_{i,w,y} \wedge J_{j,w,y,z}) \dot{-} F_{i,j,x,y,z}),$$

$$R := \bigvee_{\substack{i,j \\ x,y,w}} ((H_{i,w,y} \wedge L_{i,j,x,w}) \dot{-} \bigvee_{z'} J_{j,w,y,z'}),$$

$$S := \bigvee_{i,j,x,y,z,w} ((K_{i,j,x,w} \wedge J_{j,w,y,z}) \dot{-} H_{i,w,y}), \text{ and}$$

$$T := \bigvee_{i,y,w} (H_{i,w,y} \dot{-} \bigvee_{j',x'} L_{i,j',x',w}).$$

We have

$$\text{nPre}_{E,F}^{I \times J \times X \times Y \times Z}$$

$$\begin{aligned}
&= \bigvee_{i,j,x,y,z} (E_{i,j,x,y,z} \dot{-} F_{i,j,x,y,z}) \\
&\preceq \bigvee_{i,j,x,y,z} \left(E_{i,j,x,y,z} \dot{-} \bigvee_w (K_{i,j,x,w} \wedge G_{i,w,y} \wedge I_{j,w,y,z}) \right. \\
&\quad \left. \vee \bigvee_w (K_{i,j,x,w} \wedge G_{i,w,y} \wedge I_{j,w,y,z}) \dot{-} F_{i,j,x,y,z} \right) \quad (\text{C.1.}(iii)) \\
&\preceq \bigvee_{i,j,x,y,z} \left(E_{i,j,x,y,z} \dot{-} \bigvee_w (K_{i,j,x,w} \wedge G_{i,w,y} \wedge I_{j,w,y,z}) \right) \\
&\quad \vee \bigvee_{i,j,x,y,z,w} ((K_{i,j,x,w} \wedge G_{i,w,y} \wedge I_{j,w,y,z}) \dot{-} F_{i,j,x,y,z}) \quad (\text{L.15}) \\
&= \text{P2}_{E,G,I,K}^{I,J,X,Y,Z,W} \vee \bigvee_{i,j,x,y,z,w} ((K_{i,j,x,w} \wedge G_{i,w,y} \wedge I_{j,w,y,z}) \dot{-} F_{i,j,x,y,z})
\end{aligned}$$

Moreover,

$$\begin{aligned}
&\bigvee_{i,j,x,y,z,w} ((K_{i,j,x,w} \wedge G_{i,w,y} \wedge I_{j,w,y,z}) \dot{-} F_{i,j,x,y,z}) \\
&\preceq \bigvee_{i,j,x,y,z,w} \left(((K_{i,j,x,w} \wedge G_{i,w,y} \wedge I_{j,w,y,z}) \dot{-} H_{i,w,y}) \right. \\
&\quad \left. \vee (K_{i,j,x,w} \wedge G_{i,w,y} \wedge I_{j,w,y,z} \wedge H_{i,w,y}) \dot{-} F_{i,j,x,y,z} \right) \quad (\text{C.1.}(ii)) \\
&\preceq \bigvee_{i,j,x,y,z,w} ((K_{i,j,x,w} \wedge G_{i,w,y} \wedge I_{j,w,y,z}) \dot{-} H_{i,w,y}) \quad (18) \\
&\quad \vee \bigvee_{i,j,x,y,z,w} ((K_{i,j,x,w} \wedge G_{i,w,y} \wedge I_{j,w,y,z} \wedge H_{i,w,y}) \dot{-} F_{i,j,x,y,z}) \quad (\text{L.15}) \quad (19)
\end{aligned}$$

In case (18) we have:

$$\begin{aligned}
&\bigvee_{i,j,x,y,z,w} ((K_{i,j,x,w} \wedge G_{i,w,y} \wedge I_{j,w,y,z}) \dot{-} H_{i,w,y}) \\
&= \bigvee_{i,j,x,y,z,w} \left(\left(K_{i,j,x,w} \wedge G_{i,w,y} \wedge I_{j,w,y,z} \wedge \bigvee_{z'} J_{j,w,y,z'} \right) \dot{-} H_{i,w,y} \right. \\
&\quad \left. \vee \left((K_{i,j,x,w} \wedge G_{i,w,y} \wedge I_{j,w,y,z}) \dot{-} \bigvee_{z'} J_{j,w,y,z'} \right) \dot{-} H_{i,w,y} \right) \quad (\text{C.1.}(ii), E4) \\
&= \bigvee_{i,j,x,y,z,w} \left(\left(K_{i,j,x,w} \wedge G_{i,w,y} \wedge I_{j,w,y,z} \wedge \bigvee_{z'} J_{j,w,y,z'} \right) \dot{-} H_{i,w,y} \right)
\end{aligned}$$

$$\begin{aligned}
& \bigvee_{i,j,x,y,z,w} \left((K_{i,j,x,w} \wedge G_{i,w,y} \wedge I_{j,w,y,z}) \dot{-} \left(\bigvee_{z'} J_{j,w,y,z'} \vee H_{i,w,y} \right) \right) \\
& \quad (L.15, 11) \\
& \preceq \bigvee_{i,j,x,y,z,w} ((K_{i,j,x,w} \wedge J_{j,w,y,z}) \dot{-} H_{i,w,y}) \\
& \bigvee_{i,j,x,y,z,w} \left((K_{i,j,x,w} \wedge G_{i,w,y} \wedge I_{j,w,y,z}) \dot{-} \left(\bigvee_{z'} J_{j,w,y,z'} \vee H_{i,w,y} \right) \right) \\
& \quad (M, O, L.1.(ii)) \\
& = S \bigvee_{\substack{i,j \\ x,y \\ z,w}} \left((K_{i,j,x,w} \wedge G_{i,w,y} \wedge I_{j,w,y,z}) \dot{-} \left(\bigvee_{z'} J_{j,w,y,z'} \vee H_{i,w,y} \right) \right)
\end{aligned}$$

and we have

$$\begin{aligned}
& \bigvee_{i,j,x,y,z,w} \left((K_{i,j,x,w} \wedge G_{i,w,y} \wedge I_{j,w,y,z}) \dot{-} \left(\bigvee_{z'} J_{j,w,y,z'} \vee H_{i,w,y} \right) \right) \\
& \preceq \bigvee_{i,j,x,y,z,w} \left((K_{i,j,x,w} \wedge G_{i,w,y} \wedge I_{j,w,y,z}) \dot{-} \bigvee_{z'} J_{j,w,y,z'} \right. \\
& \quad \left. \wedge (K_{i,j,x,w} \wedge G_{i,w,y} \wedge I_{j,w,y,z}) \dot{-} H_{i,w,y} \right) \\
& \quad (L.1.(viii)) \\
& \preceq \bigvee_{i,j,y,z,w} \left(I_{j,w,y,z} \dot{-} \bigvee_{z'} J_{j,w,y,z'} \wedge G_{i,w,y} \dot{-} H_{i,w,y} \right) \\
& \quad (M, O, L.1.(ii)) \\
& \preceq \bigvee_{j,y,z,w} \left(I_{j,w,y,z} \dot{-} \bigvee_{z'} J_{j,w,y,z'} \right) \wedge \bigvee_{i,y,w} (G_{i,w,y} \dot{-} H_{i,w,y}) \\
& \quad (L.15) \\
& = \text{pnPre}_{I,J}^{J \times W \times Y \times Z} \wedge \text{nPre}_{G,H}^{I \times W \times Y}
\end{aligned}$$

In case (19), we have

$$\begin{aligned}
& \bigvee_{i,j,x,y,z,w} ((K_{i,j,x,w} \wedge G_{i,w,y} \wedge I_{j,w,y,z} \wedge H_{i,w,y}) \dot{-} F_{i,j,x,y,z}) \\
& \preceq \bigvee_{i,j,x,y,z,w} \left((K_{i,j,x,w} \wedge G_{i,w,y} \wedge I_{j,w,y,z} \wedge H_{i,w,y}) \dot{-} \bigvee_{j',x'} L_{i,j',x',w} \right)
\end{aligned}$$

$$\begin{aligned}
& \vee \left(K_{i,j,x,w} \wedge G_{i,w,y} \wedge I_{j,w,y,z} \wedge H_{i,w,y} \wedge \bigvee_{j',x'} L_{i,j',x',w} \right) \dot{-} F_{i,j,x,y,z} \\
& \hspace{15em} \text{(C.1.(iii))} \\
& \preceq \bigvee_{i,j,x,y,z,w} \left((K_{i,j,x,w} \wedge G_{i,w,y} \wedge I_{j,w,y,z} \wedge H_{i,w,y}) \dot{-} \bigvee_{j',x'} L_{i,j',x',w} \right) \\
& \vee \bigvee_{\substack{i,j,x, \\ y,z,w}} \left(\left(K_{i,j,x,w} \wedge G_{i,w,y} \wedge I_{j,w,y,z} \wedge H_{i,w,y} \wedge \bigvee_{j',x'} L_{i,j',x',w} \right) \dot{-} F_{i,j,x,y,z} \right) \\
& \hspace{15em} \text{(L.15)} \\
& \preceq \bigvee_{i,y,w} \left(H_{i,w,y} \dot{-} \bigvee_{j',x'} L_{i,j',x',w} \right) \\
& \vee \bigvee_{\substack{i,j,x, \\ y,z,w}} \left(\left(K_{i,j,x,w} \wedge G_{i,w,y} \wedge I_{j,w,y,z} \wedge H_{i,w,y} \wedge \bigvee_{j',x'} L_{i,j',x',w} \right) \dot{-} F_{i,j,x,y,z} \right) \\
& \hspace{15em} \text{(M, O, L.1.(ii))} \\
& = T \\
& \vee \bigvee_{\substack{i,j,x, \\ y,z,w}} \left(\left(K_{i,j,x,w} \wedge G_{i,w,y} \wedge I_{j,w,y,z} \wedge H_{i,w,y} \wedge \bigvee_{j',x'} L_{i,j',x',w} \right) \dot{-} F_{i,j,x,y,z} \right)
\end{aligned}$$

Moreover, we have

$$\begin{aligned}
& \bigvee_{\substack{i,j,x, \\ y,z,w}} \left(\left(K_{i,j,x,w} \wedge G_{i,w,y} \wedge I_{j,w,y,z} \wedge H_{i,w,y} \wedge \bigvee_{j',x'} L_{i,j',x',w} \right) \dot{-} F_{i,j,x,y,z} \right) \\
& = \bigvee_{\substack{i,j,j', \\ x,x',y,z,w}} ((K_{i,j,x,w} \wedge G_{i,w,y} \wedge I_{j,w,y,z} \wedge H_{i,w,y} \wedge L_{i,j',x',w}) \dot{-} F_{i,j,x,y,z}) \quad (20)
\end{aligned}$$

(D, E4)

$$= \bigvee_{\substack{i,j \\ x,y,z,w}} ((K_{i,j,x,w} \wedge G_{i,w,y} \wedge I_{j,w,y,z} \wedge H_{i,w,y} \wedge L_{i,j,x,w}) \dot{-} F_{i,j,x,y,z}) \quad (21)$$

$$\begin{aligned}
& \vee \bigvee_{\substack{i,j,j', \\ x,x',y,z,w \\ (j,x) \neq (j',x')}} ((K_{i,j,x,w} \wedge G_{i,w,y} \wedge I_{j,w,y,z} \wedge H_{i,w,y} \wedge L_{i,j',x',w}) \dot{-} F_{i,j,x,y,z}) \\
& \hspace{15em} (22)
\end{aligned}$$

In case (22) we have

$$\begin{aligned}
& \bigvee_{\substack{i,j,j', \\ x,x',y,z,w \\ (j,x) \neq (j',x')}} ((K_{i,j,x,w} \wedge G_{i,w,y} \wedge I_{j,w,y,z} \wedge H_{i,w,y} \wedge L_{i,j',x',w}) \dot{-} F_{i,j,x,y,z}) \\
& \preceq \bigvee_{\substack{i,j,j', \\ x,x',y,z,w \\ (j,x) \neq (j',x')}} ((K_{i,j,x,w} \wedge L_{i,j',x',w}) \dot{-} F_{i,j,x,y,z}) \\
& \hspace{25em} (M, O, L.1.(ii)) \\
& \preceq \bigvee_{\substack{i,j,j', \\ x,x',w \\ (j,x) \neq (j',x')}} (K_{i,j,x,w} \wedge L_{i,j',x',w}) \preceq \bigvee_{\substack{i,i',j,j', \\ x,x',w \\ (i,j,x) \neq (i',j',x')}} (K_{i,j,x,w} \wedge L_{i',j',x',w}) \\
& \hspace{25em} (M, L.1.(i)) \\
& = \text{Agr}_{K,L}^{(I \times J \times X), W}
\end{aligned}$$

In case (21) we have:

$$\begin{aligned}
& \bigvee_{\substack{i,j \\ x,y,z,w}} ((K_{i,j,x,w} \wedge G_{i,w,y} \wedge I_{j,w,y,z} \wedge H_{i,w,y} \wedge L_{i,j,x,w}) \dot{-} F_{i,j,x,y,z}) \\
& \preceq \bigvee_{\substack{i,j \\ x,y,z,w}} \left(\left((K_{i,j,x,w} \wedge G_{i,w,y} \wedge I_{j,w,y,z} \wedge H_{i,w,y} \wedge L_{i,j,x,w}) \dot{-} \bigvee_{z'} J_{j,w,y,z'} \right) \right. \\
& \quad \left. \vee \left(K_{i,j,x,w} \wedge G_{i,w,y} \wedge I_{j,w,y,z} \wedge H_{i,w,y} \wedge L_{i,j,x,w} \wedge \bigvee_{z'} J_{j,w,y,z'} \right) \right) \\
& \quad \dot{-} F_{i,j,x,y,z} \\
& \hspace{25em} (C.1.(ii)) \\
& \preceq \bigvee_{\substack{i,j \\ x,y \\ z,w}} \left((K_{i,j,x,w} \wedge G_{i,w,y} \wedge I_{j,w,y,z} \wedge H_{i,w,y} \wedge L_{i,j,x,w}) \dot{-} \bigvee_{z'} J_{j,w,y,z'} \right) \dot{-} F_{i,j,x,y,z} \\
& \quad \vee \bigvee_{\substack{i,j \\ x,y \\ z,w}} \left(K_{i,j,x,w} \wedge G_{i,w,y} \wedge I_{j,w,y,z} \wedge H_{i,w,y} \wedge L_{i,j,x,w} \wedge \bigvee_{z'} J_{j,w,y,z'} \right) \dot{-} F_{i,j,x,y,z} \\
& \hspace{25em} (E4, L.15) \\
& \preceq \bigvee_{\substack{i,j \\ x,y,z,w}} (K_{i,j,x,w} \wedge G_{i,w,y} \wedge I_{j,w,y,z} \wedge H_{i,w,y} \wedge L_{i,j,x,w}) \dot{-} \bigvee_{z'} J_{j,w,y,z'}
\end{aligned}$$

$$\begin{aligned}
& \vee \bigvee_{\substack{i,j \\ x,y \\ z,z',w}} (\mathsf{K}_{i,j,x,w} \wedge \mathsf{G}_{i,w,y} \wedge \mathsf{I}_{j,w,y,z} \wedge \mathsf{H}_{i,w,y} \wedge \mathsf{L}_{i,j,x,w} \wedge \mathsf{J}_{j,w,y,z'}) \dot{-} \mathsf{F}_{i,j,x,y,z} \\
& \hspace{15em} (\text{L.1.}(i), D, E4) \\
& \preceq \bigvee_{\substack{i,j \\ x,y,w}} (\mathsf{H}_{i,w,y} \wedge \mathsf{L}_{i,j,x,w}) \dot{-} \bigvee_{z'} \mathsf{J}_{j,w,y,z'} \\
& \vee \bigvee_{\substack{i,j \\ x,y \\ z,z',w}} (\mathsf{I}_{j,w,y,z} \wedge \mathsf{H}_{i,w,y} \wedge \mathsf{L}_{i,j,x,w} \wedge \mathsf{J}_{j,w,y,z'}) \dot{-} \mathsf{F}_{i,j,x,y,z} \\
& \hspace{15em} (M, O, \text{L.1.}(ii)) \\
& = \mathsf{R} \vee \bigvee_{\substack{i,j \\ x,y \\ z,z',w}} (\mathsf{I}_{j,w,y,z} \wedge \mathsf{H}_{i,w,y} \wedge \mathsf{L}_{i,j,x,w} \wedge \mathsf{J}_{j,w,y,z'})
\end{aligned}$$

Finally,

$$\begin{aligned}
& \bigvee_{\substack{i,j \\ x,y,z,z',w}} (\mathsf{I}_{j,w,y,z} \wedge \mathsf{H}_{i,w,y} \wedge \mathsf{L}_{i,j,x,w} \wedge \mathsf{J}_{j,w,y,z'}) \dot{-} \mathsf{F}_{i,j,x,y,z} \\
& = \bigvee_{\substack{i,j \\ x,y,z,z',w \\ z \neq z'}} (\mathsf{I}_{j,w,y,z} \wedge \mathsf{H}_{i,w,y} \wedge \mathsf{L}_{i,j,x,w} \wedge \mathsf{J}_{j,w,y,z'}) \dot{-} \mathsf{F}_{i,j,x,y,z} \\
& \vee \bigvee_{\substack{i,j \\ x,y,z,w}} (\mathsf{I}_{j,w,y,z} \wedge \mathsf{H}_{i,w,y} \wedge \mathsf{L}_{i,j,x,w} \wedge \mathsf{J}_{j,w,y,z}) \dot{-} \mathsf{F}_{i,j,x,y,z} \\
& \preceq \bigvee_{\substack{i,j \\ x,y,z,z',w \\ z \neq z'}} (\mathsf{I}_{j,w,y,z} \wedge \mathsf{J}_{j,w,y,z'}) \dot{-} \mathsf{F}_{i,j,x,y,z} \\
& \vee \bigvee_{\substack{i,j \\ x,y,z,w}} (\mathsf{L}_{i,j,x,w} \wedge \mathsf{H}_{i,w,y} \wedge \mathsf{J}_{j,w,y,z}) \dot{-} \mathsf{F}_{i,j,x,y,z} \hspace{10em} (M, O, \text{L.1.}(ii)) \\
& \preceq \bigvee_{\substack{i,j \\ x,y,z,z',w \\ z \neq z'}} (\mathsf{I}_{j,w,y,z} \wedge \mathsf{J}_{j,w,y,z'}) \\
& \vee \bigvee_{\substack{i,j \\ x,y,z,w}} (\mathsf{L}_{i,j,x,w} \wedge \mathsf{H}_{i,w,y} \wedge \mathsf{J}_{j,w,y,z}) \dot{-} \mathsf{F}_{i,j,x,y,z} \hspace{10em} (\text{L.1.}(i)) \\
& = \mathsf{Q} \vee \mathsf{Agr}_{\mathsf{I},\mathsf{I}}^{(J \times W \times Y), Z}
\end{aligned}$$

The result is obtained by combining the above inequalities using monotonicity of $\vee$.

F.3 Lean Code for the Formal Verification of Lemma 4

The following allows to formally verify Lemmas 3 and 4 (and the lemmas in Section 2) in mathlib (Lean).

```
import Mathlib.Order.Heyting.Basic
import MIL.Common

class DualHeytingAlgebra α( : Type* ) extends Lattice α, BoundedOrder α, SDiff α
where
E1 (a : α) : a \ a = ⊥
E2 (a b : α) : a = (a \ b) ∪ a
E3 (a b : α) : a ∪ b = (a \ b) ∪ b
E4 (a b c : α) : (a ∪ b) \ c = (a \ c) ∪ (b \ c)

--- This is the paper's definition
class EventAlgebra α( : Type* ) extends DualHeytingAlgebra α where
E5 (a b : α) : (a \ b) ∩ (b \ a) = ⊥

variable [DualHeytingAlgebra α] {a b c d : α}

---LEMMA 1 (i)
lemma lemma_1_i (a b : α) : a \ b ≤ a := by
  rw <[ sup_eq_left, sup_comm]
  rw <[ DualHeytingAlgebra.E2 a b]

---LEMMA 1 (ii)
lemma lemma_1_ii (a b c : α) (h : a ≤ b) : (a \ c) ≤ (b \ c) := by
  rw <[ sup_eq_left<-, DualHeytingAlgebra.E4]
  rw [sup_eq_left.mpr h]

---LEMMA 1 (iii)
lemma lemma_1_iii (a b c : α) : (a ≤ b ∪ c) ↔ (a \ b ≤ c) := by
  constructor·
  intro h1a
  apply lemma_1_ii a (b ∪ c) b at h1a
  calc
  a \ b ≤ (b ∪ c) \ b := by exact h1a
  _ = (b \ b) ∪ (c \ b) := by rw [DualHeytingAlgebra.E4]
  _ = ⊥ ∪ (c \ b) := by rw [DualHeytingAlgebra.E1]
  _ = (c \ b) := by rw [bot_sup_eq]
  _ ≤ c := by apply lemma_1_i·
  intro h2a
  apply sup_eq_left.mpr at h2a
  have h2 : c ∪ a \ b ∪ b = c ∪ b
  rw [h2a]
  rw [sup_assoc<-, DualHeytingAlgebra.E3<-, sup_assoc] at h2
  nth_rw 2 [sup_comm] at h2
  rw [sup_assoc, sup_comm] at h2
  nth_rw 2 [sup_comm] at h2
  nth_rw 3 [sup_comm] at h2
```

```

exact sup_eq_left.mp h2

---LEMMA 1 (iv)
lemma lemma_1_iv (a b c :  $\alpha$ ) (h :  $a \leq b$ ) :  $(c \setminus b \leq c \setminus a)$  := by
  have h1 :  $c \leq b \cup (c \setminus a)$ 
  calc
     $c \leq c \cup a$  := by apply le_sup_left
     $\_ \leq (c \setminus a) \cup a$  := by rw [DualHeytingAlgebra.E3]
     $\_ \leq (c \setminus a) \cup b$  := by apply sup_le_sup_left h
     $\_ = b \cup (c \setminus a)$  := by rw [sup_comm]
  apply (lemma_1_iii c b (c \ a)).mp at h1
exact h1

---LEMMA 2 (Nontrivial direction), the other one is Lattice.sup_inf_le
lemma lemma_2 (a b c :  $\alpha$ ) :  $(a \cup b) \cap (a \cup c) \leq a \cup b \cap c$  := by
  have h1 :  $((a \cup b) \cap (a \cup c)) \setminus a \leq b$  := by
    have h1a :  $(a \cup b) \cap (a \cup c) \leq a \cup b$ 
    apply inf_le_left
    apply (lemma_1_iii ((a \cup b) \cap (a \cup c)) a b).mp
    exact h1a
  have h2 :  $((a \cup b) \cap (a \cup c)) \setminus a \leq c$  := by
    have h2a :  $(a \cup b) \cap (a \cup c) \leq a \cup c$ 
    apply inf_le_right
    apply (lemma_1_iii ((a \cup b) \cap (a \cup c)) a c).mp
    exact h2a
  have h3 :  $((a \cup b) \cap (a \cup c)) \setminus a \leq b \cap c$ 
  apply le_inf h1 h2
  apply (lemma_1_iii ((a \cup b) \cap (a \cup c)) a (b \cap c)).mpr
exact h3

---Instance DistribLattice from DualHeytingAlgebra
instance DualHeytingAlgebra.toDistribLattice [DualHeytingAlgebra  $\alpha$ ] :
  DistribLattice  $\alpha$  :=
  { (inferInstance : Lattice  $\alpha$ ) with
    le_sup_inf := by apply lemma_2
  }

---LEMMA 1 (v)
lemma lemma_1_v (a b :  $\alpha$ ) :  $a = (a \cap b) \cup (a \setminus b)$  :=
  calc
     $a = a \cap (a \cup b)$  := by rw [inf_sup_self]
     $\_ = a \cap (a \setminus b \cup b)$  := by rw [DualHeytingAlgebra.E3]
     $\_ = (a \setminus b \cup a) \cap (a \setminus b \cup b)$  := by rw [DualHeytingAlgebra.E2]
     $\_ = (a \cap b) \cup (a \setminus b)$  := by rw [sup_inf_left, sup_comm]

---LEMMA 11
lemma lemma_11 (a b c :  $\alpha$ ) :  $a \setminus (b \cup c) = (a \setminus b) \setminus c$  := by
  have h1 :  $a \setminus (b \cup c) \leq (a \setminus b) \setminus c$ 
  have h1a :  $a \leq (b \cup c) \cup (a \setminus b) \setminus c$ 

```

```

calc
  a ≤ a ∪ b := by apply le_sup_left
  _ = (a \ b) ∪ b := by rw [DualHeytingAlgebra.E3]
  _ ≤ (a \ b) ∪ b ∪ c := by apply le_sup_left
  _ ≤ (b ∪ c) ∪ (a \ b) \ c := by rw [sup_assoc, sup_comm b c←,
    sup_assoc, (DualHeytingAlgebra.E3 (a \ b) c), sup_assoc, sup_comm]
  apply (lemma_1_iii a (b ∪ c) ((a \ b) \ c)).mp at h1a
  exact h1a
  have h2 : (a \ b) \ c ≤ a \ (b ∪ c)
  have h2a : a ≤ b ∪ (c ∪ (a \ (b ∪ c)))
  calc
    a ≤ a ∪ (b ∪ c) := by apply le_sup_left
    _ = (a \ (b ∪ c)) ∪ (b ∪ c) := by rw [DualHeytingAlgebra.E3]
    _ = b ∪ (c ∪ (a \ (b ∪ c))) := by rw [sup_comm←, sup_assoc]
  apply (lemma_1_iii a b _).mp at h2a
  apply (lemma_1_iii (a \ b) c _).mp at h2a
  exact h2a
  apply le_antisymm h1 h2

---LEMMA 1 (vii)
lemma lemma_1_vii (a b c : α) : a \ (b ∩ c) = (a \ b) ∪ (a \ c) := by
  have h1 : (a \ b) ∪ (a \ c) ≤ a \ (b ∩ c)
  calc
    (a \ b) ∪ (a \ c) ≤ (a \ (b ∩ c)) ∪ (a \ c) := by apply sup_le_sup_right
    (lemma_1_iv (b ∩ c) b a inf_le_left)
    _ ≤ (a \ (b ∩ c)) ∪ (a \ (b ∩ c)) := by apply sup_le_sup_left (lemma_1_iv (b
      ∩ c) c a inf_le_right)
    _ = (a \ (b ∩ c)) := by rw [sup_idem]
  have h2 : a \ (b ∩ c) ≤ (a \ b) ∪ (a \ c)
  have h2a : a ≤ (b ∩ c) ∪ ((a \ b) ∪ (a \ c))
  calc
    a = a ∩ a := by rw [inf_idem]
    _ ≤ (a ∪ b) ∩ (a ∪ c) := by apply inf_le_inf le_sup_left le_sup_left
    _ = (a \ b ∪ b) ∩ (a \ c ∪ c) := by rw [DualHeytingAlgebra.E3←,
      DualHeytingAlgebra.E3]
    _ = (a \ b ∩ (a \ c ∪ c)) ∪ (b ∩ (a \ c ∪ c)) := by rw [inf_sup_right]
    _ ≤ a \ b ∪ (b ∩ (a \ c ∪ c)) := by apply sup_le_sup_right (inf_le_left)
    _ = a \ b ∪ ((b ∩ a \ c) ∪ (b ∩ c)) := by rw [inf_sup_left]
    _ ≤ a \ b ∪ (a \ c ∪ (b ∩ c)) := by apply sup_le_sup_left (sup_le_sup_right
      (inf_le_right) (b ∩ c))
    _ ≤ (b ∩ c) ∪ ((a \ b) ∪ (a \ c)) := by rw [sup_assoc, sup_comm]
  apply (lemma_1_iii a (b ∩ c) _).mp at h2a
  exact h2a
  apply le_antisymm h2 h1

---COROLLARY 1 (i)
lemma corollary_1_i (a b c : α) : a \ b ≤ (a ∩ c) \ b ∪ a \ c :=
  calc
    a \ b = (a ∩ c ∪ (a \ c)) \ b := by nth_rw 1 [lemma_1_v a c]

```

```

    _ = (a  $\sqcap$  c)  $\setminus$  b  $\sqcup$  (a  $\setminus$  c)  $\setminus$  b := by rw [DualHeytingAlgebra.E4]
    _  $\leq$  (a  $\sqcap$  c)  $\setminus$  b  $\sqcup$  (a  $\setminus$  c) := by apply sup_le_sup_left (lemma_1_i (a  $\setminus$  c)
b)

---COROLLARY 1 (ii)
lemma corollary_1_ii (a b c :  $\alpha$ ) : a  $\setminus$  b  $\leq$  a  $\setminus$  c  $\sqcup$  c  $\setminus$  b :=
calc
  a  $\setminus$  b  $\leq$  (a  $\sqcap$  c)  $\setminus$  b  $\sqcup$  a  $\setminus$  c := by apply corollary_1_i
  _  $\leq$  c  $\setminus$  b  $\sqcup$  a  $\setminus$  c := by apply sup_le_sup_right (lemma_1_ii (a  $\sqcap$  c) c b
inf_le_right)
  _ = a  $\setminus$  c  $\sqcup$  c  $\setminus$  b := by rw [sup_comm]

---COROLLARY 1 (iii)
lemma corollary_1_iii (a b :  $\alpha$ ) : (a  $\leq$  b)  $\leftrightarrow$  (a  $\setminus$  b =  $\perp$ ) :=
calc
  (a  $\leq$  b)  $\leftrightarrow$  (a  $\leq$  b  $\sqcup$   $\perp$ ) := by rw [sup_bot_eq]
  _  $\leftrightarrow$  (a  $\setminus$  b  $\leq$   $\perp$ ) := by apply lemma_1_iii
  _  $\leftrightarrow$  (a  $\setminus$  b =  $\perp$ ) := by apply le_bot_iff

---LEMMA 1 (vi) and (viii) need (E5)
variable [EventAlgebra  $\beta$ ] {a b c d :  $\beta$ }

---LEMMA 1 (vi)
lemma lemma_1_vi (a b c :  $\beta$ ) : a  $\setminus$  c  $\sqcap$  b  $\setminus$  c = (a  $\sqcap$  b)  $\setminus$  c := by
have h1 : a  $\setminus$  c  $\sqcap$  b  $\setminus$  c  $\leq$  (a  $\sqcap$  b)  $\setminus$  c
calc
  a  $\setminus$  c  $\sqcap$  b  $\setminus$  c = ((a  $\sqcap$  b)  $\sqcup$  (a  $\setminus$  b))  $\setminus$  c  $\sqcap$  b  $\setminus$  c := by nth_rw 1 [lemma_1_v a b]
  _ = ((a  $\sqcap$  b)  $\sqcup$  (a  $\setminus$  b))  $\setminus$  c  $\sqcap$  ((b  $\sqcap$  a)  $\sqcup$  (b  $\setminus$  a))  $\setminus$  c := by nth_rw 3
    [lemma_1_v b a]
  _ = ((a  $\sqcap$  b)  $\setminus$  c  $\sqcup$  (a  $\setminus$  b)  $\setminus$  c)  $\sqcap$  ((b  $\sqcap$  a)  $\sqcup$  (b  $\setminus$  a))  $\setminus$  c := by rw
    [DualHeytingAlgebra.E4]
  _ = ((a  $\sqcap$  b)  $\setminus$  c  $\sqcup$  (a  $\setminus$  b)  $\setminus$  c)  $\sqcap$  ((b  $\sqcap$  a)  $\setminus$  c  $\sqcup$  (b  $\setminus$  a)  $\setminus$  c) := by rw
    [DualHeytingAlgebra.E4]
  _ = (a  $\sqcap$  b)  $\setminus$  c  $\sqcup$  ((a  $\setminus$  b)  $\setminus$  c  $\sqcap$  (b  $\setminus$  a)  $\setminus$  c) := by rw [inf_comm a
b, sup_inf_left]
  _  $\leq$  (a  $\sqcap$  b)  $\setminus$  c  $\sqcup$  ((a  $\setminus$  b)  $\sqcap$  (b  $\setminus$  a)  $\setminus$  c) := by apply sup_le_sup_left
    (inf_le_inf_right _ (lemma_1_i (a  $\setminus$  b) c))
  _  $\leq$  (a  $\sqcap$  b)  $\setminus$  c  $\sqcup$  ((a  $\setminus$  b)  $\sqcap$  (b  $\setminus$  a)) := by apply sup_le_sup_left
    (inf_le_inf_left _ (lemma_1_i (b  $\setminus$  a) c))
  _ = (a  $\sqcap$  b)  $\setminus$  c  $\sqcup$   $\perp$  := by rw [EventAlgebra.E5]
  _ = (a  $\sqcap$  b)  $\setminus$  c := by rw [sup_bot_eq]
have h2 : (a  $\sqcap$  b)  $\setminus$  c  $\leq$  a  $\setminus$  c  $\sqcap$  b  $\setminus$  c
calc
  (a  $\sqcap$  b)  $\setminus$  c = (a  $\sqcap$  b)  $\setminus$  c  $\sqcap$  (a  $\sqcap$  b)  $\setminus$  c := by rw [inf_idem]
  _  $\leq$  a  $\setminus$  c  $\sqcap$  (a  $\sqcap$  b)  $\setminus$  c := by apply inf_le_inf_right _ (lemma_1_ii (a  $\sqcap$  b) a c
inf_le_left)
  _  $\leq$  a  $\setminus$  c  $\sqcap$  b  $\setminus$  c := by apply inf_le_inf_left _ (lemma_1_ii (a  $\sqcap$  b) b c
inf_le_right)

```

```
apply le_antisymm h1 h2
```

```
---LEMMA 1 (viii)
```

```
lemma lemma_1_viii (a b c :  $\beta$ ) :  $a \setminus (b \sqcup c) = a \setminus b \sqcap a \setminus c$  := by
```

```
have h1 :  $a \setminus b \sqcap a \setminus c \leq a \setminus (b \sqcup c)$ 
```

```
calc
```

```

 $a \setminus b \sqcap a \setminus c \leq (a \sqcup c) \setminus b \sqcap a \setminus c$  := by apply inf_le_inf_right _ (lemma_1_ii
  a (a  $\sqcup$  c) b le_sup_left)
 $\leq (a \sqcup c) \setminus b \sqcap (a \sqcup b) \setminus c$  := by apply inf_le_inf_left _ (lemma_1_ii a (a  $\sqcup$ 
  b) c le_sup_left)
 $= (a \setminus c \sqcup c) \setminus b \sqcap (a \setminus b \sqcup b) \setminus c$  := by rw [DualHeytingAlgebra.E3 a
  c, DualHeytingAlgebra.E3 a b]
 $= ((a \setminus c) \setminus b \sqcup c \setminus b) \sqcap ((a \setminus b) \setminus c \sqcup b \setminus c)$  := by
  rw [DualHeytingAlgebra.E4, DualHeytingAlgebra.E4]
 $= (a \setminus (c \sqcup b) \sqcup c \setminus b) \sqcap (a \setminus (b \sqcup c) \sqcup b \setminus c)$  := by rw [lemma_11, lemma_11]
 $= a \setminus (c \sqcup b) \sqcup (c \setminus b \sqcap b \setminus c)$  := by rw [sup_comm b c, sup_inf_left]
 $= a \setminus (c \sqcup b) \sqcup \perp$  := by rw [EventAlgebra.E5]
 $= a \setminus (b \sqcup c)$  := by rw [sup_bot_eq, sup_comm]
```

```
have h2 :  $a \setminus (b \sqcup c) \leq a \setminus b \sqcap a \setminus c$  :=
```

```
calc
```

```

 $a \setminus (b \sqcup c) = a \setminus (b \sqcup c) \sqcap a \setminus (b \sqcup c)$  := by rw [inf_idem]
 $\leq a \setminus b \sqcap a \setminus (b \sqcup c)$  := by apply inf_le_inf_right _ (lemma_1_iv b (b  $\sqcup$  c) a
  le_sup_left)
 $\leq a \setminus b \sqcap a \setminus c$  := by apply inf_le_inf_left _ (lemma_1_iv c (b  $\sqcup$  c) a
  le_sup_right)
```

```
apply le_antisymm h2 h1
```

```
---LEMMA 12
```

```
lemma lemma_12
```

```
(hyp :  $\forall (a b c : \alpha), (a \sqcap b) \setminus c = (a \setminus c) \sqcap (b \setminus c)$ )  $\vee \forall (a b c : \alpha), a \setminus (b$ 
```

```
 $\sqcup c) = (a \setminus b) \sqcap (a \setminus c)$ ) :
```

```
 $\forall a b : \alpha, a \setminus b \sqcap (b \setminus a) = \perp$  := by
```

```
intro a b
```

```
rcases hyp with hyp1 | hyp2
```

```
calc
```

```

 $a \setminus b \sqcap (b \setminus a) = \perp (a \setminus a \setminus b) \sqcap \perp (a \setminus b \setminus a)$  := by rw [bot_sup_eq,
  bot_sup_eq]
 $= (a \setminus a \sqcup a \setminus b) \sqcap \perp (a \setminus b \setminus a)$  := by rw [DualHeytingAlgebra.E1]
 $= (a \setminus a \sqcup a \setminus b) \sqcap (b \setminus b \sqcup b \setminus a)$  := by rw  $\leftarrow$  [DualHeytingAlgebra.E1]
 $= (a \setminus (a \sqcap b)) \sqcap (b \setminus (b \sqcap a))$  := by rw [lemma_1_vii, lemma_1_vii]
 $= (a \setminus (a \sqcap b)) \sqcap (b \setminus (a \sqcap b))$  := by rw [inf_comm b a]
 $= ((a \sqcap b) \setminus (a \sqcap b))$  := by rw [hyp1]
 $= \perp$  := by rw [DualHeytingAlgebra.E1]
```

```
calc
```

```

 $a \setminus b \sqcap (b \setminus a) = \perp (a \setminus a \setminus b) \sqcap \perp (a \setminus b \setminus a)$  := by rw [bot_sup_eq,
  bot_sup_eq]
 $= (b \setminus b \sqcup a \setminus b) \sqcap \perp (a \setminus b \setminus a)$  := by nth_rw 1  $\leftarrow$  [DualHeytingAlgebra.E1]
 $= (b \setminus b \sqcup a \setminus b) \sqcap (a \setminus a \sqcup b \setminus a)$  := by nth_rw 1  $\leftarrow$  [
  DualHeytingAlgebra.E1]
```

```

_ = ((b ∪ a) \ b) ∩ ((a ∪ b) \ a) := by
rw[DualHeytingAlgebra.E4,DualHeytingAlgebra.E4]
_ = ((b ∪ a) \ b) ∩ ((b ∪ a) \ a) := by rw[sup_comm a b]
_ = ((b ∪ a) \ (b ∪ a)) := by rw[hypr]
_ = ⊥ := by rw[DualHeytingAlgebra.E1]

---mathlib-style construction of lemma 12
def EventAlgebra.ofJ10rJ2 [h : DualHeytingAlgebra α]
  (hyp : ∀( a b c : α, (a ∩ b) \ c = (a \ c) ∩ (b \ c))
    ∨ ∀( a b c : α, a \ (b ∪ c) = (a \ b) ∩ (a \ c))) :
  EventAlgebra α :=
{ h with
  E5 := by apply lemma_12 hyp
}

---CHAPTER IV
--- This lemma allows the change between two sups and one sup over pairs
lemma finset_sup_product {X Y : Type*} α{ : Type*} [SemilatticeSup α] [OrderBot
α]
  (sX : Finset X) (sY : Finset Y) (f : X → Y → α) :
  (sX ×s sY).sup (fun p => f p.1 p.2) = sX.sup (fun x => sY.sup (f x)) := by
rw [Finset.sup_product_left]

---Helper lemma
lemma bot_without (a : α) : ⊥ \ a = ⊥ := by
  apply le_bot_iff.mp
  apply (lemma_1_iii ⊥ a ⊥).mp
  rw [sup_bot_eq]
  apply bot_le

--- This is a finite join version of (E4)
lemma finset_sup_sdiff {X : Type*} α{ : Type*} [DualHeytingAlgebra α]
  (s : Finset X) (f : X → α) (b : α) :
  s.sup f \ b = s.sup (fun x => f x \ b) := by
  refine s.cons_induction ?_ ?_
  · simp [bot_without]
  · intro a s' _ ih
    rw [Finset.sup_cons, Finset.sup_cons, DualHeytingAlgebra.E4, ih]

---LEMMA 15
lemma lemma_15 {X : Type*} [BoundedOrder α]
  (s : Finset X) (A B : X → α) :
  s.sup (fun x => A x ∪ B x) ≤ s.sup A ∪ s.sup B := by
  apply Finset.sup_le
  intro x hx
  exact sup_le_sup (Finset.le_sup hx) (Finset.le_sup hx)

---LEMMA 3 (transitivity of authentication)
lemma lemma_3 [DualHeytingAlgebra δ] {X : Type*} (s : Finset X) (E F G : X → δ) :
```

```

s.sup (fun x => E x \ G x) ≤ s.sup (fun x => E x \ F x) ∪ s.sup (fun x => F
x \ G x) :=
calc s.sup (fun x => E x \ G x)
  ≤ s.sup (fun x => E x \ F x ∪ (F x \ G x)) := by apply
Finset.sup_mono_fun; intro x _; exact corollary_1_ii (E x) (G x) (F x)
_ ≤ s.sup (fun x => E x \ F x) ∪ s.sup (fun x => F x \ G x) := by apply
lemma_15

---LEMMA 4
lemma lemma_4 [DualHeytingAlgebra δ] {X Y : Type*} (sX : Finset X) (sY : Finset
Y) (E F : X → δ) (G H : X → Y → δ) :
sX.sup (fun x => E x \ F x) ≤
sX.sup (fun x => E x \ sY.sup (G x)) ∪
(sX ×s sY).sup (fun p => G p.1 p.2 \ sY.sup (H p.1)) ∪
(sX ×s sY).sup (fun p => H p.1 p.2 \ F p.1) := by
---first proof block
have h1 : sX.sup (fun x => E x \ F x) ≤ sX.sup (fun x => (E x \ sY.sup (G x)))
  ∪ sX.sup (fun x => ((sY.sup (G x)) \ F x))
calc
sX.sup (fun x => E x \ F x) ≤ sX.sup (fun x => E x \ sY.sup (G x) ∪ (sY.sup
(G x) \ F x)) := by
  apply Finset.sup_mono_fun; intro x _; exact corollary_1_ii (E x) (F x)
(sY.sup (G x))
_ ≤ sX.sup (fun x => E x \ sY.sup (G x)) ∪ sX.sup (fun x => sY.sup (G x) \ F
x) := by apply lemma_15
have h1a : sX.sup (fun x => ((sY.sup (G x)) \ F x)) = (sX ×s sY).sup (fun p =>
(G p.1 p.2 \ F p.1))
calc
sX.sup (fun x => sY.sup (G x) \ F x)
= sX.sup (fun x => sY.sup (fun y => G x y \ F x)) := by apply
Finset.sup_congr rfl; intro x _; exact finset_sup_sdiff sY (G x) (F x)
_ = (sX ×s sY).sup (fun p => (G p.1 p.2 \ F p.1)) := by rw-[
finset_sup_product]
rw [h1a] at h1
---second proof block
have h2 : (sX ×s sY).sup (fun p => (G p.1 p.2 \ F p.1)) ≤ (sX ×s sY).sup (fun
p => (G p.1 p.2 \ sY.sup (H p.1))) ∪ (sX ×s sY).sup (fun p => (sY.sup (H
p.1) \ F p.1))
calc
(sX ×s sY).sup (fun p => (G p.1 p.2 \ F p.1))
  ≤ (sX ×s sY).sup (fun p => (G p.1 p.2 \ sY.sup (H p.1) ∪ sY.sup (H p.1) \
F p.1)) := by apply Finset.sup_mono_fun; intro p _; exact corollary_1_ii (G
p.1 p.2) (F p.1) (sY.sup (H p.1))
_ ≤ (sX ×s sY).sup (fun p => (G p.1 p.2 \ sY.sup (H p.1))) ∪ (sX ×s sY).sup
(fun p => (sY.sup (H p.1) \ F p.1)) := by apply lemma_15
---The following eliminates the unused variable:
have h2a : (sX ×s sY).sup (fun p => (sY.sup (H p.1) \ F p.1)) ≤ (sX ×s sY).sup
(fun p => (H p.1 p.2 \ F p.1)) := by
calc
(sX ×s sY).sup (fun p => sY.sup (H p.1) \ F p.1)

```



```

≤ sX.sup (fun x => sY.sup (H x) \ F x) := by
  apply Finset.sup_le
  intro p hp
  exact Finset.le_sup (f := fun x => sY.sup (H x) \ F x)
(Finset.mem_product.mp hp).1
_ = sX.sup (fun x => sY.sup (fun y => H x y \ F x)) := by
  apply Finset.sup_congr rfl
  intro x _
  exact finset_sup_sdiff sY (H x) (F x)
_ = (sX ×s sY).sup (fun p => H p.1 p.2 \ F p.1) := by
  rw ←[ finset_sup_product]
---Combine the two proof blocks:
have h3 : (sX ×s sY).sup (fun p => (G p.1 p.2 \ F p.1)) ≤ (sX ×s sY).sup (fun
  p => (G p.1 p.2 \ sY.sup (H p.1))) ∪ (sX ×s sY).sup (fun p => (H p.1 p.2 \
  F p.1)) := by
  apply le_trans h2 (sup_le_sup_left h2a ((sX ×s sY).sup (fun p => (G p.1 p.2
    \ sY.sup (H p.1)))))
rw [sup_assoc]
exact le_trans h1 (sup_le_sup_left h3 _)

```

Listing 1.1: Lean code that for the formal verification of [Lemma 4](#).